Presentation of the course and setting up R

MATH 2740 – Mathematics of Data Science – Unit U01

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

Foreword

- "Numerical dates" are in the form YYYY-MM-DD (e.g, these slides were compiled on 2026-10-14)
- Times are 24h (HHMM)
- Units are SI

In case you want to know, the slides in the course are `Rnw` compiled using `R`. All (including source code) are available on GitHub here

Getting in touch

- julien.arino@umanitoba.ca
- Please use your `myumanitoba` email address. Use a tag such as `[MATH 2740]` in your subject line, if you want to be read..
- Word of warning: I am bad with email! I do answer questions after class or during office hours

Office hours

- Because of the ongoing renovation of Machray Hall, I am sharing an office with 8 colleagues, connected to another office shared by 8 colleagues
- $\Longrightarrow$ it is **impossible** to see you in my office
- I have booked 316 St Paul's from 1400 to 1800 on Tuesday for office hours
- Seeing you outside of office hours is impossible without ample warning, since I actually need to book a room

Course website - UMLearn

- All information about the course is posted on UMLearn
- It is **your responsibility** to check the UMLearn site regularly: *Announcements* is how I normally communicate with you about the course
- (Remember to hit the link at the top of the page that says *MATH-2740-A01 - Mathematics of Data Science*, sometimes UMLearn takes you directly to Content, which is not where Announcements are)

Important – I **DO NOT** answer emails that ask about things easy to find on UMLearn

Lectures

- TR 1130–1245 in 100 Fletcher Argue

- Videos for the course as I taught it in 2021 are available as a YouTube playlist. There is no guarantee that that the content will be the same this year, but there will be commonalities for sure

Tutorials

Section	Day and time	Location
B01	W 0830–0920	P230 Duff Roblin
B03	W 0930–1020	303 Tier
B04	W 1130–1220	P230 Duff Roblin

- In tutorials, you will review some of the mathematical content and work on computations
- Three (3) Lab Tests (45 minutes) will be held during tutorials (dates later in the slides)

Work in labs

- The lecture notes contain problems that will be worked on in labs (when not in a Lab Test)
- During the tutorial, the TA will go over some of the problems
- A dedicated solutions manual will be posted on UMLearn
- Solutions will only be posted after the corresponding tutorial period has ended

R worksheets

- Working on R coding is important in this course, although because of LLMs, it is not realistic anymore to use assignments to test this capacity
- Worksheets will be posted on UMLearn at the start of each week
- Solutions will be posted on UMLearn the next week
- They are meant as practice and will not be graded
- Lab Tests, the Midterm and Final Examinations will all contain *one* question asking you to explain what a piece of R code does

Evaluation

- 3 Lab Tests during tutorials (**10%** each for a total **30%**)
- 1 midterm on Thursday 5 November 2026 1800–2000 (**30%**)
- 1 final examination during the December 2026 Examination period (**40%**)
- In all written work, explain what you are doing. No explanation $\implies$ lost marks (even if correct)

Lab Tests

- Held during tutorials on 23 Sept, 21 Oct, 25 Nov
- 45 minutes long
- You **must** write Lab Tests in the tutorial you are registered in
- Tests your capacity to remember definitions or results, do simple proofs, carry out computations and explain R code

Absences from scheduled evaluations

- If you miss a lab test or the midterm examination, you can either use a self-declaration form or provide proper documentation (MD's note)
- Self-declaration **must be made within 48 hours** of the event you are using it for
- They are intended for occasional and unforeseen circumstances, I **will not accept more than one during the term**
- You **must** use the form on this page to self-declare your absence

What if you missed an evaluation?

If you have proper documentation (self-declaration or MD's note), then

- Lab Tests: the weight of the missed test is shifted to the final examination
- Midterm Examination: you will write a Deferred Midterm Examination (date set by Department of Mathematics)

Miss more than two (2) evaluations (Lab Tests or Midterm) $\Longrightarrow$ **F in the course**

Unhappy about a mark

If you realise that some points were forgotten (this happens) or feel that you have been marked unfairly, feel free to reach **me** about it, preferably in person after class or during office hours

Do not bother the TAs with this, they do not have the authority to change your marks

- I will only consider requests made within seven (7) calendar days from the date you receive your mark
- When processing requests, I **remark the entire examination** $\Longrightarrow$ your overall mark could increase, stay the same **or even decrease**

Colour coding in slides & lecture notes

Memorising the definitions is part of the course. To help, definitions are colour coded

Definition 1 (Important definitions)

These definitions are important, you need to know them

Definition 2 (Definitions)

These definitions are a little less important, you will not be asked to state them (although it is a good idea to know them anyway)

In lecture notes, colour coding is only for important stuff

Results are colour coded

Memorising some of the results is part of the course. To help, results are colour coded

Theorem 3 (Theorems)

Theorems in blue boxes are worth knowing but you will not be asked to reproduce them

Theorem 4 (Important theorems)

Theorems in red boxes are important, you should know them and be able to reproduce them

You must know how to do some proofs

There are a few proofs (not many!) that I want you to know how to do

Such proofs appear on slides like the present one, with a red background

BECOME A VOLUNTEER NOTETAKER

Why?

- Help reduce barriers for your peers
- Contribute to creating an inclusive University of Manitoba
- Receive recognition on your Experience Record

How?

- Login and upload your notes directly at https://sasclockwork.cc.umanitoba.ca/ClockWork/
- Submit notes in your preferred style (typed or handwritten)
- It only takes an extra 15 minutes weekly
- Full instructions on to the Student Accessibility Services website

https://umanitoba.ca/student-supports/accessibility#volunteer-note-sharing

Student Accessibility Services | UM

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

In days of yore (circa 2010)

> *Big data is like teenage sex: everyone talks about it, nobody really knows how to do it, everyone thinks everyone else is doing it, so everyone claims they are doing it...*

Attributed to Dan Ariely (Duke University)

The vocabulary has evolved, big data $\rightarrow$ complex data $\rightarrow$ data science, but Data Science remains a loosely defined concept, although things are becoming better

Data Science (according to Wikipedia)

Data science is an **interdisciplinary field** that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from **structured** and **unstructured data**, and apply knowledge and actionable insights from data across a broad range of application domains.

[..] It uses techniques and theories drawn from many fields within the context of **mathematics**, **statistics**, **computer science**, **information science**, and **domain knowledge**.

The data deluge

► Data science is nothing new (some statisticians argue it is just another name for statistics), but it has become prominent in recent years as a consequence of the unprecedented mass of information generated and collected by our modern societies

► One speaks of *information explosion* or *data deluge*. See some considerations, e.g., here

A wide variety of jobs

We have absolutely insane amounts of data and we try to make sense of it

$\Longrightarrow$ data science

However, except for the name, the situation has not improved significantly since the days of yore of Ariely's quote: data science is a hodge-podge that contains everything but the kitchen sink

To caricature

- two main types of data: structured and unstructured
- two main branches: statistics and computer science
- two main types of jobs: users and developpers

Math of Data Science?

- DS has two main branches: statistics and computer science
- DS has two main types of jobs: users and developpers

So why a course on Math of Data Science?

If you plan to be a user and are not curious about *the how* and *the why* and can tolerate errors due to misuse of methods, then you probably don't care about this course

In other cases, many of the concepts used have their roots in math and to understand where the methods are coming from and, even more importantly, to develop new methods, math is often required

Warning! We barely brush the surface

- Some techniques from linear algebra
- Some graph theory ideas
- A little bit of multivariable calculus

There is a lot more to see!!!

Prerequisites / What you will learn

► (MATH 1210 or MATH 1220 or MATH 1300) and (MATH 1232 or MATH 1700 or MATH 1710)

$\Rightarrow$ You **must know and be comfortable** with 1st year linear algebra (3 CH) and 1st year calculus (6 CH)

► We need more: some stuff you would learn in 2090 (Linear Algebra 2), some stuff from 2130, 2150 or 2720 (Multivariable Calculus) and some stuff from 2070 (Graph Theory)

Focus here is not on mathematical “precision”

- We won’t do complicated proofs. I will show some when they are useful in understanding *why* something works

- We will cover just enough of the mentioned math topics that you can understand *how* to do things

- In classic math courses, we work on “small” examples so we can work them by hand and are able to do them in tests

Here, we will do small examples by hand, indeed. But you will do regularly sized examples in computer worksheets

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

We will be programming quite a bit

As already indicated, Data Science is a very hands-on discipline. We will be programming quite a bit in this course

Indeed, we can work out some of the examples "by hand", but to make things interesting, we typically need to consider larger examples where hand calculations are not pleasant or not even feasible

R versus Python

Slightly different take on life :)

In short: `Python` is more CS, `R` is more Stats/Math

Both are good languages for data science

In this course, we use `R`

R was originally for stats but is now more

- Open source version of S
- Appeared in 1993
- Now version 4.6
- Uses a lot of C and Fortran code. E.g., `deSolve`:

  The functions provide an interface to the FORTRAN functions 'lsoda', 'lsodar', 'lsode', 'lsodes' of the 'ODEPACK' collection, to the FORTRAN functions 'dvode', 'zvode' and 'daspk' and a C-implementation of solvers of the 'Runge-Kutta' family with fixed or variable time steps

- Very active community on the web, easy to find solutions

Getting your computer ready for the course

All computer coding is in R

$\Longrightarrow$ you need to find a way to run R. Next slide: some methods, from the easiest to the most challenging.

Note that all options described below are Open Source (completely free)

In short...

- Terminal version, not very friendly
- Nicer terminal: radian
- Execute R scripts by using `Rscript name_of_script.R`. Useful to run code in `cron`, for instance
- Use IDEs:
  - RStudio has become the reference
  - RKWard is useful if you are for instance using an ARM processor (Raspberry Pi, some Chromebooks..)
- Integrate into jupyter notebooks

Install R and RStudio

This is probably the best option if you intend to go a little further than what we will do in the course. `R` is available on most platforms, while `RStudio` is available on most platforms except for Linux ARM devices (but can be compiled there)

Visit https://www.r-project.org/

Choose your version: Windows or Mac. Under Linux, you can install directly from your package manager (e.g., `sudo apt install R-base` for Debian-based distros)

To install RStudio, see here

Going further

- RStudio server: run RStudio on a Linux server and connect via a web interface
- Shiny: easily create an interactive web site running R code
- Shiny server: run Shiny apps on a Linux server
- Rmarkdown: markdown that incorporates R commands. Useful for generating reports in html or pdf, can make slides as well..
- Sweave/knitr: LaTeX incorporating R commands. Useful for generating reports. Not used as much as Rmarkdown these days (but used for these slides)

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

R Markdown?

- File format for making dynamic documents with R
- Combines code, its results and narrative text in a single document
- Uses simple `Markdown` syntax for text and `R` code chunks for analysis
- From one '.Rmd' file, you can create HTML, PDF, Word documents, presentations, ...

The YAML header

Every R Markdown document starts with a YAML header, enclosed by `---`. This controls the overall properties of the document

Example YAML for a PDF document

```
---
title: "My Report"
author: "My Name"
date: "2025-08-17"
output:
  pdf_document:
    toc: true
    number_sections: true
---
```

Basic Text Formatting

Markdown provides a simple syntax for formatting text

- `# Header` creates a section title, `## Sub-header` creates a subsection title, etc.
- `*italic text*` produces *italic text*
- `**bold text**` produces **bold text**
- `` `code font` `` produces `code font`

Lists

Unordered lists start with * or -

```
* Item 1
* Item 2
```

Ordered lists use numbers

```
1. First Item
2. Second Item
```

(Once the numbered list is “initiated”, the number doesn’t matter, you can write 1., 1., 1., etc., provided you have a new line each time)

Chunks: the core of R Markdown

R code is placed in “chunks”, which start with ```` ```{r chunk-name} ```` and end with ```` ``` ````

````
```{r cars-summary}
# Your R code goes here
summary(cars)
```
````

Chunk names (`cars-summary` here) are not mandatory (you could write ‘‘‘`{r}` but are very useful, in particular when debugging

Chunk output

When the document is "knit," the R code is executed and its output is embedded in the final document

````
```{r cars-summary}
summary(cars)
```
````

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Controlling chunks with options

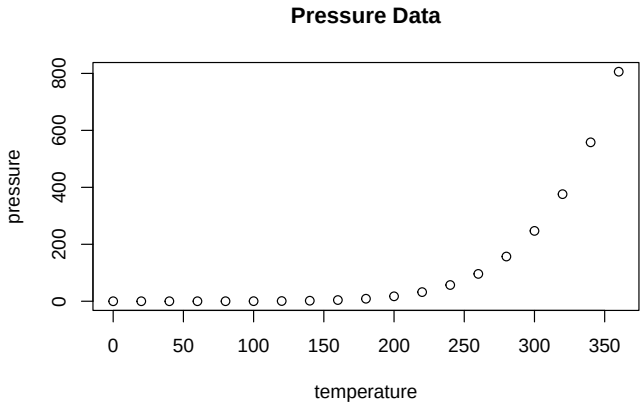
Chunk behavior is controlled by options inside the curly braces

- `echo=FALSE` hides the `R` code, but shows the output
- `eval=FALSE` shows the code, but does not execute it
- `include=FALSE` executes the code, but hides both code and output
- `warning=FALSE`, `message=FALSE` hides warnings or messages
- `fig.width=5`, `fig.height=4` sets figure dimensions

Figure chunks

Plots are automatically embedded

```
```{r pressure-plot, echo=FALSE, fig.cap="Pressure Data"}
plot(pressure)
```
```



Inline R Code

You can embed R code directly into text with backticks: `` `r ...` ``

```
The `cars` dataset has `r nrow(cars)` rows.
```

The cars dataset has 50 rows.

General information about the course

Mathematics of data science?

Setting things up for the course

Introduction to R Markdown

(Super brief) Introduction to programming in R

R is a scripted language

- Interactive
- Allows you to work in real time
  - Be careful: what is in memory might involve steps not written down in a script
  - If you want to reproduce your steps, it is good to write all the steps down in a script and to test from time to time running using `Rscript`: this will ensure that all that is required to run is indeed loaded to memory when it needs to, i.e., that it is not already there..

Assignment

Two ways:

```
X <- 10
```

or

```
X = 10
```

First version is preferred by R purists.. I don't really care

Lists

A very useful data structure, quite flexible and versatile. Empty list: `L <- list()`. Convenient for things like parameters. For instance

```
L <- list()
L$a <- 10
L$b <- 3
L[["another_name"]] <- "Plouf plouf"
```

```r
L[1]
```

```
## $a
## [1] 10
```

```r
L[[2]]
```

```
## [1] 3
```

```r
L$a
```

```
## [1] 10
```

```r
L[["b"]]
```

```
## [1] 3
```

```r
L$another_name
```

```
## [1] "Plouf plouf"
```

Vectors

```
x = 1:10
y <- c(x, 12)
y
z = c("red", "blue")
z
z = c(z, 1)
z
```

Note that in z, since the first two entries are characters, the added entry is also a character. Contrary to lists, vectors have all entries of the same type

Matrices

Matrix (or vector) of zeros

```
A <- mat.or.vec(nr = 2, nc = 3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    0    0    0
## [2,]    0    0    0
```

Matrix are column-first

```
B <- matrix(c(1,2,3,4), nr = 2, nc = 2)
B
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
C <- matrix(c(1,2,3,4), nr = 2, nc = 2, byrow = TRUE)
C
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

Remark that here and elsewhere, naming the arguments (e.g., `nr = 2`) allows to use arguments in any order

Matrix operations

Probably the biggest annoyance in R compared to other languages

- The notation `A*B` is the *Hadamard product* $A \circ B$ (what would be denoted `A.*B` in matlab), not the standard matrix multiplication
- Matrix multiplication is written `A %*% B`

Vector operations

Vector addition is also frustrating. Say you create the vector

```
x = 1:10
x

##  [1]  1  2  3  4  5  6  7  8  9 10
```

Then `x+1` gives

```
x+1

##  [1]  2  3  4  5  6  7  8  9 10 11
```

i.e., adds 1 to all entries in the vector. Beware of this in particular when addressing sets of indices in lists, vectors or matrices

Flow control

```
if (condition_is_true) {
  list of stuff to do
}
```

Even if `list of stuff to do` is a single instruction, best to use curly braces

```
if (condition_is_true) {
  list of stuff to do
} else if (another_condition) {
  ...
} else {
  ...
}
```

For loops

`for` applies to lists or vectors

```
for (i in 1:10) {
  something using integer i
}
for (j in c(1,3,4)) {
  something using integer j
}
for (n in c("truc", "muche", "chose")) {
  something using string n
}
for (m in list("truc", "muche", "chose", 1, 2)) {
  something using string n or integer n, depending
}
```

lapply

Very useful function (a few others in the same spirit: `sapply`, `vapply`, `mapply`) Applies a function to each entry in a list/vector/matrix. Because there is a parallel version (`parLapply`) that we will see later, worth learning

```
l = list()
for (i in 1:10) {
        l[[i]] = runif(i)
}
lapply(X = l, FUN = mean)
```

or, to make a vector

```
unlist(lapply(X = l, FUN = mean))
```

or

```
sapply(X = l, FUN = mean)
```

“Advanced” lapply

Can “pick up” nontrivial list entries

```
l = list()
for (i in 1:10) {
        l[[i]] = list()
        l[[i]]$a = runif(i)
        l[[i]]$b = runif(2*i)
}
sapply(X = l, FUN = function(x) length(x$b))
```

gives

```
# [1]  2  4  6  8 10 12 14 16 18 20
```

Just recall: the argument to the function you define is a list entry (`l[[1]]`, `l[[2]]`, etc., here)

Avoid parameter variation loops with expand.grid

```
# Suppose we want to vary 3 parameters
variations = list(
    p1 = seq(1, 10, length.out = 10),
    p2 = seq(0, 1, length.out = 10),
    p3 = seq(-1, 1, length.out = 10)
)

# Create the list
tmp = expand.grid(variations)
PARAMS = list()
for (i in 1:dim(tmp)[1]) {
    PARAMS[[i]] = list()
    for (k in 1:length(variations)) {
        PARAMS[[i]][[names(variations)[k]]] = tmp[i, k]
    }
}
```

Can I have this wrapped up to go?

For each slide set including `R` code, we generate an `R` file in the `CODE` directory with all the code chunks in this `Rnw` file and no LaTeX

Source the file (if you are reading `SLIDES/filename.pdf`, then you want `CODE/filename.R`) to reproduce all results in the slide set without generating a pdf

Some small changes might be required (for instance, in the file for this lecture, quite a lot is commented out)

Math prelims – Linear algebra

MATH 2740 – Mathematics of Data Science – Unit U02

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

In MATH 2740, we rely on notions you acquired in MATH 1210/1220/1300. We also use some material from first-year calculus

So let us (briefly) go over material in these courses

I also add (for some of you) a few things that will be handy and establish some terminology that we use throughout the course

Outline

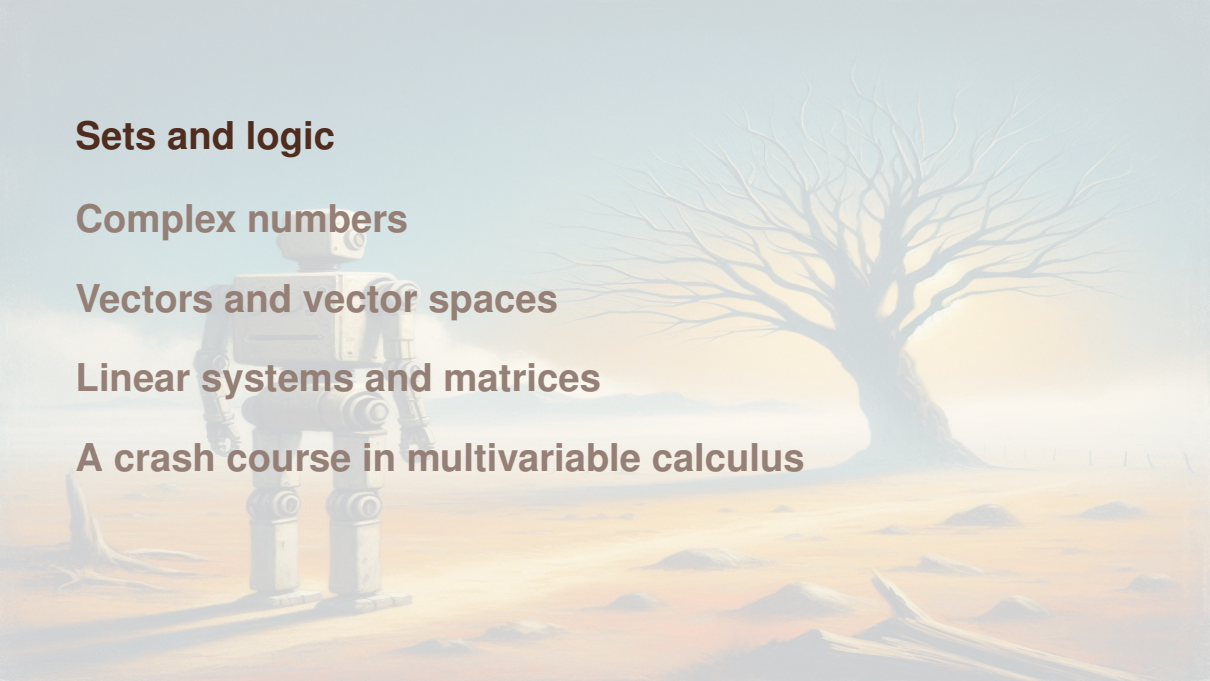
Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus


Sets and logic
Complex numbers
Vectors and vector spaces
Linear systems and matrices
A crash course in multivariable calculus

Sets and elements

Definition 1 (Set)

A **set** X is a collection of **elements**

We write $x \in X$ or $x \notin X$ to indicate that the element x belongs to the set X or does not belong to the set X, respectively

Definition 2 (Subset)

Let X be a set. The set S is a **subset** of X, which is denoted $S \subset X$, if all its elements belong to X

Not used here but worth noting: we say S is a **proper subset** of X and write $S \subsetneq X$, if it is a subset of X and not equal to X

Quantifiers

A shorthand notation for “for all elements x belonging to X” is $\forall x \in X$

For example, if $X = \mathbb{R}$, the *field* of real numbers, then $\forall x \in \mathbb{R}$ means “for all real numbers x”

A shorthand notation for “there exists an element x in the set X” is $\exists x \in X$

$\forall$ and $\exists$ are **quantifiers**

Intersection and union of sets

Let X and Y be two sets

Definition 3 (Intersection)

The intersection of X and Y, $X \cap Y$, is the set of elements that belong to X ***and*** *to Y,*

$$X \cap Y = \{x : x \in X \textbf{ and } x \in Y\}$$

Definition 4 (Union)

The union of X and Y, $X \cup Y$, is the set of elements that belong to X ***or*** *to Y,*

$$X \cup Y = \{x : x \in X \textbf{ or } x \in Y\}$$

In mathematics, or=and/or in common parlance. We also have an **exclusive or** (xor)

A teeny bit of logic

In a logical sense, a **proposition** is an assertion (or statement) whose truth value (true or false) can be asserted. For example, a theorem is a proposition that has been shown to be true. “The sky is blue” is also a proposition

Let A be a proposition. We generally write

$$A$$

to mean that A is true, and

$$\textbf{not } A$$

to mean that A is false. **not** A is the **negation** of A (or **not** A is the negative of A)

A teeny bit of logic (cont.)

Let A, B be propositions. Then

- $A \Rightarrow B$ (read A implies B) means that whenever A is true, then so is B
- $A \Leftrightarrow B$, also denoted A if and only if B (A iff B for short), means that $A \Rightarrow B$ **and** $B \Rightarrow A$
  We also say that A and B are **equivalent**

Let A and B be propositions. Then

$$(A \Rightarrow B) \Leftrightarrow (\textbf{not } B \Rightarrow \textbf{not } A)$$

Necessary or sufficient conditions

Suppose we want to establish whether a given statement P is true, depending on the truth value of a statement H. Then we say that

- H is a **necessary condition** if $P \Rightarrow H$
  (It is necessary that H be true for P to be true; so whenever P is true, so is H)

- H is a **sufficient condition** if $H \Rightarrow P$
  (It suffices for H to be true for P to also be true)

- H is a **necessary and sufficient condition** if $H \Leftrightarrow P$, i.e., H and P are equivalent

Playing with quantifiers

For the quantifiers $\forall$ (for all) and $\exists$ (there exists),

$\exists$ is the negation of $\forall$

Therefore, for example, the negation of

$$\forall x \in X, \exists y \in Y$$

is

$$\exists x \in X, \forall y \in Y$$

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Complex numbers

Definition 5 (Complex numbers)

*A **complex number** is an ordered pair (a, b), where $a, b \in \mathbb{R}$. Usually written $a + ib$ or $a + bi$, where $i^2 = -1$ (i.e., $i = \sqrt{-1}$)*
The set of all complex numbers is denoted $\mathbb{C}$,

$$\mathbb{C} = \{a + ib : a, b \in \mathbb{R}\}$$

Definition 6 (Addition and multiplication on $\mathbb{C}$)

Letting $a + ib$ and $c + id \in \mathbb{C}$, addition on $\mathbb{C}$ is defined by

$$(a + ib) + (c + id) = (a + c) + i(b + d)$$

and multiplication on $\mathbb{C}$ is defined by

$$(a + ib)(c + id) = (ac - bd) + i(ad + bc)$$

Latter is easy to obtain using regular multiplication and $i^2 = -1$

Properties

$\forall \alpha, \beta, \gamma \in \mathbb{C}$,

$\alpha + \beta = \beta + \alpha$ and $\alpha\beta = \beta\alpha$ **[commutativity]**

$(\alpha + \beta) + \gamma = \alpha + (\beta + \gamma)$ and $(\alpha\beta)\gamma = \alpha(\beta\gamma)$ **[associativity]**

$\gamma + 0 = \gamma$ and $\gamma 1 = \gamma$ **[identities]**

$\forall \alpha \in \mathbb{C}$, $\exists \beta \in \mathbb{C}$ unique s.t. $\alpha + \beta = 0$ **[additive inverse]**

$\forall \alpha \neq 0 \in \mathbb{C}$, $\exists \beta \in \mathbb{C}$ unique s.t. $\alpha\beta = 1$ **[multiplicative inverse]**

$\gamma(\alpha + \beta) = \gamma\alpha + \gamma\beta$ **[distributivity]**

Additive & multiplicative inverse, subtraction, division

Definition 7

Let $\alpha, \beta \in \mathbb{C}$

- $-\alpha$ is the **additive inverse** of α, i.e., the unique number in $\mathbb{C}$ s.t. $\alpha + (-\alpha) = 0$
- **Subtraction** on $\mathbb{C}$:
$$\beta - \alpha = \beta + (-\alpha)$$
- For $\alpha \neq 0$, $1/\alpha$ is the **multiplicative inverse** of α, i.e., the unique number in $\mathbb{C}$ s.t.
$$\alpha(1/\alpha) = 1$$
- **Division** on $\mathbb{C}$:
$$\beta/\alpha = \beta(1/\alpha)$$

Definition 8 (Real and imaginary parts)

*Let $z = a + ib$. Then $Re\ z = a$ is **real part** and $Im\ z = b$ is **imaginary part** of z*

If ambiguous, write $\mathrm{Re}\,(z)$ and $\mathrm{Im}\,(z)$

Definition 9 (Conjugate and Modulus)

Let $z = a + ib \in \mathbb{C}$. Then

- ***Complex conjugate** of z is*

$$\bar{z} = a - ib$$

- ***Modulus** (or **absolute value**) of z is*

$$|z| = \sqrt{a^2 + b^2} \geq 0$$

Properties of complex numbers

Let $w, z \in \mathbb{C}$, then

- $z + \bar{z} = 2\mathrm{Re}\, z$
- $z - \bar{z} = 2i\mathrm{Im}\, z$
- $z\bar{z} = |z|^2$
- $\overline{w+z} = \bar{w} + \bar{z}$ and $\overline{wz} = \bar{w}\bar{z}$
- $\bar{\bar{z}} = z$
- $|\mathrm{Re}\, z| \leq |z|$ and $|\mathrm{Im}\, z| \leq |z|$
- $|\bar{z}| = |z|$
- $|wz| = |w|\,|z|$
- $|w+z| \leq |w| + |z|$ **[triangle inequality]**

Solving quadratic equations

Consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. Letting

$$\Delta = a_1^2 - 4 a_0 a_2$$

you know that if $\Delta > 0$, then

$$P(x) = 0$$

has two distinct *real* solutions,

$$x_1 = \frac{-a_1 - \sqrt{\Delta}}{2a_2} \quad \text{and} \quad x_2 = \frac{-a_1 + \sqrt{\Delta}}{2a_2}$$

if $\Delta = 0$, then there is a (multiplicity 2) unique *real* solution

$$x_1 = \frac{-a_1}{2a_2}$$

while if $\Delta < 0$, there is no solution

Solving quadratic equations with complex numbers

Consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. If instead of seeking $x \in \mathbb{R}$, we seek $x \in \mathbb{C}$, then the situation is the same, except when $\Delta < 0$

In the latter case, note that

$$\sqrt{\Delta} = \sqrt{(-1)(-\Delta)} = \sqrt{-1}\sqrt{-\Delta} = i\sqrt{-\Delta}$$

Since $\Delta < 0$, $-\Delta > 0$ and the square root is the usual one

Solving quadratic equations with complex numbers

To summarize, consider the polynomial

$$P(x) = a_0 + a_1 x + a_2 x^2$$

where $x, a_0, a_1, a_2 \in \mathbb{R}$. Letting

$$\Delta = a_1^2 - 4a_0 a_2$$

Then

$$P(x) = 0$$

has two solutions,

$$x_{1,2} = \frac{-a_1 \pm \sqrt{\Delta}}{2a_2}$$

where, if $\Delta < 0$, $x_1, x_2 \in \mathbb{C}$ and take the form

$$x_{1,2} = \frac{-a_1 \pm i\sqrt{-\Delta}}{2a_2}$$

Why this matters

Recall (we will come back to this later) that to find the *eigenvalues* of the matrix

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

we seek λ solutions to $\det(A - \lambda\mathbb{I}) = 0$, i.e., λ solutions to

$$|A - \lambda\mathbb{I}| = \begin{vmatrix} a_{11} - \lambda & a_{12} \\ a_{21} & a_{22} - \lambda \end{vmatrix} = (a_{11} - \lambda)(a_{22} - \lambda) - a_{12}a_{21} = 0$$

i.e., λ solutions to

$$\lambda^2 - (a_{11} + a_{22})\lambda + a_{11}a_{22} - a_{12}a_{21} = 0$$

Why this matters (cont.)

Let

$$P(\lambda) = \lambda^2 - (a_{11} + a_{22})\lambda + a_{11}a_{22} - a_{12}a_{21}$$

From previous discussion, letting

$$\begin{aligned} \Delta &= (a_{11} + a_{22})^2 - 4(a_{11}a_{22} - a_{12}a_{21}) \\ &= a_{11}^2 + a_{22}^2 + 2a_{11}a_{22} - 4a_{11}a_{22} + 4a_{12}a_{21} \\ &= a_{11}^2 + a_{22}^2 - 2a_{11}a_{22} + 4a_{12}a_{21} \\ &= (a_{11} - a_{22})^2 + 4a_{12}a_{21} \end{aligned}$$

we have two (potentially equal) solutions to $P(\lambda) = 0$

$$x_{1,2} = \frac{a_{11} + a_{22} \pm \sqrt{\Delta}}{2}$$

that are complex if $\Delta < 0$

Example: $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Vectors

A **vector** $\boldsymbol{v}$ is an ordered n-tuple of real or complex numbers

Denote $\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$ (real or complex numbers). For $v_1, \ldots, v_n \in \mathbb{F}$,

$$\boldsymbol{v} = (v_1, \ldots, v_n) \in \mathbb{F}^n$$

is a vector. $v_1, \ldots, v_n$ are the **components** of $\boldsymbol{v}$

If unambiguous, we write v. Otherwise, $\boldsymbol{v}$ or $\vec{v}$

Vector space

Definition 10 (Vector space)

A **vector space** over $\mathbb{F}$ is a set V together with two binary operations, **vector addition**, denoted $+$, and **scalar multiplication**, that satisfy the relations:

1. $\forall \boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \in V$, $\boldsymbol{u} + (\boldsymbol{v} + \boldsymbol{w}) = (\boldsymbol{u} + \boldsymbol{v}) + \boldsymbol{w}$
2. $\forall \boldsymbol{v}, \boldsymbol{w} \in V$, $\boldsymbol{v} + \boldsymbol{w} = \boldsymbol{w} + \boldsymbol{v}$
3. $\exists \mathbf{0} \in V$, the zero vector, such that $\boldsymbol{v} + \mathbf{0} = \boldsymbol{v}$ for all $\boldsymbol{v} \in V$
4. $\forall \boldsymbol{v} \in V$, there exists an element $\boldsymbol{w} \in V$, the additive inverse of $\boldsymbol{v}$, such that $\boldsymbol{v} + \boldsymbol{w} = \mathbf{0}$
5. $\forall \alpha \in \mathbb{R}$ and $\forall \boldsymbol{v}, \boldsymbol{w} \in V$, $\alpha(\boldsymbol{v} + \boldsymbol{w}) = \alpha\boldsymbol{v} + \alpha\boldsymbol{w}$
6. $\forall \alpha, \beta \in \mathbb{R}$ and $\forall \boldsymbol{v} \in V$, $(\alpha + \beta)\boldsymbol{v} = \alpha\boldsymbol{v} + \beta\boldsymbol{v}$
7. $\forall \alpha, \beta \in \mathbb{R}$ and $\forall \boldsymbol{v} \in V$, $\alpha(\beta\boldsymbol{v}) = (\alpha\beta)\boldsymbol{v}$
8. $\forall \boldsymbol{v} \in V$, $1\boldsymbol{v} = \boldsymbol{v}$

Subspaces

Definition 11 (Subspace)

Let V be a vector space over $\mathbb{F}$. A non-empty subset W of V is a **subspace** of V if W is itself a vector space under the same operations of addition and scalar multiplication

Definition 12 (Subspace – Practical definition)

Let V be a vector space over $\mathbb{F}$. A non-empty subset W of V is a subspace if

1. *for all $\boldsymbol{u}, \boldsymbol{v} \in W$, $\boldsymbol{u} + \boldsymbol{v} \in W$ (W is closed under vector addition)*
2. *for all $\alpha \in \mathbb{F}$ and $\boldsymbol{v} \in W$, $\alpha \boldsymbol{v} \in W$ (W is closed under scalar multiplication)*

Norms

Definition 13 (Norm)

Let V be a vector space over $\mathbb{F}$, and $\boldsymbol{v} \in V$ be a vector. The ***norm*** *of $\boldsymbol{v}$, denoted $\|\boldsymbol{v}\|$, is a function from V to $\mathbb{R}_+$ that has the following properties:*

1. *For all $\boldsymbol{v} \in V$, $\|\boldsymbol{v}\| \geq 0$ with $\|\boldsymbol{v}\| = 0$ iff $\boldsymbol{v} = \boldsymbol{0}$*
2. *For all $\alpha \in \mathbb{F}$ and all $\boldsymbol{v} \in V$, $\|\alpha \boldsymbol{v}\| = |\alpha| \, \|\boldsymbol{v}\|$*
3. *For all $\boldsymbol{u}, \boldsymbol{v} \in V$, $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$*

Let V be a vector space (for example, $\mathbb{R}^2$ or $\mathbb{R}^3$)

The **zero element** (or **zero vector**) is the vector $\mathbf{0} = (0, \ldots, 0)$

The **additive inverse** of $\boldsymbol{v} = (v_1, \ldots, v_n)$ is $-\boldsymbol{v} = (-v_1, \ldots, -v_n)$

For $\boldsymbol{v} = (v_1, \ldots, v_n) \in V$, the length (or Euclidean norm) of $\boldsymbol{v}$ is the **scalar**

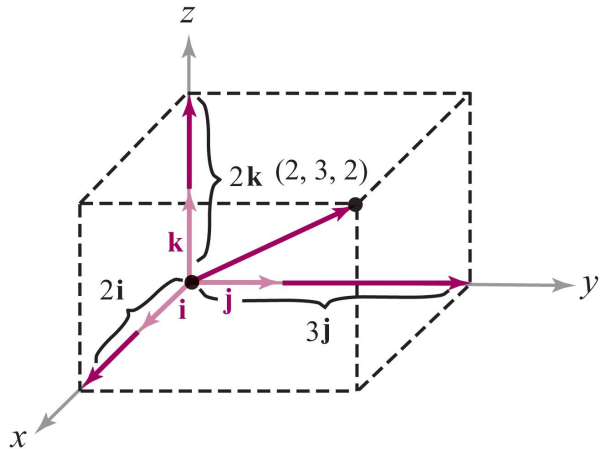
$$\|\boldsymbol{v}\| = \sqrt{v_1^2 + \cdots + v_n^2}$$

To **normalize** the vector $\boldsymbol{v}$ consists in considering $\tilde{\boldsymbol{v}} = \boldsymbol{v}/\|\boldsymbol{v}\|$, i.e., the vector in the same direction as $\boldsymbol{v}$ that has unit length

Standard basis vectors

Vectors $\boldsymbol{i} = (1,0,0)$, $\boldsymbol{j} = (0,1,0)$ and $\boldsymbol{k} = (0,0,1)$ are the **standard basis vectors** of $\mathbb{R}^3$. A vector $\boldsymbol{v} = (v_1, v_2, v_3)$ can then be written

$$\boldsymbol{v} = v_1\boldsymbol{i} + v_2\boldsymbol{j} + v_3\boldsymbol{k}$$



For V ($\mathbb{R}^n$), the standard basis vectors are usually denoted $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n$, with

$$\boldsymbol{e}_k = (\underbrace{0, \ldots, 0}_{k-1}, 1, \underbrace{0, \ldots, 0}_{n-k+1})$$

Dot product

Definition 14 (Dot product)

Let $\boldsymbol{a} = (a_1, \ldots, a_n) \in \mathbb{R}^n$, $\boldsymbol{b} = (b_1, \ldots, b_n) \in \mathbb{R}^n$. *The* ***dot product*** *of* $\boldsymbol{a}$ *and* $\boldsymbol{b}$ *is the* ***scalar***

$$\boldsymbol{a} \bullet \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + \cdots + a_n b_n$$

The dot product is a special case of **inner product**

Properties of the dot product

Theorem 15

For $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \in \mathbb{R}^n$ *and* $\alpha \in \mathbb{R}$,

- $\boldsymbol{a} \bullet \boldsymbol{a} = \|\boldsymbol{a}\|^2$ *(so* $\boldsymbol{a} \bullet \boldsymbol{a} \geq 0$*, with* $\boldsymbol{a} \bullet \boldsymbol{a} = 0$ *iff* $\boldsymbol{a} = \boldsymbol{0}$*)*
- $\boldsymbol{a} \bullet \boldsymbol{b} = \boldsymbol{b} \bullet \boldsymbol{a}$ *(*$\bullet$ *is commutative)*
- $\boldsymbol{a} \bullet (\boldsymbol{b} + \boldsymbol{c}) = \boldsymbol{a} \bullet \boldsymbol{b} + \boldsymbol{a} \bullet \boldsymbol{c}$ *(*$\bullet$ *distributive over* $+$*)*
- $(\alpha \boldsymbol{a}) \bullet \boldsymbol{b} = \alpha(\boldsymbol{a} \bullet \boldsymbol{b}) = \boldsymbol{a} \bullet (\alpha \boldsymbol{b})$
- $\boldsymbol{0} \bullet \boldsymbol{a} = 0$

Some results stemming from the dot product

Theorem 16

*If θ is the angle between the vectors **a** and **b**, then*

$$\boldsymbol{a} \bullet \boldsymbol{b} = \|\boldsymbol{a}\| \, \|\boldsymbol{b}\| \, \cos\theta$$

Corollary 17 (Cauchy-Schwarz inequality)

*For any two vectors **a** and **b**, we have*

$$|\boldsymbol{a} \bullet \boldsymbol{b}| \leq \|\boldsymbol{a}\| \, \|\boldsymbol{b}\|$$

*with equality if and only if **a** is a scalar multiple of **b**, or one of them is **0**.*

Theorem 18

***a** and **b** are orthogonal if and only if $\boldsymbol{a} \bullet \boldsymbol{b} = 0$.*

Scalar and vector projections

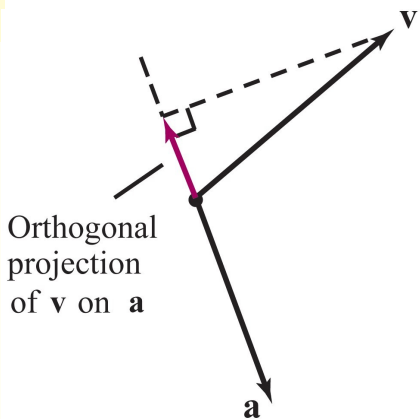
Definition 19

Let $\boldsymbol{a}$, $\boldsymbol{v}$ *be vectors. The* **scalar projection** *of* $\boldsymbol{v}$ *onto* $\boldsymbol{a}$ *(or* **component** *of* $\boldsymbol{v}$ *along* $\boldsymbol{a}$*) is*

$$\text{comp}_{\boldsymbol{a}}\boldsymbol{v} = \frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|}$$

The **vector projection** *(or* **orthogonal projection***) of* $\boldsymbol{v}$ *onto* $\boldsymbol{a}$ *is*

$$\text{proj}_{\boldsymbol{a}}\boldsymbol{v} = \left(\frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|}\right)\frac{\boldsymbol{a}}{\|\boldsymbol{a}\|} = \frac{\boldsymbol{a} \bullet \boldsymbol{v}}{\|\boldsymbol{a}\|^2}\boldsymbol{a}$$



Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

Linear systems and matrices

Linear systems and matrices
Matrix arithmetic
Eigenpairs
Similarity and diagonalisation
Linear independence/Bases/Dimension

Linear systems

Definition 20 (Linear system)

A **linear system** of m equations in n unknowns takes the form

$$\begin{array}{ccccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2n}x_n & = & b_2 \\ \vdots & & \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \cdots & + & a_{mn}x_n & = & b_n \end{array} \tag{1}$$

The a_{ij}, x_j and b_j could be in $\mathbb{R}$ or $\mathbb{C}$, although here we typically assume they are in $\mathbb{R}$

The aim is to find $x_1, x_2, \ldots, x_n$ that satisfy all equations simultaneously

Theorem 21 (Nature of solutions to a linear system)

A linear system can have

- *no solution*
- *a unique solution*
- *infinitely many solutions*

Operations on linear systems

You learned to manipulate linear systems using

- Gaussian elimination
- Gauss-Jordan elimination

with the aim to put the system in **row echelon form** (REF) or **reduced row echelon form** (RREF)

Matrices and linear systems

Writing

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

where A is an $m \times n$ **matrix**, $\boldsymbol{x}$ and $\boldsymbol{b}$ are n (column) **vectors** (or $n \times 1$ matrices), then the linear system in the previous slide takes the form

$$A\boldsymbol{x} = \boldsymbol{b}$$

Notation for vectors

We usually assume vectors are column vectors and thus write, e.g.,

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = (x_1, x_2, \ldots, x_n)^T$$

Here, ^{T} is the **transpose operator** (more on this soon)

Consider the system

$$A\boldsymbol{x} = \boldsymbol{b}$$

If $\boldsymbol{b} = \mathbf{0}$, the system is **homogeneous** and always has the solution $\boldsymbol{x} = 0$ and so the “no solution” option in Theorem 21 goes away

Linear systems and matrices

Linear systems and matrices

Matrix arithmetic

Eigenpairs

Similarity and diagonalisation

Linear independence/Bases/Dimension

Definition 22 (Matrix)

An m-by-n or $m \times n$ matrix is a rectangular array of elements of $\mathbb{R}$ or $\mathbb{C}$ with m rows and n columns,

$$A = [a_{ij}] = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}$$

We always list indices as “row,column”

We denote $\mathcal{M}_{mn}(\mathbb{F})$ or $\mathbb{F}^{mn}$ the set of $m \times n$ matrices with entries in $\mathbb{F} = \{\mathbb{R}, \mathbb{C}\}$. Often, we omit $\mathbb{F}$ in $\mathcal{M}_{mn}$ if the nature of $\mathbb{F}$ is not important

When $m = n$, we usually write $\mathcal{M}_n$

Basic matrix arithmetic

Let $A \in \mathcal{M}_{mn}, B \in \mathcal{M}_{mn}$ be matrices (of the same size) and $c \in \mathbb{F} = \{\mathbb{R}, \mathbb{C}\}$ be a scalar

- **Scalar multiplication**
$$cA = [ca_{ij}]$$
- **Addition**
$$A + B = [a_{ij} + b_{ij}]$$
- **Subtraction** (addition of $-B = (-1)B$ to A)
$$A - B = A + (-1)B = [a_{ij} + (-1)b_{ij}] = [a_{ij} - b_{ij}]$$
- **Transposition** of A gives a matrix $A^T = \mathcal{M}_{nm}$ with
$$A^T = [a_{ji}], \quad j = 1, \ldots, n, \quad i = 1, \ldots, m$$

Matrix multiplication

The (matrix) **product** of A and B, AB, requires the "inner dimensions" to match, i.e., the number of columns in A must equal the number of rows in B

Suppose that is the case, i.e., let $A \in \mathcal{M}_{mn}$, $B \in \mathcal{M}_{np}$. Then the i, j entry in $C := AB$ takes the form

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

Recall that the matrix product is not commutative, i.e., in general, $AB \neq BA$ (when both those products are defined, i.e., when $A, B \in \mathcal{M}_n$)

Special matrices

Definition 23 (Zero and identity matrices)

*The **zero** matrix is the matrix 0_{mn} whose entries are all zero. The **identity** matrix is a square $n \times n$ matrix $\mathbb{I}_n$ with all entries on the main diagonal equal to one and all off diagonal entries equal to zero*

Definition 24 (Symmetric matrix)

*A square matrix $A \in \mathcal{M}_n$ is **symmetric** if $\forall i, j = 1, \ldots, n$, $a_{ij} = a_{ji}$. In other words, $A \in \mathcal{M}_n$ is symmetric if $A = A^T$*

Making symmetric matrices

Theorem 25

1. *If $A \in \mathcal{M}_n$, then $A + A^T$ is symmetric*
2. *If $A \in \mathcal{M}_{mn}$, then $AA^T \in \mathcal{M}_m$ and $A^TA \in \mathcal{M}_n$ are symmetric*

Proof of Theorem 25

X symmetric $\iff X = X^T$, so use $X =$ the matrix whose symmetric property you want to check

1. True if $A + A^T = (A + A^T)^T$. We have

$$(A + A^T)^T = A^T + (A^T)^T = A^T + A = A + A^T$$

2. AA^T symmetric if $AA^T = (AA^T)^T$. We have

$$(AA^T)^T = (A^T)^T A^T = AA^T$$

$A^T A$ works similarly

Determinants

Definition 26 (Determinant)

Let $A \in \mathcal{M}_n$ with $n \geq 2$. The **determinant** of A is the *scalar*

$$\det(A) = |A| = \sum_{j=1}^{n} a_{ij} C_{ij}$$

where $C_{ij} = (-1)^{i+j} \det(A_{ij})$ is the (i, j)-**cofactor** of A and A_{ij} is the submatrix of A from which the ith row and jth column have been removed

This is a cofactor expansion along the ith row
This is a recursive formula: it gives result in terms of n $\mathcal{M}_{n-1}$ matrices, to which it must in turn be applied, all the way down to

$$\det \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = a_{11} a_{22} - a_{12} a_{21}$$

Two special matrices and their determinants

Definition 27

*$A \in \mathcal{M}_n$ is **upper triangular** if $a_{ij} = 0$ when $i > j$, **lower triangular** if $a_{ij} = 0$ when $j > i$, **triangular** if it is either upper or lower triangular and **diagonal** if it is both upper and lower triangular*

When A diagonal, we often write $A = \text{diag}(a_{11}, a_{22}, \ldots, a_{nn})$

Theorem 28

Let $A \in \mathcal{M}_n$ be triangular or diagonal. Then

$$\det(A) = \prod_{i=1}^{n} a_{ii} = a_{11}a_{22}\cdots a_{nn}$$

Inversion/Singularity

Definition 29 (Matrix inverse)

*$A \in \mathcal{M}_n$ is **invertible** (or **nonsingular**) if $\exists A^{-1} \in \mathcal{M}_n$ s.t.*

$$AA^{-1} = A^{-1}A = \mathbb{I}$$

*A^{-1} is the **inverse** of A. If A^{-1} does not exist, A is **singular***

If A is invertible, its inverse A^{-1} is unique

Linear algebra in a nutshell

Theorem 30

Let $A \in \mathcal{M}_n$. The following statements are equivalent (TFAE):

1. *The matrix A is invertible (or nonsingular)*
2. *$\forall \boldsymbol{b} \in \mathbb{F}^n$, $A\boldsymbol{x} = \boldsymbol{b}$ has a unique solution ($\boldsymbol{x} = A^{-1}\boldsymbol{b}$)*
3. *The only solution to $A\boldsymbol{x} = \mathbf{0}$ is the trivial solution $\boldsymbol{x} = \mathbf{0}$*
4. *$RREF(A) = \mathbb{I}_n$*
5. *The matrix A is equal to a product of elementary matrices*
6. *$\forall \boldsymbol{b} \in \mathbb{F}^n$, $A\boldsymbol{x} = \boldsymbol{b}$ has a solution*
7. *There is a matrix $B \in \mathcal{M}_n$ such that $AB = \mathbb{I}_n$*
8. *There is an invertible matrix $B \in \mathcal{M}_n$ such that $AB = \mathbb{I}_n$*
9. $\det(A) \neq 0$
10. *0 is not an eigenvalue of A*

Revisiting matrix arithmetic

With addition, subtraction, scalar multiplication, multiplication, transposition and inversion, you can perform arithmetic on matrices essentially as on scalar, if you bear in mind a few rules

- The sizes have to be compatible
- The order is important since matrix multiplication is not commutative
- Transposition and inversion change the order of products:

$$(AB)^T = B^T A^T \text{ and } (AB)^{-1} = B^{-1} A^{-1}$$

Linear systems and matrices

Linear systems and matrices
Matrix arithmetic
Eigenpairs
Similarity and diagonalisation
Linear independence/Bases/Dimension

Eigenvalues / Eigenvectors / Eigenpairs

Definition 31

*Let $A \in \mathcal{M}_n$. A vector $\boldsymbol{x} \in \mathbb{F}^n$ such that $\boldsymbol{x} \neq \mathbf{0}$ is an **eigenvector** of A if $\exists \lambda \in \mathbb{F}$ called an **eigenvalue**, s.t.*

$$A\boldsymbol{x} = \lambda \boldsymbol{x}$$

*A couple $(\lambda, \boldsymbol{x})$ with $\boldsymbol{x} \neq \mathbf{0}$ s.t. $A\boldsymbol{x} = \lambda \boldsymbol{x}$ is an **eigenpair***

Parallel eigenvectors

Theorem 32

If $(\lambda, \boldsymbol{x})$ eigenpair, then for $c \neq 0$, $(\lambda, c\boldsymbol{x})$ also eigenpair

Proof.

Assume $(\lambda, \boldsymbol{x})$ is an eigenpair of $A \in \mathcal{M}_n$, i.e., $\boldsymbol{x} \neq \mathbf{0}$ and $A\boldsymbol{x} = \lambda\boldsymbol{x}$

Take the nonzero scalar $c \in \mathbb{F}$. Then $A(c\boldsymbol{x}) = cA\boldsymbol{x}$ because scalars move freely in vector/matrix products. Therefore $A(c\boldsymbol{x}) = cA\boldsymbol{x} = c\lambda\boldsymbol{x}$ since $A\boldsymbol{x} = \lambda\boldsymbol{x}$

So finally, moving c again, $A(c\boldsymbol{x}) = \lambda(c\boldsymbol{x})$, i.e., $c\boldsymbol{x}$ is an eigenvector of A associated to λ □

Linear systems and matrices

- Linear systems and matrices
- Matrix arithmetic
- Eigenpairs
- **Similarity and diagonalisation**
- Linear independence/Bases/Dimension

Similarity

Definition 33 (Similarity)

*$A, B \in \mathcal{M}_n$ are **similar** ($A \sim B$) if $\exists P \in \mathcal{M}_n$ invertible s.t.*

$$P^{-1}AP = B$$

Theorem 34 ($\sim$ is an equivalence relation)

$A, B, C \in \mathcal{M}_n$, then

- $A \sim A$ *($\sim$ **reflexive**)*
- $A \sim B \implies B \sim A$ *($\sim$ **symmetric**)*
- $A \sim B$ and $B \sim C \implies A \sim C$ *($\sim$ **transitive**)*

Similarity (cont.)

Theorem 35

$A, B \in \mathcal{M}_n$ with $A \sim B$. Then

- $\det A = \det B$
- *A invertible* $\iff$ *B invertible*
- *A and B have the same eigenvalues*

Diagonalisation

Definition 36 (Diagonalisability)

*$A \in \mathcal{M}_n$ is **diagonalisable** if $\exists D \in \mathcal{M}_n$ diagonal s.t. $A \sim D$*

In other words, $A \in \mathcal{M}_n$ is diagonalisable if there exists a diagonal matrix $D \in \mathcal{M}_n$ and a nonsingular matrix $P \in \mathcal{M}_n$ s.t. $P^{-1}AP = D$

Could of course write $PAP^{-1} = D$ since P invertible, but $P^{-1}AP$ makes more sense for computations

Theorem 37

$A \in \mathcal{M}_n$ diagonalisable $\iff$ A has n linearly independent eigenvectors

Corollary 38 (Sufficient condition for diagonalisability)

$A \in \mathcal{M}_n$ has all its eigenvalues distinct $\implies$ A diagonalisable

For $P^{-1}AP = D$: in P, put the linearly independent eigenvectors as columns and in D, the corresponding eigenvalues

Linear systems and matrices

- Linear systems and matrices
- Matrix arithmetic
- Eigenpairs
- Similarity and diagonalisation
- **Linear independence/Bases/Dimension**

Linear combination and span

Definition 39 (Linear combination)

Let V be a vector space. A ***linear combination*** *of a set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ of vectors in V is a vector*

$$c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k$$

where $c_1, \ldots, c_k \in \mathbb{F}$

Definition 40 (Span)

The set of all linear combinations of a set of vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ is the ***span*** *of $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$,*

$$\operatorname{span}(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k) = \{c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k : c_1, \ldots, c_k \in \mathbb{F}\}$$

Finite/infinite-dimensional vector spaces

Theorem 41

The span of a set of vectors in V is the smallest subspace of V containing all the vectors in the set

Definition 42 (Set of vectors spanning a space)

If $\text{span}(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k) = V$, we say $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ **spans** V

Definition 43 (Dimension of a vector space)

A vector space V is **finite-dimensional** if some set of vectors in it spans V. A vector space V is **infinite-dimensional** if it is not finite-dimensional

Linear (in)dependence

Definition 44 (Linear independence/Linear dependence)

A set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ *of vectors in a vector space* V *is* ***linearly independent*** *if*

$$(c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k = 0) \Leftrightarrow (c_1 = \cdots = c_k = 0),$$

where $c_1, \ldots, c_k \in \mathbb{F}$*. A set of vectors is* ***linearly dependent*** *if it is not linearly independent.*

If linearly dependent, assume w.l.o.g. that $c_1 \neq 0$, then

$$\boldsymbol{v}_1 = -\frac{c_2}{c_1}\boldsymbol{v}_2 - \cdots - \frac{c_k}{c_1}\boldsymbol{v}_k$$

i.e., $\boldsymbol{v}_1$ is a linear combination of the other vectors in the set

Theorem 45

Let V be a finite-dimensional vector space. Then the ***cardinal*** *(number of elements) of every linearly independent set of vectors is less than or equal to the number of elements in every spanning set of vectors*

E.g., in $\mathbb{R}^3$, a set with 4 or more vectors is automatically linearly dependent

Basis

Definition 46 (Basis)

Let V be a vector space. A ***basis*** *of V is a set of vectors in V that is both linearly independent and spanning*

Theorem 47 (Criterion for a basis)

A set $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ of vectors in a vector space V is a basis of $V \iff \forall \boldsymbol{v} \in V$, $\boldsymbol{v}$ can be written uniquely in the form

$$\boldsymbol{v} = c_1 \boldsymbol{v}_1 + \cdots + c_k \boldsymbol{v}_k,$$

where $c_1, \ldots, c_k \in \mathbb{F}$

More on bases

Theorem 48 (Basis of finite-dimensional vector space)

Every finite-dimensional vector space has a basis

Theorem 49

Any two bases of a finite-dimensional vector space have the same number of vectors

Definition 50 (Dimension)

The ***dimension*** $\dim V$ *of a finite-dimensional vector space* V *is the number of vectors in any basis of the vector space*

Theorem 51 (Dimension of a subspace)

Let V *be a finite-dimensional vector space and* $U \subset V$ *be a subspace of* V*. Then* $\dim U \leq \dim V$

Constructing bases

Theorem 52

Let V be a finite-dimensional vector space. Then every linearly independent set of vectors in V with $\dim V$ *elements is a basis of V*

Theorem 53

Let V be a finite-dimensional vector space. Then every spanning set of vectors in V with $\dim V$ *elements is a basis of V*

Sets and logic

Complex numbers

Vectors and vector spaces

Linear systems and matrices

A crash course in multivariable calculus

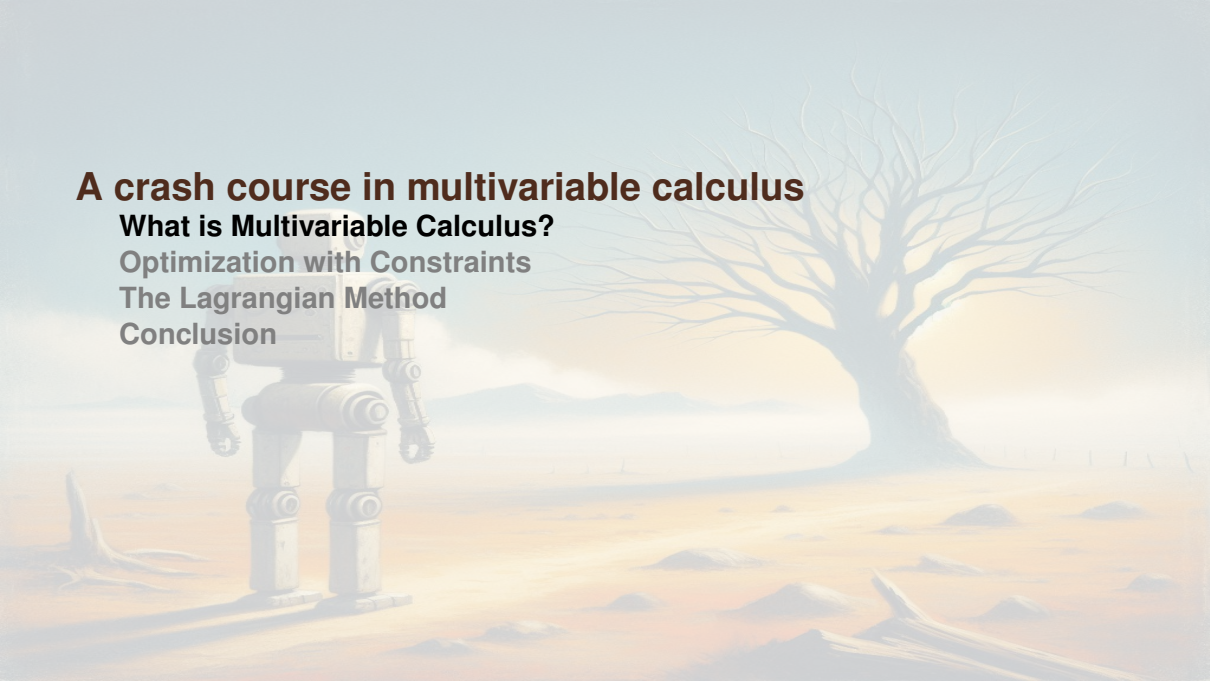
A crash course in multivariable calculus

What is Multivariable Calculus?

Optimization with Constraints

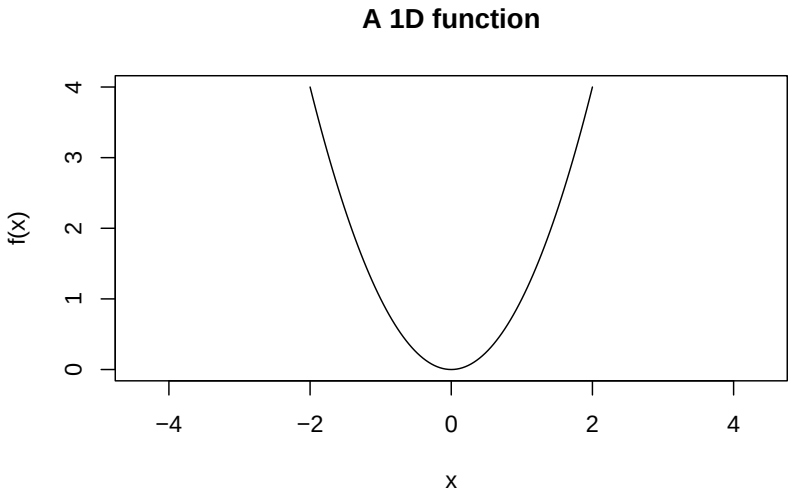
The Lagrangian Method

Conclusion



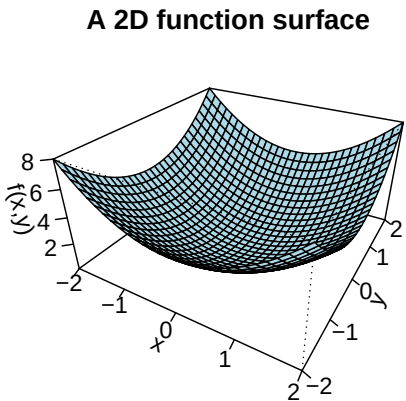
One dimension

MATH 1500 & 1700 deal with functions of one variable, like $f(x) = x^2$



Multivariable calculus

Multivariable calculus extends this to functions of two or more variables, like $f(x, y) = x^2 + y^2$



Partial derivatives

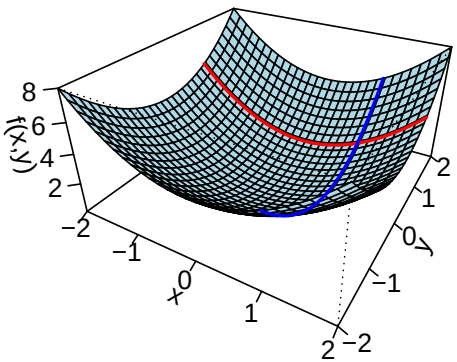
How do we measure the "slope" on a 3D surface?

Definition 54 (Partial derivative)

Let $f : \mathbb{R}^n \to \mathbb{R}$ be a function of variables $x_1, \ldots, x_n$. The ***partial derivative*** *of f with respect to x_i, denoted $\dfrac{\partial f}{\partial x_i}$, is the derivative of f with respect to x_i while treating all other variables as constants*

- $\dfrac{\partial f}{\partial x}$ measures height change as we move only in the x direction. Treat y as a constant
- $\dfrac{\partial f}{\partial y}$ measures height change as we move only in the y direction. Treat x as a constant

Partial derivatives



The Steepest path: the gradient

Definition 55 (Gradient)

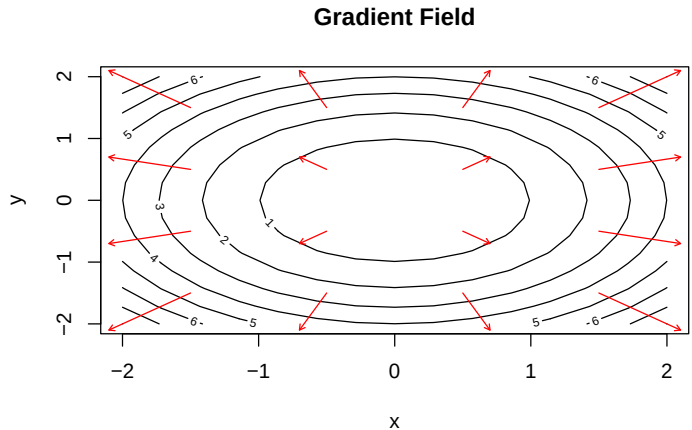
*The **gradient** of a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$, denoted ∇f, is the vector-valued function composed of all its first-order partial derivatives:*

$$\nabla f(x_1, \ldots, x_n) = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

What does it tell us?

- **Direction:** it points in the direction of the *steepest ascent*
- **Magnitude:** its length represents the steepness of that ascent

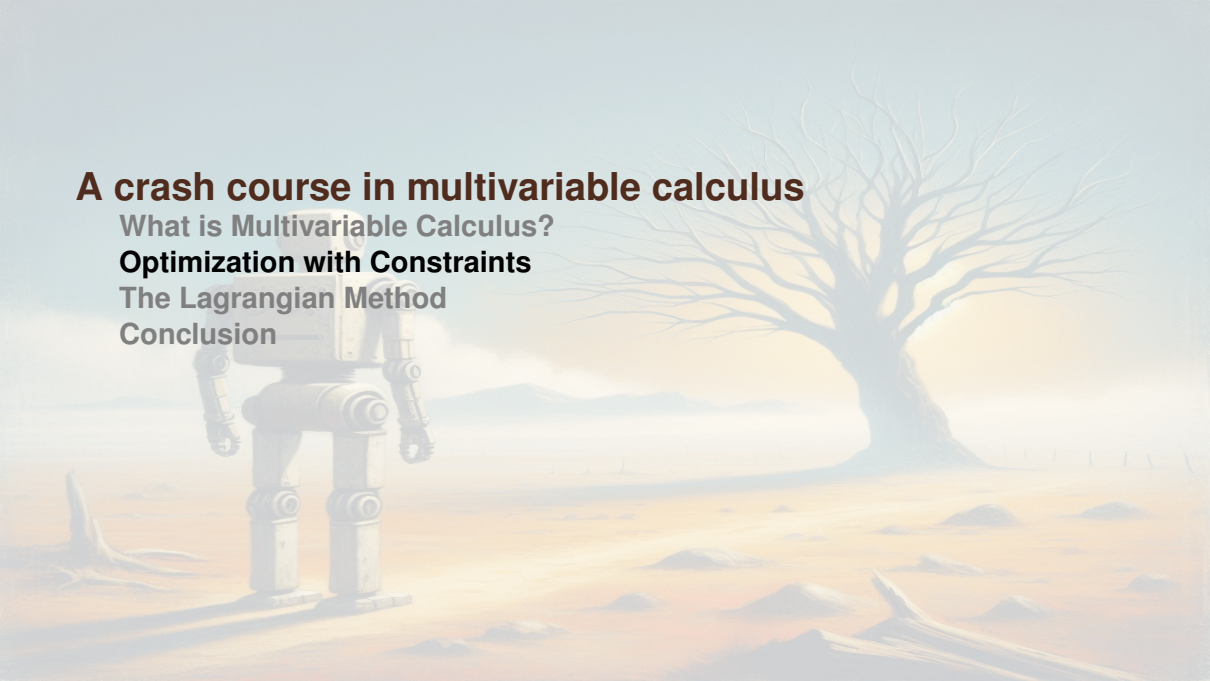
Follow the gradient



At a peak or a valley (a local max/min), the ground is flat. So, $\nabla f = (0, 0)$

A crash course in multivariable calculus

What is Multivariable Calculus?
Optimization with Constraints
The Lagrangian Method
Conclusion



The real-world problem

Often, we want to maximize or minimize a function, but we don't have unlimited freedom. We have **constraints**

- Maximize the profit of your company... *subject to a limited budget*
- Minimize the material used for a can... *that must hold a specific volume*
- Find the highest point on a mountain... *while staying on a specific trail*

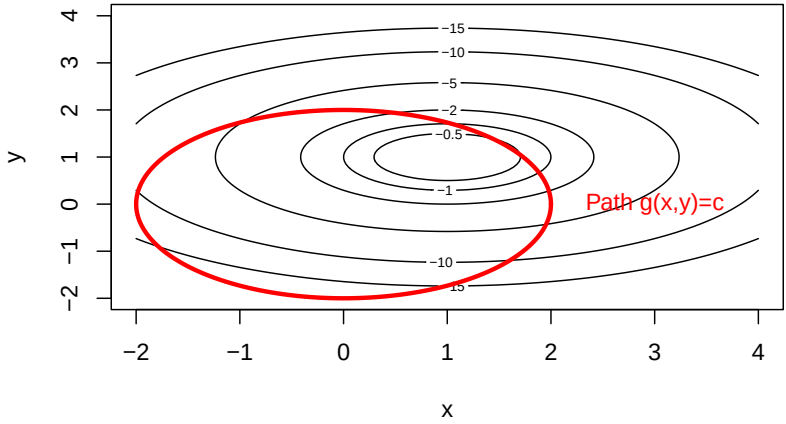
Setting the gradient to zero ($\nabla f = 0$) finds the highest point on the whole mountain, which might not be on our trail!

Visualizing the problem

Imagine our function $f(x, y)$ is the altitude on a map (contour lines)

Our constraint, $g(x, y) = c$, is a specific path we must walk on

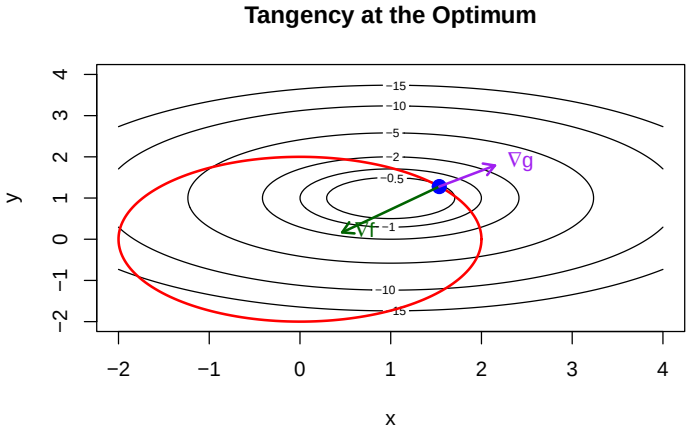
Optimization with a Constraint



We are looking for the highest (or lowest) point *along the red path*

The key insight

At the optimal point on the path, the path will be perfectly **tangent** to the contour line of the surface

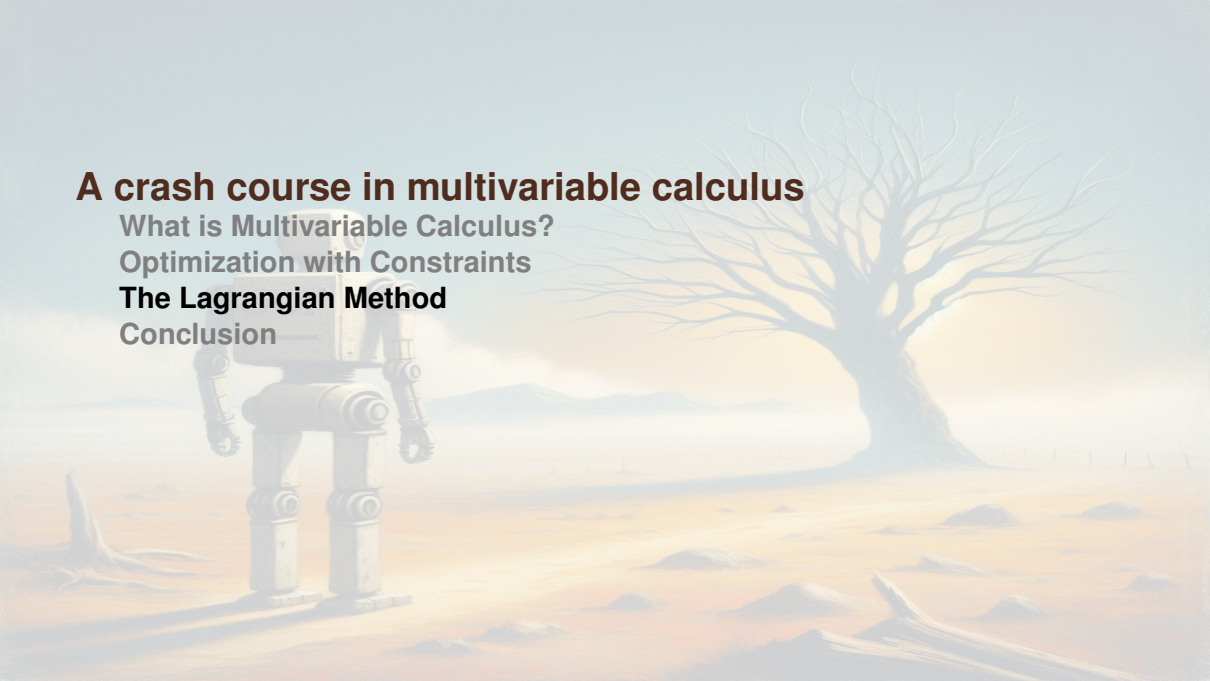


Why? If the path crossed the contour line, you could move along the path to get to a higher (or lower) contour

Mathematically, this tangency means the gradient vectors of the function and the constraint are **parallel**

$$\nabla f = \lambda \nabla g$$

The scalar λ (lambda) is called the **Lagrange multiplier**



A crash course in multivariable calculus

What is Multivariable Calculus?
Optimization with Constraints
The Lagrangian Method
Conclusion

The Lagrangian function

The condition $\nabla f = \lambda \nabla g$ is clever, but solving it can be messy

Instead, we combine our function and constraint into a single, new function called the **Lagrangian**

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda[g(x, y) - c]$$

- $f(x, y)$ the function we want to optimize
- $g(x, y) = c$ the constraint we must follow
- λ the Lagrange multiplier

Finding the unconstrained optimum of $\mathcal{L}$ solves the original constrained problem!

The method – step-by-step

To find the optimum of the Lagrangian $\mathcal{L}(x, y, \lambda)$, we find where its gradient is zero

We take the partial derivative with respect to *all* its variables (x, y, and λ) and set them to zero

1. $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f}{\partial x} - \lambda \frac{\partial g}{\partial x} = 0$
2. $\frac{\partial \mathcal{L}}{\partial y} = \frac{\partial f}{\partial y} - \lambda \frac{\partial g}{\partial y} = 0$
3. $\frac{\partial \mathcal{L}}{\partial \lambda} = -(g(x, y) - c) = 0 \implies g(x, y) = c$

The first two equations rearrange to $\nabla f = \lambda \nabla g$ and the third equation is the original constraint

Example: Fencing a Field

Problem: You have 40 meters of fence. What is the largest rectangular area you can enclose?

- **Maximize Area:** $A(x, y) = xy$
- **Constraint (Perimeter):** $2x + 2y = 40$

1. Form the Lagrangian:

$$\mathcal{L}(x, y, \lambda) = xy - \lambda(2x + 2y - 40)$$

2. Take Partial Derivatives:

- $\frac{\partial \mathcal{L}}{\partial x} = y - 2\lambda = 0 \implies y = 2\lambda$
- $\frac{\partial \mathcal{L}}{\partial y} = x - 2\lambda = 0 \implies x = 2\lambda$
- $\frac{\partial \mathcal{L}}{\partial \lambda} = -(2x + 2y - 40) = 0$

Example: Solution

From the first two equations, we see that $x = y$

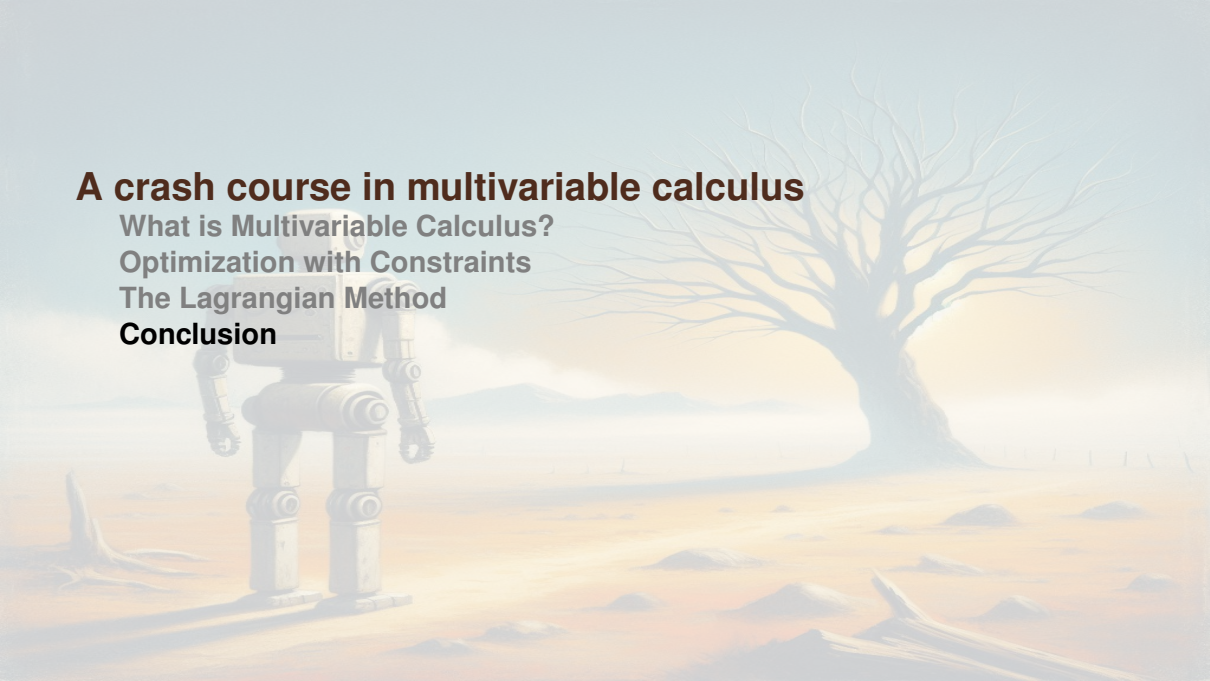
Now, substitute this into the third equation (the constraint):

$$2x + 2(x) = 40$$

$$4x = 40$$

$$x = 10$$

Since $x = y$, we have $y = 10$, i.e., optimal dimensions are 10m by 10m (a square), giving a maximum area of 100 m^2



A crash course in multivariable calculus

- What is Multivariable Calculus?
- Optimization with Constraints
- The Lagrangian Method
- **Conclusion**

What does λ mean?

The Lagrange multiplier λ has a very useful interpretation

It tells you how much the optimal value of your function f will change if you slightly relax the constraint c

$$\lambda = \frac{df_{optimal}}{dc}$$

In our example: If we had 41 meters of fence instead of 40 (so c changes by 1), how much would the max area increase?

$x = y = 2\lambda$, so $\lambda = x/2 = 10/2 = 5$. The maximum area would increase by approximately 5 m^2

Least squares problems

MATH 2740 – Mathematics of Data Science – Unit U03

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Remark

This is the slide version of `Canada-census-get-plot-data.Rmd` in the folder `CODE` of the course repository

The `html` file generated there (link) is much easier to read than these slides

Canadian census data

We want to consider the evolution of the population of Canada through time

For this, we grab the Canadian census data

Search for (Google) “Canada historical census data csv", since csv (comma separated values) is a very easy format to use with R

Here, we find a csv for 1851 to 1976

We follow the link to Table A2-14, where we find another link, this time to a csv file. This is what we use in R

Grabing the Canadian census data

The function `read.csv` reads in a file (potentially directly from the web)

```
prefix = "https://www150.statcan.gc.ca/n1/en/pub/11-516-x/sectiona/"
page = "A2_14-eng.csv?st=L7vSnqio"
url = paste0(prefix, page)
data_old = read.csv(url)
```

This directly assigns the result to the variable `data`

We then use the function `head` to show the first few lines in the result

```
head(data_old)
```

```
##    X Series.A2.14.
## 1 NA
## 2 NA          Year
## 3 NA
## 4 NA
## 5 NA
## 6 NA
##   Population.of.Canada..by.province..census.dates..1851.to.1976 X.1
## 1                                                                NA
## 2                                                        Canada  NA Newfo
## 3                                                                NA
## 4                                                                NA
## 5                                                             2  NA
## 6                                                                NA
##      X.3 X.4    X.5       X.6    X.7     X.8       X.9 X.10    X.11 X.12
## 1             NA                                          NA           NA
```

Obviously, this does not make a lot of sense. This is normal: take a look at the first few lines in the file. They take the form

```
head(data_old, n = 1)

##    X Series.A2.14.
## 1 NA
##   Population.of.Canada..by.province..census.dates..1851.to.1976 X.1 X.2 X
## 1                                                                 NA
##   X.5 X.6 X.7 X.8 X.9 X.10 X.11 X.12 X.13 X.14 X.15 X.16 X.17 X.18 X.19 X
## 1                      NA        NA        NA        NA             NA
```

This happens often: the first few lines are here to describe the information contained in the data set, i.e., the so-called **metadata**

It is easy to deal with this: the function `read.csv` takes the optional argument `skip`, which indicates how many lines to skip at the beginning of the file

The second line is also empty, so let us skip it too

```
data_old = read.csv(url, skip = 2)
head(data_old, n = 2)
```

```
##    X Year Canada X.1 Newfound. Prince X.2   Nova       New Quebec Ontario
## 1 NA              NA      land Edward  NA Scotia Brunswick
## 2 NA              NA           Island  NA
##   Manitoba X.3 Saskat. X.4 Alberta X.5  British X.6     Yukon   Northwest
## 1           NA  chewan  NA          NA Columbia  NA Territory Territories
## 2           NA          NA          NA           NA
##   X.8
## 1  NA
## 2  NA
```

To make things legible, the table authors used 3 rows to write long names (e.g., Prince Edward Island is written over 3 rows)

Note, however, that `read.csv` has rightly picked up on the first row being the column names

Because we are only interested in the total population of the country and the year, let us get rid of the first 4 rows and of all columns except the second (Year) and third (Canada)

```
data_old = data_old[5:dim(data_old)[1], 2:3]
head(data_old, n=4)
```

```
##   Year     Canada
## 5 1976 22,992,604
## 6
## 7 1971 21,568,311
## 8 1966 20,014,880
```

Still not perfect...

```
head(data_old, n=3)

##   Year     Canada
## 5 1976 22,992,604
## 6
## 7 1971 21,568,311

tail(data_old, n=2)

##                            Year Canada
## 28 see notes to series A15-53.
## 29                1848 figure.
```

1. there are some empty rows
2. the last few rows need to be removed too, they contain remarks about the data
3. the population counts contain commas
4. it would be better if years were increasing

Fixing these issues

1 and 2 are easy: remark that the Canada column is empty for both issues. Entries in the column are strings. Looking for empty content therefore means looking for empty strings

So to fix 1 and 2, we keep the rows where Canada does not equal the empty string

To get rid of commas, we just need to substitute an empty string for ","

To sort, we find the order for the years and apply it to the entire table

Finally, as remarked above, for now, both the year and the population are considered as strings. To plot, we will have to indicate that these are numbers, not characters

```r
data_old = data_old[which(data_old$Canada != ""),]
data_old$Canada = gsub(",", "", data_old$Canada)
order_data = order(data_old$Year)
data_old = data_old[order_data,]
data_old$Year = as.numeric(data_old$Year)
data_old$Canada = as.numeric(data_old$Canada)
head(data_old)
```

```
##    Year  Canada
## 23 1851 2436297
## 22 1861 3229633
## 21 1871 3689257
## 20 1881 4324810
## 19 1891 4833239
## 17 1901 5371315
```

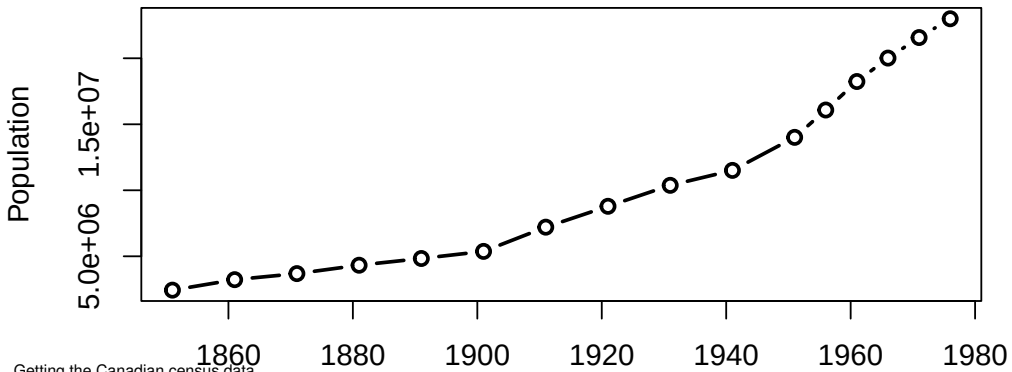
Row numbers are a little weird

```
row.names(data_old) = 1:dim(data_old)[1]
head(data_old)
```

```
##   Year  Canada
## 1 1851 2436297
## 2 1861 3229633
## 3 1871 3689257
## 4 1881 4324810
## 5 1891 4833239
## 6 1901 5371315
```

Well, that looks about right! Let's see what this looks like in a graph

```r
plot(data_old$Year, data_old$Canada,
    type = "b", lwd = 2,
    xlab = "Year", ylab = "Population")
```



But wait, this is only to 1976..!

Looking around, we find another table here with a download csv link (this one)

StatCan has added Cloudflare bot protection, so automated downloads from R fail. We download it via browser and save to `DATA/98-315-XWE2011001-001.csv`

```
data_new = read.csv("../DATA/98-315-XWE2011001-001.csv",
                    fileEncoding = "latin1")
```

The table is 720KB, so surely there must be more to this than just the population

To get a sense of that, we dump a few random rows

```
data_new[sort(unique(c(1,sample(nrow(data_new), 4)))), ]
```

```
##      GEOGRAPHY.NAME                          CHARACTERISTIC YEAR.S.   TOTA
## 1            Canada              Population (in thousands)    1956 16081.
## 12           Canada              Population (in thousands)    2011 33477.
## 1981     Saint John % of the population aged 0 to 14 years    2011    16.
## 6778       Winnipeg             % of two-person households    2006    32.
## 7235         Regina           % of three-person households    2001    15.
##      FLAG_TOTAL
## 1
## 12
## 1981
## 6778
## 7235
```

Flat tables

This is a **flat table**: each row contains a piece of information, the columns describe what this piece of information is about

This has *way more* information than we need: we just want the population of Canada and here we get 9960 rows over 213 characteristics

Also, population is expressed in thousands, so once we selected what we want, we need to multiply by 1,000

Selecting rows

We want the rows where the geography is “Canada” and the characteristic is “Population (in thousands)”

Find indices of rows that satisfy the first criterion and those that satisfy the second; intersecting these two sets of indices, we have selected the rows we want

```
idx_CAN = which(data_new$GEOGRAPHY.NAME == "Canada")
idx_char = which(data_new$CHARACTERISTIC == "Population (in thousands)")
idx_keep = intersect(idx_CAN, idx_char)
```

Let us keep only these rows

```
data_new = data_new[idx_keep,]
head(data_new, n = 8)
```

```
##   GEOGRAPHY.NAME            CHARACTERISTIC YEAR.S. TOTAL FLAG_TOTAL
## 1         Canada Population (in thousands)    1956 16081
## 2         Canada Population (in thousands)    1961 18238
## 3         Canada Population (in thousands)    1966 20015
## 4         Canada Population (in thousands)    1971 21568
## 5         Canada Population (in thousands)    1976 22993
## 6         Canada Population (in thousands)    1981 24343
## 7         Canada Population (in thousands)    1986 25309
## 8         Canada Population (in thousands)    1991 27297
```

We want to concatenate this data frame with the one from earlier

To do this, we need the two data frames to have the same number of columns, same column names and same entry types (notice that `YEAR.S.` in `data_new` is a column of characters)

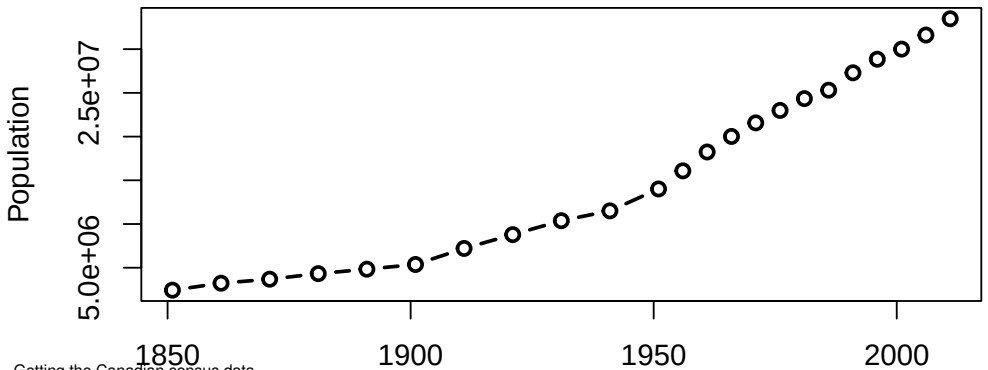
What remains to be done

- Rename the columns in the pruned old data (data_pruned) to `year` and `population`. Personally, I prefer lowercase column names.. and `population` is more informative than `Canada`
- Keep only the relevant columns in `data_new`, rename them accordingly and multiply population by 1,000 there
- Transform year in `data_new` to numbers
- We already have data up to and including 1976 in `data_old`, so get rid of that in `data_new`
- Append the rows of `data_new` to those of `data_pruned`

```r
colnames(data_old) = c("year", "population")
data_new = data_new[,c("YEAR.S.","TOTAL")]
colnames(data_new) = c("year", "population")
data_new$year = as.numeric(data_new$year)
data_new = data_new[which(data_new$year>1976),]
data_new$population = data_new$population*1000

data = rbind(data_old,data_new)
```

```r
plot(data$year, data$population,
    type = "b", lwd = 2,
    xlab = "Year", ylab = "Population")
```



Save the processed data

In case we need the data elsewhere, we save the data to a `csv` file

```
write.csv(data, file = "../CODE/Canada_census.csv")
```

Using `readr` saves the data without row numbers (by default), so we can do this instead

```
readr::write_csv(data, file = "../CODE/Canada_census.csv")
```

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Least squares problems

Initial considerations

The least squares problem

Fitting something more complicated

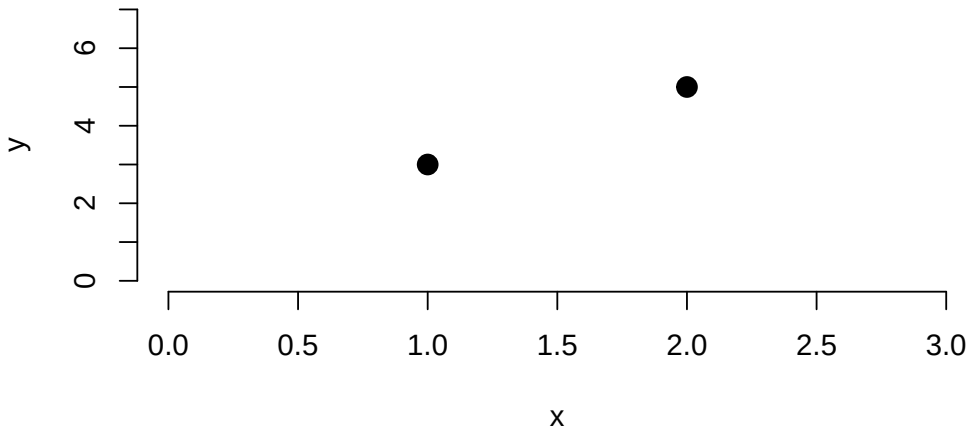
We just collected the census data for Canada

Suppose we want to predict the population of Canada in 20 years given the historical population growth seen in the previous plot. What can we do?

If there were just two points, we could easily "drive" a line through these two points. However, we have much more than two points, so we will use *fitting*, *i.e.*, try to make a curve come as close to possible to the points

We start with a line, giving rise to **linear least squares**

Least squares approximation – A trivial case



We want to find the equation of a line $y = a + bx$ that goes through these two points, i.e., we seek a and b such that

$$3 = a + b$$
$$5 = a + 2b$$

i.e., they satisfy $y = a + bx$ for $(x, y) = (1, 3)$ and $(x, y) = (2, 5)$

This is a linear system with 2 equations and 2 unknowns a and b

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}$$

We know from Theorem **??** (Linear algebra in a nutshell) that this system has a unique solution if the matrix

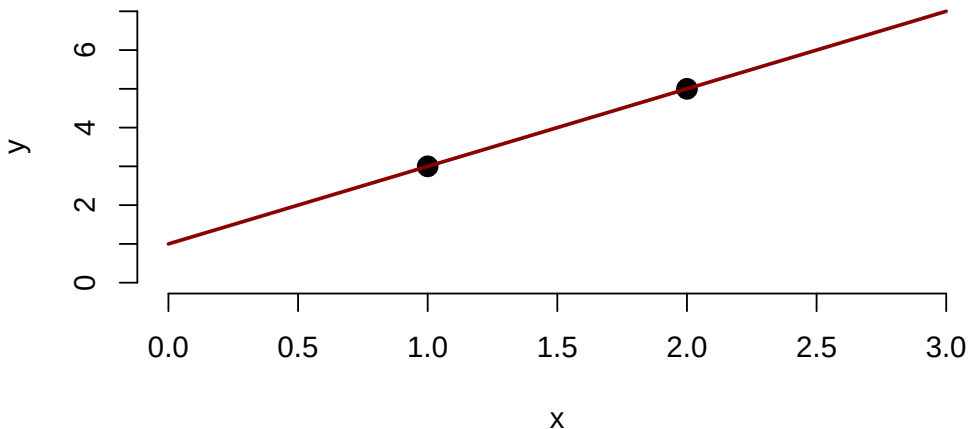
$$M = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$$

is invertible

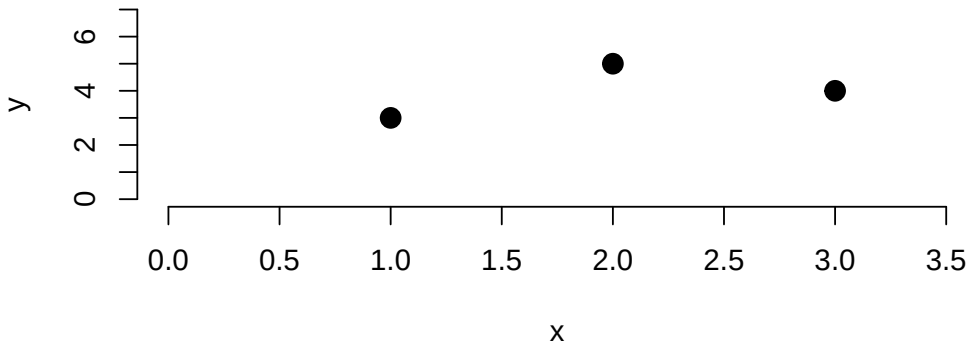
$\det(M) = 1$, so we are good, we'll find a and b easily..

```
A = matrix(c(1,1,1,2), nr = 2, nc = 2, byrow = TRUE)
rhs = matrix(c(3,5), nr = 2, nc =1)
coefs = solve(A,rhs)
coefs
```

```
##      [,1]
## [1,]    1
## [2,]    2
```


y
0
2
4
6
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x

Now let's add another point



These points are clearly not colinear, so there is not one line going through the 3

We end up with an *overdetermined* system

$$
\begin{aligned}
3 &= a + b \\
5 &= a + 2b \\
4 &= a + 3b
\end{aligned}
$$

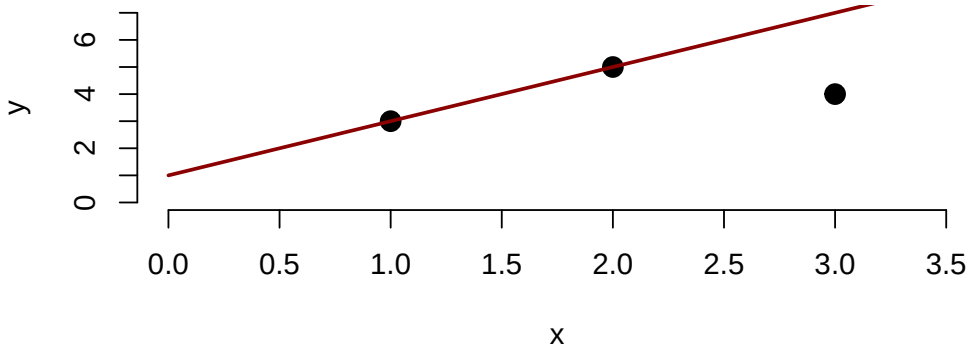
i.e.,

$$
\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 4 \end{pmatrix}
$$

We have verified visually that the points are not colinear, so this system has no solution

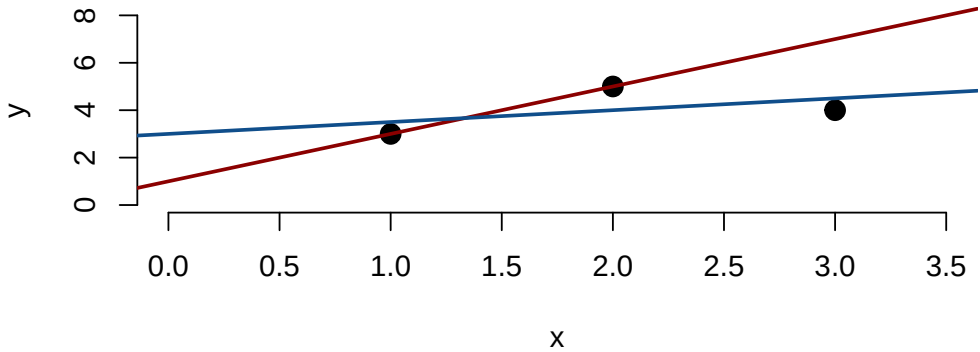
(If you had to do it for good, you consider two vectors stemming from these 3 points and compute the angle between them or check that one is a multiple of the other)

So let us instead try to find the line that comes "closest" to the 3 points



Obviously, not sensational..

Let's eyeball it – Less steep and higher y-intercept



The blue line looks better, but how do we quantify this?

How do we find "how far away" we are?

- We could use projections onto the line (which we know minimises the distance)

- However, this will be a problem if we later decide that rather than a straight line, we want to use something more "funky" like a quadratic or an exponential

Compare, for a given value x, the distance between the true value y and the value of y obtained using the curve (line, here) that we use to fit the data

Let (x_i, y_i) be the data points, i.e., here, $(x_1, y_1) = (1, 3)$, $(x_2, y_2) = (2, 5)$ and $(x_3, y_3) = (3, 4)$

Suppose we use a line with equation $y = a + bx$ and that we pick a value for a and b. Then at x_1,

$$\tilde{y}_1 = a + bx_1 = a + b$$

at x_2

$$\tilde{y}_2 = a + bx_2 = a + 2b$$

and at x_3,

$$\tilde{y}_3 = a + bx_3 = a + 3b$$

Consider x_1, for instance. The error we made by using the line with coefficients (a, b) is $\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)}$

For future use, let us create a function for $y = a_0 + a_1 x$

```
my_line = function(x, a_0, a_1){
    return(a_0 + a_1*x)
}
```

Functions are super useful when programming

```
my_line(1,2,3)
```

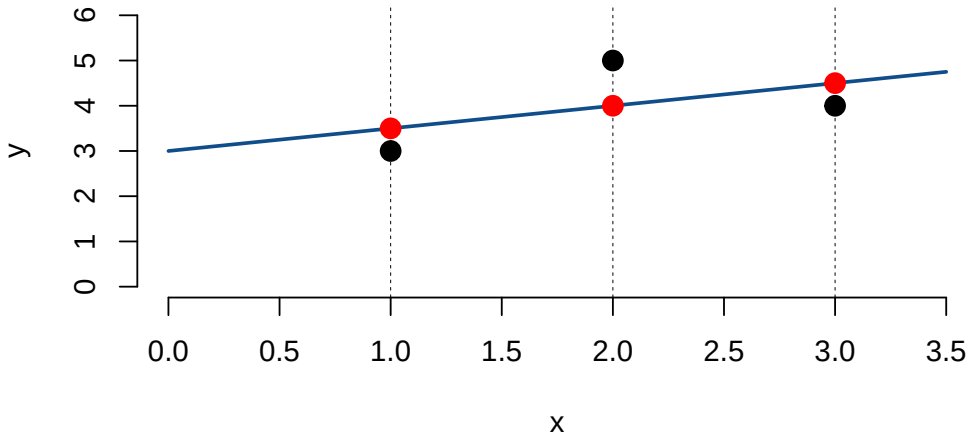
```
## [1] 5
```

```
my_line(a_0 = 2, a_1 = 3, x = 1)
```

```
## [1] 5
```

```
my_line(x = c(1,2,3), a_0 = 2, a_1 = 3)
```

```
## [1]  5  8 11
```


y
0 1 2 3 4 5 6
0.0 0.5 1.0 1.5 2.0 2.5 3.0 3.5
x

Let us return to the error

$$\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)}$$

We have

$$\overrightarrow{(x_1, y_1)(x_1, \tilde{y}_1)} = (x_1 - x_1, y_1 - \tilde{y}_1) = (0, y_1 - \tilde{y}_1)$$

Let us call

$$\varepsilon_1 = y_1 - \tilde{y}_1$$

We can compute ε_2 and ε_3 too. And we can then form the **error vector**

$$\mathbf{e} = (\varepsilon_1, \varepsilon_2, \varepsilon_3)^T$$

The norm of $\mathbf{e}$, $\|\mathbf{e}\|$, then tells us how much error we are making for the choice of (a, b) we are using

The norm of $\|\mathbf{e}\|$ tells us how much error we are making for the choice of (a, b) we are using

So our objective is to find (a, b) such that $\|\mathbf{e}\|$ is minimal

We could use various norms, but the Euclidean norm has some very interesting properties, so we use

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \varepsilon_2^2 + \varepsilon_3^2}$$

The linear least squares problem

Given a collection of data points $(x_1, y_1), \ldots, (x_n, y_n)$, find the coefficients a, b of the line $y = a + bx$ such that

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \cdots + \varepsilon_n^2} = \sqrt{(y_1 - \tilde{y}_1)^2 + \cdots + (y_n - \tilde{y}_n)^2}$$

is minimal, where $\tilde{y}_i = a + bx_i$, for $i = 1, \ldots, n$

Let us first hack a brute force solution! (For the example we have been using this far)

We have our three points in the list `points`, the function `my_line` that computes the value $\tilde{y}$ given x and a, b, so let us make a new function that, given a, b, computes **e**

We'll also pass the points `points`

A function to compute the error

```
error = function(a_0, a_1, points) {
    y_tilde = my_line(points$x, a_0 = a_0, a_1 = a_1)
    e = points$y - y_tilde
    return(sqrt(sum(e^2)))
}
error(a_0 = 2, a_1 = 3, points)
```

```
## [1] 7.874008
```

```
error(a_0 = 3, a_1 = 0.5, points)
```

```
## [1] 1.224745
```

```
error(a_0 = 3.1, a_1 = 0.48, points)
```

```
## [1] 1.229471
```

We can't be doing this by hand..!

Let's minimise using something cool – *genetic algorithms*

- Genetic algorithms are a stochastic *optimisation* method. There are other types, e.g., gradient descent (deterministic)
- The idea is to use a mechanism mimicking evolution's drive towards higher fitness
- The function value is its fitness
- We try different genes (here, different values of a, b) and evaluate their fitness.. keep the good ones
- We mutate or crossover genes, throw in new ones, etc.
- We keep doing this until we reach a stopping criterion
- We then return the best gene we found

```r
if (!require("GA", quietly = TRUE)) {
  install.packages("GA")
  library(GA)
}
GA = ga(type = "real-valued",
        fitness = function(x) -error(a_0 = x[1], a_1 = x[2], points),
        suggestions = c(a_0 = 2, a_1 = 3),
        lower = c(-10, -10), upper = c(10, 10),
        popSize = 200, maxiter = 150)
```

GA returns an S4 object of class ga

```
GA

## An object of class "ga"
##
## Call:
## ga(type = "real-valued", fitness = function(x) -error(a_0 = x[1],     a_1
##
## Available slots:
##  [1] "call"         "type"         "lower"        "upper"        "nBits"
##  [6] "names"        "popSize"      "iter"         "run"          "maxiter
## [11] "suggestions"  "population"   "elitism"      "pcrossover"   "pmutati
## [16] "optim"        "fitness"      "summary"      "bestSol"      "fitness
## [21] "solution"
```

Access slots using @. For instance, the GA finds the coefficients

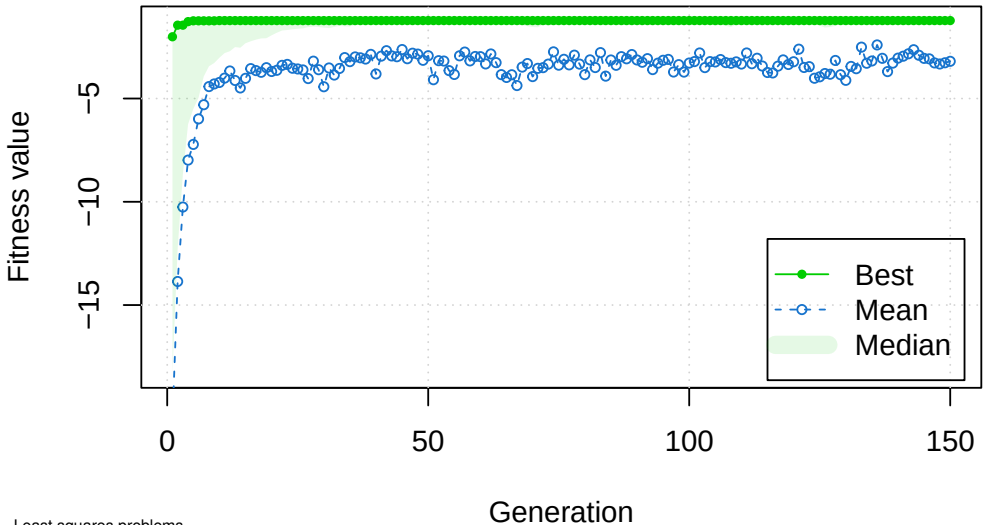
```
GA@solution
```

```
##             x1        x2
## [1,] 3.001388 0.4992915
```

with fitness value (i.e., $\|\boldsymbol{e}\|$)

```
-GA@fitnessValue
```

```
## [1] 1.224745
```


Fitness value
−5
−10
−15
0
50
100
150
Generation
Best
Mean
Median

- Here, however, we do not have to go brute force: we can reason using mathematics

- We now take a little detour on the math side of things, we will come back to code in a while..

Least squares problems

- Initial considerations
- **The least squares problem**
- Fitting something more complicated

The least squares problem (simplest version)

Definition 1

Given a collection of points $(x_1, y_1), \ldots, (x_n, y_n)$*, find the coefficients* a, b *of the line* $y = a + bx$ *such that*

$$\|\mathbf{e}\| = \sqrt{\varepsilon_1^2 + \cdots + \varepsilon_n^2} = \sqrt{(y_1 - \tilde{y}_1)^2 + \cdots + (y_n - \tilde{y}_n)^2}$$

is minimal, where $\tilde{y}_i = a + bx_i$ *for* $i = 1, \ldots, n$

We just saw how to solve this by brute force using a genetic algorith to minimise $\|e\|$, let us now see how to solve this problem "properly"

For a data point $i = 1, \ldots, n$

$$\varepsilon_i = y_i - \tilde{y}_i = y_i - (a + bx_i)$$

So if we write this for all data points,

$$\begin{aligned} \varepsilon_1 &= y_1 - (a + bx_1) \\ &\vdots \\ \varepsilon_n &= y_n - (a + bx_n) \end{aligned}$$

In matrix form

$$\boldsymbol{e} = \boldsymbol{b} - A\boldsymbol{x}$$

with

$$\boldsymbol{e} = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix}, A = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \boldsymbol{x} = \begin{pmatrix} a \\ b \end{pmatrix} \text{ and } \boldsymbol{b} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

The least squares problem (reformulated)

Definition 2 (Least squares solutions)

Consider a collection of points $(x_1, y_1), \ldots, (x_n, y_n)$*, a matrix* $A \in \mathcal{M}_{mn}$*,* $\boldsymbol{b} \in \mathbb{R}^m$*. A* ***least squares solution*** *of* $A\boldsymbol{x} = \boldsymbol{b}$ *is a vector* $\tilde{\boldsymbol{x}} \in \mathbb{R}^n$ *s.t.*

$$\forall \boldsymbol{x} \in \mathbb{R}^n, \quad \|\boldsymbol{b} - A\tilde{\boldsymbol{x}}\| \leq \|\boldsymbol{b} - A\boldsymbol{x}\|$$

Needed to solve the problem

Definition 3 (Best approximation)

Let V be a vector space, $W \subset V$ and $\mathbf{v} \in V$. The **best approximation** to $\mathbf{v}$ in W is $\tilde{\mathbf{v}} \in W$ s.t.

$$\forall \mathbf{w} \in W, \mathbf{w} \neq \tilde{\mathbf{v}}, \quad \|\mathbf{v} - \tilde{\mathbf{v}}\| < \|\mathbf{v} - \mathbf{w}\|$$

Theorem 4 (Best approximation theorem)

Let V be a vector space with an inner product, $W \subset V$ and $\mathbf{v} \in V$. Then $\text{proj}_W(\mathbf{v})$ is the best approximation to $\mathbf{v}$ in W

Let us find the least squares solution

$\forall \boldsymbol{x} \mathbb{R}^n$, $A\boldsymbol{x}$ is a vector in the **column space** of A (the space spanned by the vectors making up the columns of A)

Since $\boldsymbol{x} \in \mathbb{R}^n$, $A\boldsymbol{x} \in \text{col}(A)$

$\implies$ least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$ is a vector $\tilde{\boldsymbol{y}} \in \text{col}(A)$ s.t.

$$\forall \boldsymbol{y} \in \text{col}(A), \quad \|\boldsymbol{b} - \tilde{\boldsymbol{y}}\| \leq \|\boldsymbol{b} - \boldsymbol{y}\|$$

This looks very much like Best approximation and Best approximation theorem

Putting things together

We just stated: The least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$ is a vector $\tilde{\boldsymbol{y}} \in \text{col}(A)$ s.t.

$$\forall \boldsymbol{y} \in \text{col}(A), \quad \|\boldsymbol{b} - \tilde{\boldsymbol{y}}\| \leq \|\boldsymbol{b} - \boldsymbol{y}\|$$

We know (reformulating a tad):

Theorem 5 (Best approximation theorem)

Let V be a vector space with an inner product, $W \subset V$ and $\mathbf{v} \in V$. Then $\text{proj}_W(\mathbf{v}) \in W$ is the best approximation to $\mathbf{v}$ in W, i.e.,

$$\forall \mathbf{w} \in W, \mathbf{w} \neq \text{proj}_W(\mathbf{v}), \quad \|\mathbf{v} - \text{proj}_W(\mathbf{v})\| < \|\mathbf{v} - \mathbf{w}\|$$

$\implies W = \text{col}(A)$, $\boldsymbol{v} = \boldsymbol{b}$ and $\tilde{\boldsymbol{y}} = \text{proj}_{\text{col}(A)}(\mathbf{b})$

So if $\tilde{\boldsymbol{x}}$ is a least squares solution of $A\boldsymbol{x} = \boldsymbol{b}$, then

$$\tilde{\boldsymbol{y}} = A\tilde{\boldsymbol{x}} = \text{proj}_{\text{col}(A)}(\mathbf{b})$$

We have

$$\boldsymbol{b} - A\tilde{\boldsymbol{x}} = \boldsymbol{b} - \text{proj}_{\text{col}(A)}(\mathbf{b}) = \text{perp}_{\text{col}(A)}(\mathbf{b})$$

and it is easy to show that

$$\text{perp}_{\text{col}(A)}(\mathbf{b}) \perp \text{col}(A)$$

So for all columns $\boldsymbol{a}_i$ of A

$$\boldsymbol{a}_i \cdot (\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$$

which we can also write as $\boldsymbol{a}_i^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$

For all columns $\boldsymbol{a}_i$ of A,

$$\boldsymbol{a}_i^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = 0$$

This is equivalent to saying that

$$A^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = \mathbf{0}$$

We have

$$\begin{aligned} A^T(\boldsymbol{b} - A\tilde{\boldsymbol{x}}) = \mathbf{0} &\iff A^T\boldsymbol{b} - A^TA\tilde{\boldsymbol{x}} = \mathbf{0} \\ &\iff A^T\boldsymbol{b} = A^TA\tilde{\boldsymbol{x}} \\ &\iff A^TA\tilde{\boldsymbol{x}} = A^T\boldsymbol{b} \end{aligned}$$

The latter system constitutes the **normal equations** for $\tilde{\boldsymbol{x}}$

Invertibility of A^TA

Theorem 6 (Invertibility of A^TA)

Let $A \in \mathcal{M}_{mn}$. Then A has linearly independent columns $\iff$ A^TA invertible

Proof. Since $A^TA \in \mathcal{M}_n$ is square, it is invertible $\iff A^TA\boldsymbol{x} = \boldsymbol{0} \implies \boldsymbol{x} = \boldsymbol{0}$. Also, A has linearly independent columns $\iff A\boldsymbol{x} = \boldsymbol{0} \implies \boldsymbol{x} = \boldsymbol{0}$. Thus, it suffices to show that $A^TA\boldsymbol{x} = \boldsymbol{0} \iff A\boldsymbol{x} = \boldsymbol{0}$.

- ($\impliedby$) If $A\boldsymbol{x} = \boldsymbol{0}$, then multiplying on the left by A^T gives $A^TA\boldsymbol{x} = A^T\boldsymbol{0} = \boldsymbol{0}$.
- ($\implies$) Suppose $A^TA\boldsymbol{x} = \boldsymbol{0}$. Multiplying on the left by $\boldsymbol{x}^T$ gives

$$\boldsymbol{x}^TA^TA\boldsymbol{x} = \boldsymbol{x}^T\boldsymbol{0} = 0$$

But $\boldsymbol{x}^TA^TA\boldsymbol{x} = (A\boldsymbol{x})^T(A\boldsymbol{x}) = \|A\boldsymbol{x}\|^2$. Since the norm of a vector is zero if and only if the vector is the zero vector, we have $\|A\boldsymbol{x}\|^2 = 0 \implies A\boldsymbol{x} = \boldsymbol{0}$. □

Least squares theorem

Theorem 7 (Least squares theorem)

$A \in \mathcal{M}_{mn}$, $\boldsymbol{b} \in \mathbb{R}^m$. Then

1. *$A\boldsymbol{x} = \boldsymbol{b}$ always has at least one least squares solution $\tilde{\boldsymbol{x}}$*
2. *$\tilde{\boldsymbol{x}}$ least squares solution to $A\boldsymbol{x} = \boldsymbol{b} \iff \tilde{\boldsymbol{x}}$ is a solution to the normal equations $A^T A\tilde{\boldsymbol{x}} = A^T\boldsymbol{b}$*
3. *If $A^T A$ invertible, then the least squares solution is unique and*

$$\tilde{\boldsymbol{x}} = \left(A^T A\right)^{-1} A^T \boldsymbol{b}$$

We have seen 1 and 2, and we just proved 3!

Least squares problems

Initial considerations

The least squares problem

Fitting something more complicated

Suppose we want to fit something a bit more complicated..

For instance, instead of the affine function

$$y = a + bx$$

suppose we want to do the quadratic

$$y = a_0 + a_1 x + a_2 x^2$$

or even

$$y = k_0 e^{k_1 x}$$

How do we proceed?

Fitting the quadratic

We have the data points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ and want to fit

$$y = a_0 + a_1 x + a_2 x^2$$

At (x_1, y_1),

$$\tilde{y}_1 = a_0 + a_1 x_1 + a_2 x_1^2$$

$\vdots$

At (x_n, y_n),

$$\tilde{y}_n = a_0 + a_1 x_n + a_2 x_n^2$$

In terms of the error

$$\begin{aligned}\varepsilon_1 &= y_1 - \tilde{y}_1 = y_1 - (a_0 + a_1 x_1 + a_2 x_1^2)\\ &\vdots\\ \varepsilon_n &= y_n - \tilde{y}_n = y_n - (a_0 + a_1 x_n + a_2 x_n^2)\end{aligned}$$

i.e.,

$$\boldsymbol{e} = \boldsymbol{b} - A\boldsymbol{x}$$

where

$$\boldsymbol{e} = \begin{pmatrix}\varepsilon_1\\ \vdots\\ \varepsilon_n\end{pmatrix}, A = \begin{pmatrix}1 & x_1 & x_1^2\\ \vdots & \vdots & \vdots\\ 1 & x_n & x_n^2\end{pmatrix}, \boldsymbol{x} = \begin{pmatrix}a_0\\ a_1\\ a_2\end{pmatrix} \text{ and } \boldsymbol{b} = \begin{pmatrix}y_1\\ \vdots\\ y_n\end{pmatrix}$$

Theorem 7 applies, with here $A \in \mathcal{M}_{n3}$ and $\boldsymbol{b} \in \mathbb{R}^n$

Fitting the exponential

Things are a bit more complicated here

If we proceed as before, we get the system

$$\begin{aligned} y_1 &= k_0 e^{k_1 x_1} \\ &\vdots \\ y_n &= k_0 e^{k_1 x_n} \end{aligned}$$

$e^{k_1 x_i}$ is a nonlinear term, it cannot be put in a matrix

However: take the ln of both sides of the equation

$$\ln(y_i) = \ln(k_0 e^{k_1 x_i}) = \ln(k_0) + \ln(e^{k_1 x_i}) = \ln(k_0) + k_1 x_i$$

If $y_i, k_0 > 0$, then their ln are defined and we're in business..

$$\ln(y_i) = \ln(k_0) + k_1 x_i$$

So the system is

$$\boldsymbol{y} = A\boldsymbol{x} + \boldsymbol{b}$$

with

$$A = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \boldsymbol{x} = \begin{pmatrix} k_1 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} \ln(k_0) \end{pmatrix} \text{ and } \boldsymbol{y} = \begin{pmatrix} \ln(y_1) \\ \vdots \\ \ln(y_n) \end{pmatrix}$$

Getting the Canadian census data

Least squares problems

Example – Canadian population data

Example: Canadian census data

Let's consider data from the Canadian census. We first load and visualize it.

```
canada = read.csv("../DATA/Canada_census.csv")
canada = as.data.frame(canada)
plot(canada, main = "Canadian Population over Time",
     xlab = "Year", ylab = "Population")
```

Fitting a linear model

Let's first try to fit a linear model $y = a_0 + a_1 \times \text{year}$.

We construct the matrix A and the vector $\boldsymbol{b}$ (the population data):

```
b = canada$population
A = matrix(ncol = 2, nrow = dim(canada)[1],
           data = c(rep(1, dim(canada)[1]),
                    canada$year))
```

Recall that the normal equations are $A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b}$, where $\tilde{\boldsymbol{x}} = (a_0, a_1)^T$.

Solving the normal equations

We can compute $A^T A$ and solve the normal equations in R:

```
ATA = t(A) %*% A
invATA = solve(ATA)
x_tilde = invATA %*% t(A) %*% b
```

The resulting coefficients $\tilde{\boldsymbol{x}}$ are:

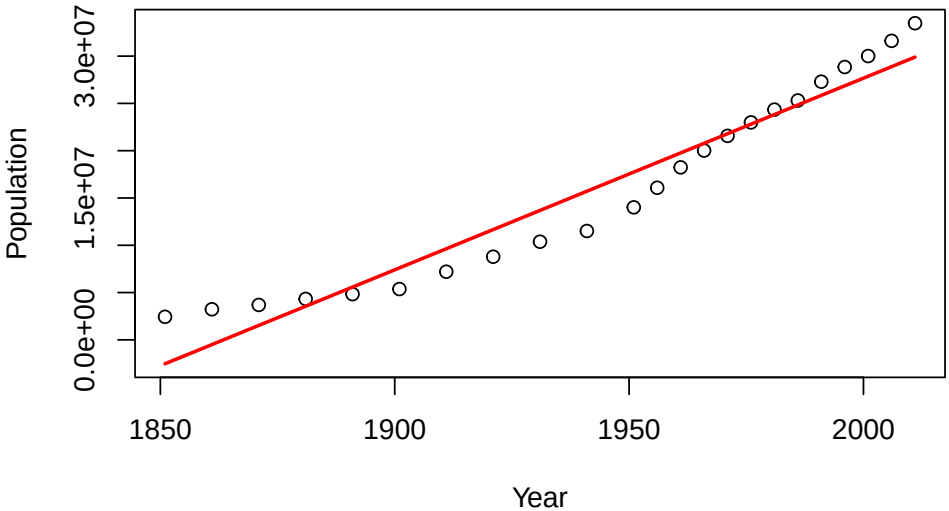
```
##               [,1]
## [1,] -377660589.2
## [2,]     202664.8
```

Visualizing the linear fit

We compute the fitted values and plot them alongside the data:

```
x_axis = seq(canada$year[1], max(canada$year), 1)
y_fitted = x_tilde[1] + x_tilde[2] * x_axis

plot(canada, ylim = c(min(y_fitted), max(canada$population)),
xlab = "Year", ylab = "Population")
lines(x_axis, y_fitted, col = "red", lwd = 2)
```


Population
0.0e+00
1.5e+07
3.0e+07
1850
1900
1950
2000
Year

Fitting a quadratic model

The linear fit misses the curvature in the population growth. Let us try a quadratic model:

$$y = a_0 + a_1 \times \text{year} + a_2 \times \text{year}^2$$

We can set up the matrix A_2:

```
A2 = matrix(ncol = 3, nrow = dim(canada)[1],
            data = c(rep(1, dim(canada)[1]),
                     canada$year,
                     canada$year^2))
```

Numerical issues

Let's try to compute $A_2^T A_2$ and its inverse:

```
ATA2 = t(A2) %*% A2
invATA2 = solve(ATA2)

## Error in `solve.default()`:
## !  system is computationally singular:  reciprocal condition number
= 2.05592e-20
```

The years are around 1900-2000, so year$^2 \approx 4 \times 10^6$. The columns of A_2 have vastly different scales, which makes $A_2^T A_2$ very poorly **conditioned**!

A better conditioned formulation

To avoid numerical issues, we can shift years so they start at 0:

$$\text{shifted_year} = \text{year} - \min(\text{year})$$

We are now fitting $y = a_0 + a_1 \times \text{shifted_year} + a_2 \times \text{shifted_year}^2$.

```
A3 = matrix(ncol = 3, nrow = dim(canada)[1],
            data = c(rep(1, dim(canada)[1]),
                     canada$year - min(canada$year),
                     (canada$year - min(canada$year))^2))
```

This makes the values in the columns of A_3 much closer in magnitude.

Solving the shifted quadratic model

We can now safely compute the normal equations for the shifted model:

```
ATA3 = t(A3) %*% A3
invATA3 = solve(ATA3)
x_tilde = invATA3 %*% t(A3) %*% b
```

The resulting coefficients $\tilde{\boldsymbol{x}}$ are:

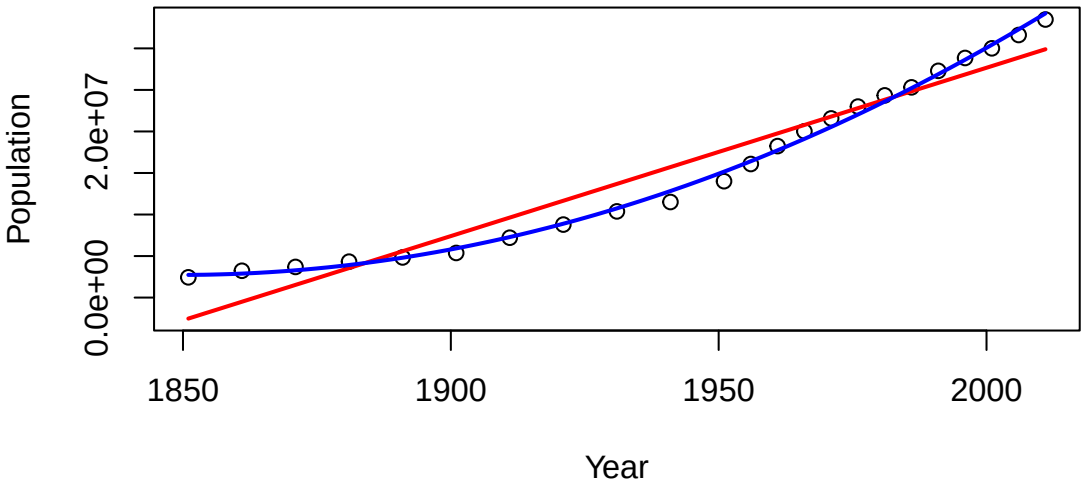
```
##             [,1]
## [1,] 2730500.544
## [2,]    3442.989
## [3,]    1208.270
```

Visualizing the quadratic fit

We compute the fitted quadratic curve and plot it:

```
y_fitted_quad = x_tilde[1] +
  x_tilde[2] * (x_axis - min(canada$year)) +
  x_tilde[3] * (x_axis - min(canada$year))^2

plot(canada, ylim = c(min(y_fitted), max(canada$population)),
xlab = "Year", ylab = "Population")
lines(x_axis, y_fitted, col = "red", lwd = 2) # Linear
lines(x_axis, y_fitted_quad, col = "blue", lwd = 2) # Quadratic
```

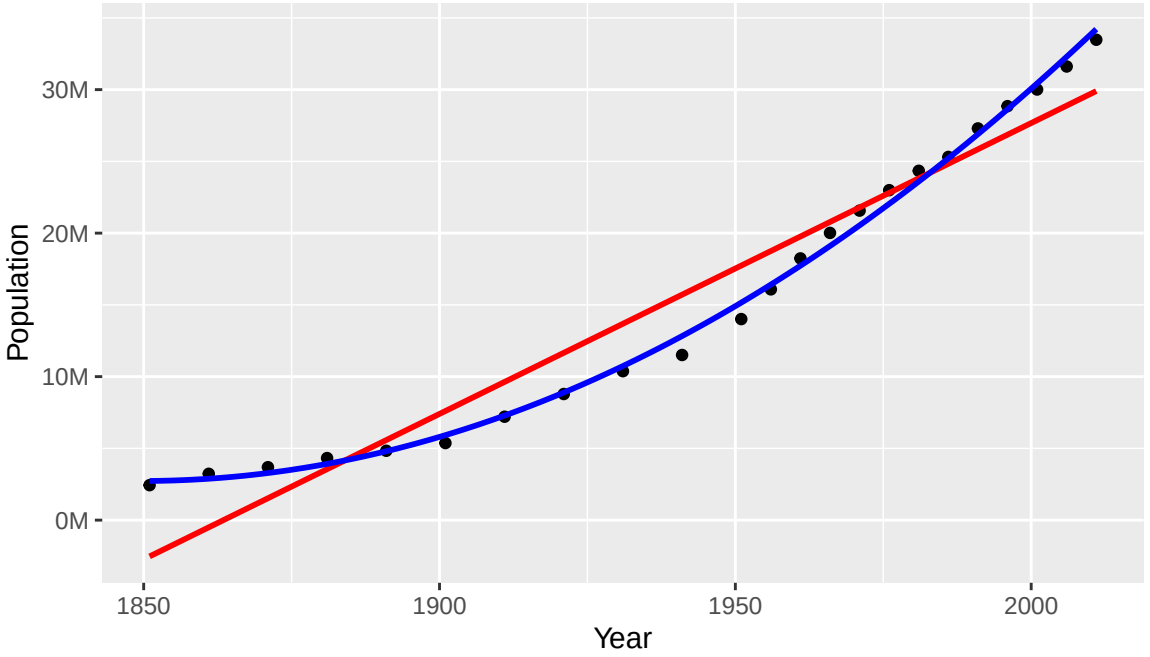

Population
0.0e+00
2.0e+07
1850
1900
1950
2000
Year

Visualizing the quadratic fit (ggplot2)

```
df_fit <- data.frame(year = x_axis,
                     linear = y_fitted,
                     quad = y_fitted_quad)

ggplot() +
  geom_point(data = canada, aes(x = year, y = population)) +
  geom_line(data = df_fit, aes(x = year, y = linear), color = "red", linewic
  geom_line(data = df_fit, aes(x = year, y = quad), color = "blue", linewidt
  scale_y_continuous(labels = function(x) paste0(x / 1e6, "M")) +
  labs(x = "Year", y = "Population")
```

`ggplot2` is a graphic package providing a “grammar of graphics”. Nicer output than default `R` plots, but also harder (LLMs help a lot here)


30M
20M
10M
0M
Population
1850
1900
1950
2000
Year

Takeaways

- Matrix formulations of least squares give us a straightforward way to solve polynomial curve fitting problems
- By adding columns corresponding to higher powers (or other transformations), we can easily expand the model
- **Numerical stability is crucial**: always be careful when dealing with polynomials of large numbers. Shifting and scaling your data can prevent matrices from becoming ill-conditioned
- You could shift to the mean or even $[-1, 1]$. Just remember to "unshift" back the years

QR factorisation & SVD

MATH 2740 – Mathematics of Data Science – Unit U04

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

The QR decomposition

The SVD (singular values decomposition)

Matrix factorisations

Matrix factorisations are popular because they allow to perform some computations more easily

There are several different types of factorisations. Here, we study just the QR factorisation (useful for many least squares problems) and SVD (useful in many contexts)

The QR decomposition

The SVD (singular values decomposition)

The QR decomposition

Orthogonality and projections

The Gram-Schmidt orthogonalisation process

Orthogonal matrices

The QR factorisation & Least squares

Definition 57 (Orthogonal set of vectors)

The set of vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *is an* ***orthogonal set*** *if*

$$\forall i, j = 1, \ldots, k, \quad i \neq j \implies \boldsymbol{v}_i \bullet \boldsymbol{v}_j = 0$$

Theorem 58

$\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *with* $\forall i$, $\boldsymbol{v}_i \neq \boldsymbol{0}$, *orthogonal set* $\implies$ $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *linearly independent*

Definition 59 (Orthogonal basis)

Let S be a basis of the subspace $W \subset \mathbb{R}^n$ composed of an orthogonal set of vectors. We say S is an **orthogonal basis** of W

Proof of Theorem 58

Assume $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ orthogonal set with $\boldsymbol{v}_i \neq \boldsymbol{0}$ for all $i = 1, \ldots, k$. Recall $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ is LI if

$$c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0} \iff c_1 = \cdots = c_k = 0$$

So assume $c_1, \ldots, c_k \in \mathbb{R}$ are s.t. $c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0}$. Recall that $\forall \boldsymbol{x} \in \mathbb{R}^k$, $\boldsymbol{0}_k \bullet \boldsymbol{x} = 0$. So for some $\boldsymbol{v}_i \in \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$

$$\begin{aligned} 0 &= \boldsymbol{0} \bullet \boldsymbol{v}_i \\ &= (c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k) \bullet \boldsymbol{v}_i \\ &= c_1\boldsymbol{v}_1 \bullet \boldsymbol{v}_i + \cdots + c_k\boldsymbol{v}_k \bullet \boldsymbol{v}_i \end{aligned} \tag{1}$$

As $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ orthogonal, $\boldsymbol{v}_j \bullet \boldsymbol{v}_i = 0$ when $i \neq j$, (1) reduces to

$$c_i\boldsymbol{v}_i \bullet \boldsymbol{v}_i = 0 \iff c_i\|\boldsymbol{v}_i\|^2 = 0$$

As $\boldsymbol{v}_i \neq 0$ for all i, $\|\boldsymbol{v}_i\| \neq 0$ and so $c_i = 0$. This is true for all i, hence the result □

Example – Vectors of the standard basis of $\mathbb{R}^3$

For $\mathbb{R}^3$, we denote

$$\boldsymbol{i} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{j} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \text{ and } \boldsymbol{k} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

($\mathbb{R}^k$ for $k > 3$, we denote them $\boldsymbol{e}_i$)
Clearly, $\{\boldsymbol{i}, \boldsymbol{j}\}$, $\{\boldsymbol{i}, \boldsymbol{k}\}$, $\{\boldsymbol{j}, \boldsymbol{k}\}$ and $\{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ orthogonal sets. The standard basis vectors are also $\neq \mathbf{0}$, so the sets are LI. And

$$\{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$$

is an orthogonal basis of $\mathbb{R}^3$ since it spans $\mathbb{R}^3$ and is LI

$$c_1\boldsymbol{i} + c_2\boldsymbol{j} + c_3\boldsymbol{k} = c_1 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + c_3 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix}$$

Orthonormal version of things

Definition 60 (Orthonormal set)

The set of vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ *is an* ***orthonormal set*** *if it is an orthogonal set and furthermore*

$$\forall i = 1, \ldots, k, \quad \|\boldsymbol{v}_i\| = 1$$

Definition 61 (Orthonormal basis)

A basis of the subspace $W \subset \mathbb{R}^n$ *is an* ***orthonormal basis*** *if the vectors composing it are an orthonormal set*

$\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\} \in \mathbb{R}^n$ is orthonormal if

$$\boldsymbol{v}_i \bullet \boldsymbol{v}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

The QR decomposition

- Orthogonality and projections
- **The Gram-Schmidt orthogonalisation process**
- Orthogonal matrices
- The QR factorisation & Least squares

What this aims to do is to construct an orthogonal basis for a subspace $W \subset \mathbb{R}^n$

To do this, we use the *Gram-Schmidt orthogonalisation process*, which turn s a basis of W into an orthogonal basis of W

Projections

Definition 62 (Orthogonal projection onto a subspace)

*$W \subset \mathbb{R}^n$ a subspace and $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ an orthogonal basis of W. $\forall \boldsymbol{v} \in \mathbb{R}^n$, the **orthogonal projection** of **v onto** W is*

$$\text{proj}_W(\boldsymbol{v}) = \frac{\boldsymbol{u}_1 \bullet \boldsymbol{v}}{\|\boldsymbol{u}_1\|^2}\boldsymbol{u}_1 + \cdots + \frac{\boldsymbol{u}_k \bullet \boldsymbol{v}}{\|\boldsymbol{u}_k\|^2}\boldsymbol{u}_k$$

Definition 63 (Component orthogonal to a subspace)

*$W \subset \mathbb{R}^n$ a subspace and $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k\}$ an orthogonal basis of W. $\forall \boldsymbol{v} \in \mathbb{R}^n$, the **component** of **v orthogonal to** W is*

$$\text{perp}_W(\boldsymbol{v}) = \boldsymbol{v} - \text{proj}_W(\boldsymbol{v})$$

Theorem 64 (Gram-Schmidt orthogonalisation process)

Let W be a subspace of $\mathbb{R}^n$ and $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ be a basis of W. Let

$$
\begin{aligned}
\boldsymbol{v}_1 &= \boldsymbol{x}_1 \\
\boldsymbol{v}_2 &= \boldsymbol{x}_2 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_2}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 \\
\boldsymbol{v}_3 &= \boldsymbol{x}_3 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_3}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 - \frac{\boldsymbol{v}_2 \bullet \boldsymbol{x}_3}{\|\boldsymbol{v}_2\|^2}\boldsymbol{v}_2 \\
&\vdots \\
\boldsymbol{v}_k &= \boldsymbol{x}_k - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{x}_k}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 - \cdots - \frac{\boldsymbol{v}_{k-1} \bullet \boldsymbol{x}_k}{\|\boldsymbol{v}_{k-1}\|^2}\boldsymbol{v}_{k-1}
\end{aligned}
$$

and

$$W_1 = \operatorname{span}(\boldsymbol{x}_1), W_2 = \operatorname{span}(\boldsymbol{x}_1, \boldsymbol{x}_2), \ldots, W_k = \operatorname{span}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k)$$

Then $\forall i = 1, \ldots, k$, $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_i\}$ is an orthogonal basis of W_i

Theorem 65 (Gram-Schmidt orthonormalisation process)

Let W be a subspace of $\mathbb{R}^n$ and $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ be a basis of W. Let

$$\tilde{\boldsymbol{v}}_1 = \boldsymbol{x}_1 / \|\boldsymbol{x}_1\|$$

$$\boldsymbol{v}_2 = \boldsymbol{x}_2 - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_2)\tilde{\boldsymbol{v}}_1 \qquad \tilde{\boldsymbol{v}}_2 = \frac{\boldsymbol{v}_2}{\|\boldsymbol{v}_2\|}$$

$$\boldsymbol{v}_3 = \boldsymbol{x}_3 - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_3)\tilde{\boldsymbol{v}}_1 - (\tilde{\boldsymbol{v}}_2 \bullet \boldsymbol{x}_3)\tilde{\boldsymbol{v}}_2 \qquad \tilde{\boldsymbol{v}}_3 = \frac{\boldsymbol{v}_3}{\|\boldsymbol{v}_3\|}$$

$$\vdots$$

$$\boldsymbol{v}_k = \boldsymbol{x}_k - (\tilde{\boldsymbol{v}}_1 \bullet \boldsymbol{x}_k)\tilde{\boldsymbol{v}}_1 - \cdots - (\tilde{\boldsymbol{v}}_{k-1} \bullet \boldsymbol{x}_k)\tilde{\boldsymbol{v}}_{k-1} \qquad \tilde{\boldsymbol{v}}_k = \frac{\boldsymbol{v}_k}{\|\boldsymbol{v}_k\|}$$

and

$$W_1 = \operatorname{span}(\boldsymbol{x}_1), W_2 = \operatorname{span}(\boldsymbol{x}_1, \boldsymbol{x}_2), \ldots, W_k = \operatorname{span}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k)$$

Then $\forall i = 1, \ldots, k$, $\{\tilde{\boldsymbol{v}}_1, \ldots, \tilde{\boldsymbol{v}}_i\}$ is an orthonormal basis of W_i

Links between the two processes

- To get an orthonormal basis, you can also use the orthogonalisation process then normalise the result

- There is no difference between the two methods, the only gain in the orthonormalisation process is that you don't need to divide by $\|\cdot\|^2$ when projecting, since you have already done so just before

The QR decomposition

Orthogonality and projections

The Gram-Schmidt orthogonalisation process

Orthogonal matrices

The QR factorisation & Least squares

Theorem 66

Let $Q \in \mathcal{M}_{mn}$. The columns of Q form an orthonormal set if and only if

$$Q^T Q = \mathbb{I}_n$$

Definition 67 (Orthogonal matrix)

*$Q \in \mathcal{M}_n$ is an **orthogonal matrix** if its columns form an orthonormal set*

So $Q \in \mathcal{M}_n$ orthogonal if $Q^T Q = \mathbb{I}$, i.e., $Q^T = Q^{-1}$

Theorem 68 (NSC for orthogonality)

$Q \in \mathcal{M}_n$ orthogonal $\iff$ $Q^{-1} = Q^T$

Orthogonal matrices “encode" isometries

Theorem 69 (Orthogonal matrices “encode" isometries)

Let $Q \in \mathcal{M}_n$. TFAE

1. *Q orthogonal*
2. $\forall \boldsymbol{x} \in \mathbb{R}^n$, $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$
3. $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$

Proof of Theorem 69

1 $\implies$ **3**: Assume Q is orthogonal. Then $Q^TQ = \mathbb{I}_n$, so

$$Q\boldsymbol{x} \bullet Q\boldsymbol{y} = (Q\boldsymbol{x})^T(Q\boldsymbol{y}) = \boldsymbol{x}^TQ^TQ\boldsymbol{y} = \boldsymbol{x}^T\mathbb{I}_n\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$$

3 $\implies$ **1**: Assume $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}\ \forall \boldsymbol{x}, \boldsymbol{y}$. For standard basis vectors $\boldsymbol{e}_i, \boldsymbol{e}_j$:

$$(Q\boldsymbol{e}_i) \bullet (Q\boldsymbol{e}_j) = \boldsymbol{e}_i \bullet \boldsymbol{e}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Since $Q\boldsymbol{e}_i$ is the i-th column of Q, the columns of Q are orthonormal, so Q is orthogonal.

3 $\implies$ **2**: Assume $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$. Letting $\boldsymbol{y} = \boldsymbol{x}$ gives

$$\|Q\boldsymbol{x}\|^2 = Q\boldsymbol{x} \bullet Q\boldsymbol{x} = \boldsymbol{x} \bullet \boldsymbol{x} = \|\boldsymbol{x}\|^2 \implies \|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$$

2 $\implies$ **3**: Assume $\|Q\boldsymbol{z}\| = \|\boldsymbol{z}\|$ for all $\boldsymbol{z}$. Using $\boldsymbol{z} = \boldsymbol{x} + \boldsymbol{y}$, we expand $\|Q(\boldsymbol{x} + \boldsymbol{y})\|^2 = \|\boldsymbol{x} + \boldsymbol{y}\|^2$:

$$\|Q\boldsymbol{x}\|^2 + 2Q\boldsymbol{x} \bullet Q\boldsymbol{y} + \|Q\boldsymbol{y}\|^2 = \|\boldsymbol{x}\|^2 + 2\boldsymbol{x} \bullet \boldsymbol{y} + \|\boldsymbol{y}\|^2$$

Since $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$ and $\|Q\boldsymbol{y}\| = \|\boldsymbol{y}\|$, we get $Q\boldsymbol{x} \bullet Q\boldsymbol{y} = \boldsymbol{x} \bullet \boldsymbol{y}$. □

Theorem 70

Let $Q \in \mathcal{M}_n$ be orthogonal. Then

1. *The rows of Q form an orthonormal set*
2. *Q^{-1} orthogonal*
3. $\det Q = \pm 1$
4. $\forall \lambda \in \sigma(Q)$, $|\lambda| = 1$
5. *If $Q_2 \in \mathcal{M}_n$ also orthogonal, then QQ_2 orthogonal*

Proof of 1 and 2 in Theorem 70

1. The rows of Q form an orthonormal set

- By definition, Q is orthogonal $\iff Q^T Q = \mathbb{I}_n$
- Since Q is square, $Q^T Q = \mathbb{I}_n \implies Q^{-1} = Q^T \implies QQ^T = \mathbb{I}_n$
- This is equivalent to $(Q^T)^T Q^T = \mathbb{I}_n$, so the columns of Q^T (the rows of Q) are orthonormal □

2. Q^{-1} is orthogonal

- We must show that $(Q^{-1})^T Q^{-1} = \mathbb{I}_n$
- Using $Q^{-1} = Q^T$, we get $(Q^{-1})^T Q^{-1} = (Q^T)^T Q^T = QQ^T = \mathbb{I}_n$
- Thus Q^{-1} is an orthogonal matrix □

Proof of 3 and 5 in Theorem 70

3. $\det Q = \pm 1$

- We have $Q^T Q = \mathbb{I}_n$. Taking determinant on both sides

$$\det(Q^T Q) = \det(\mathbb{I}_n) = 1$$

- Using $\det(AB) = \det(A)\det(B)$, we get $\det(Q^T)\det(Q) = 1$
- From the general property $\det(A^T) = \det(A)$, it follows that $(\det(Q))^2 = 1$, so $\det Q = \pm 1$ □

5. If $Q_2 \in \mathcal{M}_n$ also orthogonal, then QQ_2 orthogonal

- We must show that $(QQ_2)^T(QQ_2) = \mathbb{I}_n$
- $(QQ_2)^T(QQ_2) = Q_2^T Q^T Q Q_2 = Q_2^T (Q^T Q) Q_2$
- Since Q is orthogonal, $Q^T Q = \mathbb{I}_n$, so the expression becomes $Q_2^T \mathbb{I}_n Q_2 = Q_2^T Q_2$
- Since Q_2 is orthogonal, $Q_2^T Q_2 = \mathbb{I}_n$ □

Proof of 4 in Theorem 70

4. $\forall \lambda \in \sigma(Q)$**,** $|\lambda| = 1$

Let λ be an eigenvalue of $Q \in \mathcal{M}_n$ orthogonal, i.e., $\exists \mathbb{R}^n \ni \boldsymbol{x} \neq \mathbf{0}$ s.t.

$$Q\boldsymbol{x} = \lambda \boldsymbol{x}$$

Take the norm on both sides

$$\|Q\boldsymbol{x}\| = \|\lambda \boldsymbol{x}\|$$

From 2 in Theorem 69, $\|Q\boldsymbol{x}\| = \|\boldsymbol{x}\|$ and from the properties of norms, $\|\lambda \boldsymbol{x}\| = |\lambda| \, \|\boldsymbol{x}\|$, so we have

$$\|Q\boldsymbol{x}\| = \|\lambda \boldsymbol{x}\| \iff \|\boldsymbol{x}\| = |\lambda| \, \|\boldsymbol{x}\| \iff 1 = |\lambda|$$

(we can divide by $\|\boldsymbol{x}\|$ since $\boldsymbol{x} \neq \mathbf{0}$ as an eigenvector) □

The QR decomposition

- Orthogonality and projections
- The Gram-Schmidt orthogonalisation process
- Orthogonal matrices
- **The QR factorisation & Least squares**

The QR factorisation

Theorem 71

Let $A \in \mathcal{M}_{mn}$ with LI columns. Then A can be factored as

$$A = QR$$

where $Q \in \mathcal{M}_{mn}$ has orthonormal columns and $R \in \mathcal{M}_n$ is nonsingular upper triangular

Back to least squares

So what was the point of all that..?

Theorem 72 (Least squares with QR factorisation)

$A \in \mathcal{M}_{mn}$ with linearly independent columns, $\boldsymbol{b} \in \mathbb{R}^m$. If $A = QR$ is a QR factorisation of A, then the unique least squares solution $\tilde{\boldsymbol{x}}$ of $A\boldsymbol{x} = \boldsymbol{b}$ is

$$\tilde{\boldsymbol{x}} = R^{-1}Q^T\boldsymbol{b}$$

Proof of Theorem 72

A has LI columns so

- least squares $A\boldsymbol{x} = \boldsymbol{b}$ has unique solution $\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$
- by Theorem 71, A can be written as $A = QR$ with $Q \in \mathcal{M}_{mn}$ with orthonormal columns and $R \in \mathcal{M}_n$ nonsingular and upper triangular

So

$$
\begin{aligned}
A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b} &\implies (QR)^T QR\tilde{\boldsymbol{x}} = (QR)^T \boldsymbol{b} \\
&\implies R^T Q^T QR\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies R^T \mathbb{I}_n R\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies R^T R\tilde{\boldsymbol{x}} = R^T Q^T \boldsymbol{b} \\
&\implies (R^T)^{-1} R\tilde{\boldsymbol{x}} = (R^T)^{-1} R^T Q^T \boldsymbol{b} \\
&\implies R\tilde{\boldsymbol{x}} = Q^T \boldsymbol{b} \\
&\implies \tilde{\boldsymbol{x}} = R^{-1} Q^T \boldsymbol{b} \quad \square
\end{aligned}
$$

Other applications of the QR decomposition

Focus here on least squares, but QR decomposition fundamental tool in numerical linear algebra with other important applications

- **Solving linear systems**: to solve $A\boldsymbol{x} = \boldsymbol{b}$ for a non-singular square matrix A, write $QR\boldsymbol{x} = \boldsymbol{b} \implies R\boldsymbol{x} = Q^T\boldsymbol{b}$. Since R is upper triangular, this can be quickly solved by back-substitution. This method is often more numerically stable than Gaussian elimination (called the LU decomposition)

- **Computing eigenvalues**: the **QR algorithm** is a standard method for calculating the eigenvalues of a matrix. It uses the iterative process $A_k = Q_k R_k$ and $A_{k+1} = R_k Q_k$. Under certain conditions, A_k converges to an upper triangular matrix whose diagonal entries are the eigenvalues

- **Computing determinants**: if $A = QR$, then $\det(A) = \det(Q)\det(R)$. Since Q is orthogonal, $\det(Q) = \pm 1$, and since R is upper triangular, $\det(R)$ is simply the product of its diagonal entries

Example – Solving least squares with QR (1/5)

Consider the matrix A and a vector $\boldsymbol{b}$:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}.$$

We want to find the unique least squares solution $\tilde{\boldsymbol{x}}$ to $A\boldsymbol{x} = \boldsymbol{b}$.

To do this using the QR decomposition, we must first compute Q and R.

Example – Solving least squares with QR (2/5)

We compute Q by applying the Gram-Schmidt process to the columns of A: $\boldsymbol{a}_1 = (1, 1, 0)$ and $\boldsymbol{a}_2 = (1, 0, 1)$.

First, we find the orthogonal vectors:

$$\boldsymbol{v}_1 = \boldsymbol{a}_1 = (1, 1, 0).$$

$$\boldsymbol{v}_2 = \boldsymbol{a}_2 - \frac{\boldsymbol{v}_1 \bullet \boldsymbol{a}_2}{\|\boldsymbol{v}_1\|^2}\boldsymbol{v}_1 = (1, 0, 1) - \frac{1}{2}(1, 1, 0) = \left(\frac{1}{2}, -\frac{1}{2}, 1\right).$$

Then, we normalise them to obtain the orthonormal columns of Q:

$$\boldsymbol{u}_1 = \frac{\boldsymbol{v}_1}{\|\boldsymbol{v}_1\|} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right).$$

$$\boldsymbol{u}_2 = \frac{\boldsymbol{v}_2}{\|\boldsymbol{v}_2\|} = \sqrt{\frac{2}{3}}\left(\frac{1}{2}, -\frac{1}{2}, 1\right) = \left(\frac{1}{\sqrt{6}}, -\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}\right).$$

Example – Solving least squares with QR (3/5)

Thus, the orthogonal matrix Q is:

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} \\ 0 & \frac{2}{\sqrt{6}} \end{pmatrix}.$$

To find R, recall that $A = QR$. Since Q has orthonormal columns, $Q^T Q = \mathbb{I}$. Thus:

$$Q^T A = Q^T QR = \mathbb{I}R = R.$$

We can therefore compute R directly as $Q^T A$:

$$R = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{2}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} \end{pmatrix}.$$

Example – Solving least squares with QR (4/5)

By Theorem 72, we know that $R\tilde{\boldsymbol{x}} = Q^T\boldsymbol{b}$. First, we compute $Q^T\boldsymbol{b}$:

$$Q^T\boldsymbol{b} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ \frac{7}{\sqrt{6}} \end{pmatrix}.$$

Now we solve the upper triangular system $R\tilde{\boldsymbol{x}} = Q^T\boldsymbol{b}$ via back-substitution:

$$\begin{pmatrix} \frac{2}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} \end{pmatrix} \begin{pmatrix} \tilde{x}_1 \\ \tilde{x}_2 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ \frac{7}{\sqrt{6}} \end{pmatrix}.$$

The second equation gives $\frac{3}{\sqrt{6}}\tilde{x}_2 = \frac{7}{\sqrt{6}}$, which implies $\tilde{x}_2 = \frac{7}{3}$.

Example – Solving least squares with QR (5/5)

Substituting this into the first equation yields $\frac{2}{\sqrt{2}}\tilde{x}_1 + \frac{1}{\sqrt{2}}\left(\frac{7}{3}\right) = \frac{5}{\sqrt{2}}$, which simplifies to $2\tilde{x}_1 + \frac{7}{3} = 5$. Solving for $\tilde{x}_1$ yields $\tilde{x}_1 = \frac{4}{3}$.

Thus, the least squares solution is $\tilde{\boldsymbol{x}} = \begin{pmatrix} 4/3 \\ 7/3 \end{pmatrix}$.

We can easily confirm this result in R:

```
b <- c(2, 3, 4)
QT_b <- t(Q_A) %*% b
x_tilde <- solve(R_A, QT_b)
x_tilde
```

```
##          [,1]
## [1,] 1.333333
## [2,] 2.333333
```

```
qr.coef(qr_A, b) # with the built-in solver
```

```
## [1] 1.333333 2.333333
```

Fitting the Canada census data using QR

Let us return to the Canada census data we fitted earlier using the normal equations.

We saw that computing $A^T A$ and its inverse for a quadratic fit was problematic when using the raw years, because the matrix A was badly conditioned.

This led us to shift the x-axis to start at $t = 0$.

The QR decomposition provides a much more numerically stable way to solve the least squares problem $A\boldsymbol{x} = \boldsymbol{b}$.

To illustrate this, let us try to fit the quadratic model on the unshifted raw data using the QR approach.

Setup for unshifted quadratic fit

We first load the data and set up our matrix A for a quadratic fit without shifting the years.

```
data <- read.csv("../DATA/Canada_census.csv")
data <- data[which(data$year <= 1980),]
x <- data$year
y <- data$population
A_q <- matrix(c(rep(1, length(x)), x, x^2), nr = length(x), nc = 3)
```

Normal equations failure

Recall that using the normal equations on this matrix failed because $A^T A$ is computationally singular. We can see this in R:

```
ATA <- t(A_q) %*% A_q
try(solve(ATA) %*% t(A_q) %*% y)

## Error in solve.default(ATA) :
##   system is computationally singular: reciprocal condition number = 9.166
```

However, the QR decomposition avoids forming $A^T A$, preserving numerical stability.

Solving with QR decomposition

We can compute the QR decomposition using R's built-in `qr` function, extract the Q and R components, and then use `backsolve` to solve the upper triangular system $R\boldsymbol{x} = Q^T\boldsymbol{b}$.

```
# Compute the QR decomposition of A_q
QR <- qr(A_q)
Q <- qr.Q(QR)
R <- qr.R(QR)

# Solve Rx = Q^T y
sol_qr <- backsolve(R, t(Q) %*% y)
sol_qr
```

```
##                [,1]
## [1,]  5.133645e+09
## [2,] -5.513814e+06
## [3,]  1.481439e+03
```

Direct solution with `qr.solve`

The QR decomposition successfully finds the coefficients for the unshifted data!

Alternatively, R provides the function `qr.solve` which directly solves the least squares problem using the QR decomposition under the hood:

```
sol_qr_direct <- qr.solve(A_q, y)
sol_qr_direct
```

```
## [1]  5.133645e+09 -5.513814e+06  1.481439e+03
```

This demonstrates the practical advantage of using orthogonal matrix factorisations in data science applications over the normal equations.

The QR decomposition

The SVD (singular values decomposition)

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Matrix factorisations (continued)

The singular value decomposition (known mostly by its acronym, SVD) is yet another type of factorisation/decomposition..

Singular values

Definition 73 (Singular value)

*Let $A \in \mathcal{M}_{mn}(\mathbb{R})$. The **singular values** of A are the real numbers*

$$\sigma_1 \geq \sigma_2 \geq \cdots \sigma_n \geq 0$$

that are the square roots of the eigenvalues of $A^T A$

Singular values are real and nonnegative?

Recall that $\forall A \in \mathcal{M}_{mn}$, $A^T A$ is symmetric

Theorem 74

Let $A \in \mathcal{M}_n(\mathbb{R})$ be symmetric. Then all eigenvalues of A are real

Proof of Theorem 74

$A \in \mathcal{M}_n(\mathbb{R})$ symmetric and $(\lambda, \boldsymbol{v})$ eigenpair of A, i.e, $A\boldsymbol{v} = \lambda \boldsymbol{v}$. Taking the complex conjugate, $\overline{A\boldsymbol{v}} = \overline{\lambda \boldsymbol{v}}$

Since $A \in \mathcal{M}_n(\mathbb{R})$, $\overline{A} = A$ $\qquad (z = \bar{z} \iff z \in \mathbb{R})$

So

$$A\bar{\boldsymbol{v}} = \overline{A}\bar{\boldsymbol{v}} = \overline{A\boldsymbol{v}} = \overline{\lambda \boldsymbol{v}} = \overline{\lambda}\bar{\boldsymbol{v}}$$

i.e., if $(\lambda, \boldsymbol{v})$ eigenpair, $(\bar{\lambda}, \bar{\boldsymbol{v}})$ also eigenpair

Proof of Theorem 74 (cont.)

Still assuming $A \in \mathcal{M}_n(\mathbb{R})$ symmetric and $(\lambda, \boldsymbol{v})$ eigenpair of A and using what we just proved (that $(\bar{\lambda}, \bar{\boldsymbol{v}})$ also eigenpair), take transposes

$$\begin{aligned} A\bar{\boldsymbol{v}} = \bar{\lambda}\bar{\boldsymbol{v}} &\iff (A\bar{\boldsymbol{v}})^T = (\bar{\lambda}\bar{\boldsymbol{v}})^T \\ &\iff \bar{\boldsymbol{v}}^T A^T = \bar{\lambda}\bar{\boldsymbol{v}}^T \\ &\iff \bar{\boldsymbol{v}}^T A = \bar{\lambda}\bar{\boldsymbol{v}}^T \qquad [A \text{ symmetric}] \end{aligned}$$

Let us now compute $\lambda(\bar{\boldsymbol{v}} \bullet \boldsymbol{v})$. We have

$$\begin{aligned} \lambda(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) &= \lambda\bar{\boldsymbol{v}}^T\boldsymbol{v} = \bar{\boldsymbol{v}}^T(\lambda\boldsymbol{v}) \\ &= \bar{\boldsymbol{v}}^T(A\boldsymbol{v}) = (\bar{\boldsymbol{v}}^T A)\boldsymbol{v} \\ &= (\bar{\lambda}\bar{\boldsymbol{v}}^T)\boldsymbol{v} = \bar{\lambda}(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) \\ &\iff (\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0 \end{aligned}$$

Proof of Theorem 74 (end)

We have shown

$$(\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0$$

Let

$$\boldsymbol{v} = \begin{pmatrix} a_1 + ib_1 \\ \vdots \\ a_n + ib_n \end{pmatrix}$$

Then

$$\bar{\boldsymbol{v}} = \begin{pmatrix} a_1 - ib_1 \\ \vdots \\ a_n - ib_n \end{pmatrix}$$

So

$$\bar{\boldsymbol{v}} \bullet \boldsymbol{v} = (a_1^2 + b_1^2) + \cdots + (a_n^2 + b_n^2)$$

But $\boldsymbol{v}$ eigenvector is $\neq \boldsymbol{0}$, so $\bar{\boldsymbol{v}} \bullet \boldsymbol{v} \neq 0$, so

$$(\lambda - \bar{\lambda})(\bar{\boldsymbol{v}} \bullet \boldsymbol{v}) = 0 \iff \lambda - \bar{\lambda} = 0 \iff \lambda = \bar{\lambda} \iff \lambda \in \mathbb{R} \quad \square$$

Eigenvalues of A^TA are real and nonnegative

Theorem 75

For $A \in \mathcal{M}_{mn}(\mathbb{R})$, the eigenvalues of A^TA (and AA^T) are real and nonnegative

Proof of Theorem 75

We know that for $A \in \mathcal{M}_{mn}$, A^TA symmetric and from Theorem 74, if $A \in \mathcal{M}_{mn}(\mathbb{R})$, then A^TA is symmetric, real and with real eigenvalues

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of A^TA, with $\boldsymbol{v}$ chosen so that $\|\boldsymbol{v}\| = 1$

Norms are functions $V \to \mathbb{R}_+$, so $\|A\boldsymbol{v}\|$ and $\|A\boldsymbol{v}\|^2$ are ≥ 0 and thus

$$\begin{aligned} 0 \leq \|A\boldsymbol{v}\|^2 &= (A\boldsymbol{v}) \bullet (A\boldsymbol{v}) = (A\boldsymbol{v})^T(A\boldsymbol{v}) \\ &= \boldsymbol{v}^TA^TA\boldsymbol{v} = \boldsymbol{v}^T(A^TA\boldsymbol{v}) = \boldsymbol{v}^T(\lambda\boldsymbol{v}) \\ &= \lambda(\boldsymbol{v}^T\boldsymbol{v}) = \lambda(\boldsymbol{v} \bullet \boldsymbol{v}) = \lambda\|\boldsymbol{v}\|^2 \\ &= \lambda \end{aligned}$$

Nonzero eigenvalues of A^TA and AA^T

Theorem 76

For $A \in \mathcal{M}_{mn}(\mathbb{R})$, the nonzero eigenvalues of A^TA and AA^T are the same

Proof.

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of A^TA with $\lambda \neq 0$. Then $\boldsymbol{v} \neq \mathbf{0}$ and

$$A^TA\boldsymbol{v} = \lambda\boldsymbol{v} \neq \mathbf{0}$$

Left multiply by A

$$AA^TA\boldsymbol{v} = \lambda A\boldsymbol{v}$$

Let $\boldsymbol{w} = A\boldsymbol{v}$, we thus have $AA^T\boldsymbol{w} = \lambda\boldsymbol{w}$; in other words, $A\boldsymbol{v}$ is an eigenvector of AA^T corresponding to the (nonzero) eigenvalue λ
The reverse works the same way..

□

The SVD (singular values decomposition)

Singular values

The SVD

An application of the SVD – Image compression

Computing the SVD

Applications of the SVD – Least squares

Summary – Least squares methods

The singular value decomposition (SVD)

Theorem 77 (SVD)

$A \in \mathcal{M}_{mn}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$ and $\sigma_{r+1} = \cdots = \sigma_n = 0$

Then there exists $U \in \mathcal{M}_m$ orthogonal, $V \in \mathcal{M}_n$ orthogonal and a block matrix $\Sigma \in \mathcal{M}_{mn}$ taking the form

$$\Sigma = \begin{pmatrix} D & \mathbf{0}_{r\times n-r} \\ \mathbf{0}_{m-r\times r} & \mathbf{0}_{m-r\times n-r} \end{pmatrix}$$

where

$$D = \text{diag}(\sigma_1, \ldots, \sigma_r) \in \mathcal{M}_r$$

such that

$$A = U\Sigma V^T$$

Definition 78

*We call a factorisation as in Theorem 77 the **singular value decomposition** of A. The columns of U and V are, respectively, the **left** and **right singular vectors** of A*

U and V^T are *rotation* or *reflection* matrices, Σ is a *scaling* matrix

$U \in \mathcal{M}_m$ orthogonal matrix with columns the eigenvectors of AA^T

$V \in \mathcal{M}_n$ orthogonal matrix with columns the eigenvectors of A^TA

Outer product form of the SVD

Theorem 79 (Outer product form of the SVD)

$A \in \mathcal{M}_{mn}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$ and $\sigma_{r+1} = \cdots = \sigma_n = 0$, $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$ and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r$, respectively, left and right singular vectors of A corresponding to these singular values

Then

$$A = \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^T + \cdots + \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^T \tag{2}$$

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Applications of the SVD

Many applications of the SVD, both theoretical and practical..

1. Obtaining a unique solutions to least squares when $A^T A$ singular

2. Image compression

Compressing images

Consider an image (for simplicity, assume in shades of grey). This can be stored in a matrix $A \in \mathcal{M}_{mn}$

Take the SVD of A. Then the small singular values carry information about the regions with little variation and can perhaps be omitted, whereas the large singular values carry information about more “dynamic” regions of the image

Suppose A has r nonzero singular values. For $k \leq r$, let

$$A_k = \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^T + \cdots + \sigma_k \boldsymbol{u}_k \boldsymbol{v}_k^T$$

For $k = r$ we get the usual outer product form (2) in Theorem 79

Load the image using `bmp::read.bmp`

```
my_image = bmp::read.bmp("../CODE/Julien_and_friend_1000x800.bmp")
my_image_g = pixmap::pixmapGrey(my_image)
my_image_g
```

```
## Pixmap image
##   Type          : pixmapGrey
##   Size          : 800x1000
##   Resolution    : 1x1
##   Bounding box  : 0 0 1000 800
```

Using built-in functions

We can also use the built-in function `svd` to compute the SVD of M

```
M.svd = svd(M)
```

The results are stored in a list with components `u`, `d` and `v`

Make function to recreate an image from the SVD

Given the SVD `im` of an image and a number of singular values to keep `n`, we can recreate the image using the function `compress_image`

We output the new image, but also, the amount of information required to encode this new image, as a percentage of the original image size

```
compress_image = function(im, n) {
  if (n > length(im$d)) {
    # Check that we gave a value of n within range, otherwise
    # just set to the max
    n = length(im$d)
  }
  d_tmp = im$d[1:n]
  u_tmp = im$u[,1:n]
  v_tmp = im$v[,1:n]
  # Store the results in a list (so we can return other information)
    out = list()
    # First, compute the resulting image
    out$img = mat.or.vec(nr = dim(im$u)[1], nc = dim(im$v)[1])
    for (i in 1:n) {
        out$img = out$img + d_tmp[i] * u_tmp[,i] %*% t(v_tmp[,i])
    }
```

```
    # Values of "colours" must be in [0,1], so shift and rescale
    if (min(min(out$img)) < 0 ) {
        out$img = out$img - min(min(out$img))
    }
    out$img = out$img / max(max(out$img))
    # Store some information: # points needed and % of the original
    out$nb_pixels_original = dim(im$u)[1] * dim(im$v)[2]
    out$nb_pixels_compressed = length(d_tmp) +
      dim(u_tmp)[1]*dim(u_tmp)[2] +
      dim(v_tmp)[1]*dim(v_tmp)[2]
    out$pct_of_original = out$nb_pixels_compressed /
      out$nb_pixels_original * 100
    # Return the result
    return(out)
}
```

Recreating the image

We can now recreate the image using the function `compress_image`

```
new_image = my_image_g
M.svd = svd(M)
M_tmp = compress_image(M.svd, 2)
new_image@grey = M_tmp$img
plot(new_image)
```

Using $n = 2$ singular values

Uses 0.56% of the original information

Using $n = 5$ singular values

Uses 1.41% of the original information

Using $n = 10$ singular values

Uses 2.81% of the original information

Using $n = 20$ singular values

Uses 5.63% of the original information

Using $n = 50$ singular values

Uses 14.07% of the original information

The SVD (singular values decomposition)

- Singular values
- The SVD
- An application of the SVD – Image compression
- **Computing the SVD**
- Applications of the SVD – Least squares
- Summary – Least squares methods

Computing the SVD (case of $\neq$ eigenvalues)

To compute the SVD, we use the following result

Theorem 80

Let $A \in \mathcal{M}_n$ symmetric, $(\lambda_1, \boldsymbol{u}_1)$ and $(\lambda_2, \boldsymbol{u}_2)$ be eigenpairs, $\lambda_1 \neq \lambda_2$. Then $\boldsymbol{u}_1 \bullet \boldsymbol{u}_2 = 0$

Proof of Theorem 80

$A \in \mathcal{M}_n$ symmetric, $(\lambda_1, \boldsymbol{u}_1)$ and $(\lambda_2, \boldsymbol{u}_2)$ eigenpairs with $\lambda_1 \neq \lambda_2$

$$\begin{aligned}
\lambda_1(\boldsymbol{v}_1 \bullet \boldsymbol{v}_2) &= (\lambda_1 \boldsymbol{v}_1) \bullet \boldsymbol{v}_2 \\
&= A\boldsymbol{v}_1 \bullet \boldsymbol{v}_2 \\
&= (A\boldsymbol{v}_1)^T \boldsymbol{v}_2 \\
&= \boldsymbol{v}_1^T A^T \boldsymbol{v}_2 \\
&= \boldsymbol{v}_1^T (A\boldsymbol{v}_2) \qquad [A \text{ symmetric so } A^T = A] \\
&= \boldsymbol{v}_1^T (\lambda_2 \boldsymbol{v}_2) \\
&= \lambda_2 (\boldsymbol{v}_1^T \boldsymbol{v}_2) \\
&= \lambda_2 (\boldsymbol{v}_1 \bullet \boldsymbol{v}_2)
\end{aligned}$$

So $(\lambda_1 - \lambda_2)(\boldsymbol{v}_1 \bullet \boldsymbol{v}_2) = 0$. But $\lambda_1 \neq \lambda_2$, so $\boldsymbol{v}_1 \bullet \boldsymbol{v}_2 = 0$ □

Computing the SVD (case of $\neq$ eigenvalues)

If all eigenvalues of A^TA (or AA^T) are distinct, we can use Theorem 80

1. Compute $A^TA \in \mathcal{M}_n$
2. Compute eigenvalues $\lambda_1, \ldots, \lambda_n$ of A^TA; order them as $\lambda_1 > \cdots > \lambda_n \geq 0$ ($>$ not $\geq$ since $\neq$)
3. Compute singular values $\sigma_1 = \sqrt{\lambda_1}, \ldots, \sigma_n = \sqrt{\lambda_n}$
4. Diagonal matrix D in Σ is either in $\mathcal{M}_n$ (if $\sigma_n > 0$) or in $\mathcal{M}_{n-1}$ (if $\sigma_n = 0$)

5. Since eigenvalues are distinct, Theorem 80 $\implies$ eigenvectors are orthogonal set. Compute these eigenvectors in the same order as the eigenvalues
6. Normalise them and use them to make the matrix V, i.e., $V = [\boldsymbol{v}_1 \cdots \boldsymbol{v}_n]$
7. To find the $\boldsymbol{u}_i$, compute, for $i = 1, \ldots, r$,

$$\boldsymbol{u}_i = \frac{1}{\sigma_i} A \boldsymbol{v}_i$$

and ensure that $\|\boldsymbol{u}_i\| = 1$

Why do we compute $\boldsymbol{u}_i$ this way?

Recall the definition of the SVD:

$$A = U\Sigma V^T$$

Right-multiply by V; since V is an orthogonal matrix ($V^T V = VV^T = \mathbb{I}$)

$$AV = U\Sigma V^T V = U\Sigma$$

Looking at the i-th column of both sides of this equation:

- The i-th column of AV is $A\boldsymbol{v}_i$
- The i-th column of $U\Sigma$ is $\sigma_i \boldsymbol{u}_i$

Therefore, we have $A\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$, which yields for $\sigma_i > 0$:

$$\boldsymbol{u}_i = \frac{1}{\sigma_i} A\boldsymbol{v}_i$$

Example 1: Square matrix

Let $A = \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$
2. Eigenvalues of $A^T A$: characteristic equation is $(5-\lambda)^2 - 9 = 0 \implies \lambda = 8, 2$. So $\lambda_1 = 8, \lambda_2 = 2$.
3. Singular values: $\sigma_1 = \sqrt{8} = 2\sqrt{2}, \sigma_2 = \sqrt{2}$. Matrix $\Sigma = \begin{pmatrix} 2\sqrt{2} & 0 \\ 0 & \sqrt{2} \end{pmatrix}$.

Example 1: Square matrix (continued)

4. Eigenvectors of A^TA:
   - For $\lambda_1 = 8$: null space of $\begin{pmatrix} -3 & 3 \\ 3 & -3 \end{pmatrix}$, giving $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}$
   - For $\lambda_2 = 2$: null space of $\begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$, giving $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}\begin{pmatrix} -1 \\ 1 \end{pmatrix}$

   So $V = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$.
5. Compute the $\boldsymbol{u}_i$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1}A\boldsymbol{v}_1 = \frac{1}{2\sqrt{2}}\begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}\frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{4}\begin{pmatrix} 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2}A\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}\begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}\frac{1}{\sqrt{2}}\begin{pmatrix} -1 \\ 1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

   So $U = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Example 2: Nonsquare matrix

Let $A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$
2. Eigenvalues of $A^T A$ are clearly $\lambda_1 = 2, \lambda_2 = 1$.
3. Singular values: $\sigma_1 = \sqrt{2}, \sigma_2 = 1$. The matrix Σ is 3×2, so $\Sigma = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$.
4. Eigenvectors of $A^T A$ are the standard basis vectors: $\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \boldsymbol{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

   So $V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

Example 2: Nonsquare matrix (continued)

5. Compute the $\boldsymbol{u}_i$ for $i = 1, 2$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1} A\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2} A\boldsymbol{v}_2 = \frac{1}{1} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$
6. Find $\boldsymbol{u}_3$ to complete the orthonormal basis U of $\mathcal{R}^3$. We need $\boldsymbol{u}_3$ orthogonal to $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$, with $\|\boldsymbol{u}_3\| = 1$. By inspection, $\boldsymbol{u}_3 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}$ works.
7. Finally, $U = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} \\ 0 & 1 & 0 \\ \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}$.

Return to image compression – Computations “by hand”

```
M = my_image_g@grey
MTM = t(M) %*% M
# Ensure matrix is symmetric
MTM = (MTM+t(MTM))/2
ev = eigen(MTM)
```

Given the size and nature of the entries, the matrix M^TM is symmetric only to `1e-5` precision, so we use a little trick to make it symmetric no matter what: take the average of M^TM and its transpose MM^T

Which version of the algorithm to use?

Make zero the eigenvalues that are close to zero (200 out of 1000)

```
ev$values = ev$values*(ev$values>1e-10)
```

Can we use the algorithm for all eigenvalues being distinct or do we have repeated ones?

```
any(duplicated(ev$values[ev$values>1e-10]))

## [1] FALSE
```

So we can use the standard algorithm

Computing the SVD

```
idx_positive_ev = which(ev$values>1e-10)
sv = sqrt(ev$values[idx_positive_ev])
```

Computing the SVD

Then $D = \text{diag}(\sigma_1, \ldots, \sigma_r)$, V is the matrix of normalised eigenvectors in the same order as the σ_i and for $i = 1, \ldots, r$

$$\mathbf{u}_i = \frac{1}{\sigma_i} A \mathbf{v}_i$$

ensuring that $\|\mathbf{u}_i\| = 1$

```
D = diag(sv)
V = ev$vectors[, idx_positive_ev]
# Not needed with R, other languages might not normalise
c1 = sqrt(colSums(V^2))
for (i in 1:dim(V)[2]) {
    V[,i] = V[,i]/c1[i]
}
```

Computing the SVD

Finally, we compute the $\boldsymbol{u}_i$'s

```
U = M %*% V %*% diag(1/sv)
r = length(sv)
im = list(u=U, d=sv, v=V)
```

Computing the SVD (case where some eigenvalues are $=$)

1. Compute $A^T A \in \mathcal{M}_n$
2. Compute eigenvalues $\lambda_1, \ldots, \lambda_n$ of $A^T A$; order them as $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$
3. Compute singular values $\sigma_1 = \sqrt{\lambda_1}, \ldots, \sigma_n = \sqrt{\lambda_n}$, with $r \leq n$ the index of the last positive singular value
4. For eigenvalues that are distinct, proceed as before
5. For eigenvalues with multiplicity > 1, we need to ensure that the resulting eigenvectors are LI *and* orthogonal

Dealing with eigenvalues with multiplicity > 1

When an eigenvalue has (algebraic) multiplicity > 1, e.g., characteristic polynomial contains a factor like $(\lambda - 2)^2$, things can become a little bit more complicated

The proper way to deal with this involves the so-called Jordan Normal Form (JNF, another matrix decomposition – for square matrices)

In short: not all square matrices are diagonalisable, but all square matrices admit a JNF

Sometimes, we can find several LI eigenvectors associated to the same eigenvalue. Check this. If not, need to use the following

Definition 81 (Generalised eigenvectors)

The vector $\boldsymbol{x} \neq \mathbf{0}$ is a **generalized eigenvector** of rank m of $A \in \mathcal{M}_n$ corresponding to eigenvalue λ if

$$(A - \lambda \mathbb{I})^m \boldsymbol{x} = \mathbf{0}$$

but

$$(A - \lambda \mathbb{I})^{m-1} \boldsymbol{x} \neq \mathbf{0}$$

Procedure for generalised eigenvectors

$A \in \mathcal{M}_n$ and assume λ eigenvalue with algebraic multiplicity k

Find $\boldsymbol{v}_1$, “classic" eigenvector, i.e., $\boldsymbol{v}_1 \neq \mathbf{0}$ s.t. $(A - \lambda \mathbb{I})\boldsymbol{v}_1 = \mathbf{0}$

Find generalised eigenvector $\boldsymbol{v}_2$ of rank 2 by solving for $\boldsymbol{v}_2 \neq \mathbf{0}$,

$$(A - \lambda \mathbb{I})\boldsymbol{v}_2 = \boldsymbol{v}_1$$

...

Find generalised eigenvector $\boldsymbol{v}_k$ of rank k by solving for $\boldsymbol{v}_k \neq \mathbf{0}$,

$$(A - \lambda \mathbb{I})\boldsymbol{v}_k = \boldsymbol{v}_{k-1}$$

Then $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ LI

Back to the normal procedure

With the LI eigenvectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ corresponding to λ

Apply Gram-Schmidt to get orthogonal set

For all eigenvalues with multiplicity > 1, check that you either have LI eigenvectors or do what we just did

When you are done, be back on your merry way to step 6 in the case where eigenvalues are all $\neq$

I am caricaturing a little here: there can be cases that do not work exactly like this, but this is general enough..

Example 3: Repeated eigenvalues

Let $A = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix}$.

1. Compute $A^T A = \begin{pmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$
2. Eigenvalues of $A^T A$: the characteristic polynomial is $-\lambda^3 + 6\lambda^2 - 9\lambda = 0$, which gives $\lambda_1 = 3, \lambda_2 = 3, \lambda_3 = 0$.
3. Singular values: $\sigma_1 = \sqrt{3}, \sigma_2 = \sqrt{3}, \sigma_3 = 0$.

Example 3: Repeated eigenvalues (continued)

4. Eigenvectors of A^TA:

   - For $\lambda_3 = 0$: null space of A^TA, giving $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}\begin{pmatrix}1\\1\\1\end{pmatrix}$
   - For $\lambda_1 = \lambda_2 = 3$: we need to find the null space of $A^TA - 3\mathbb{I} = \begin{pmatrix}-1 & -1 & -1\\-1 & -1 & -1\\-1 & -1 & -1\end{pmatrix}$. The eigenspace is the plane $x + y + z = 0$.

   We must choose two **orthogonal** eigenvectors in this plane.

   Let $\boldsymbol{x}_1 = \begin{pmatrix}1\\-1\\0\end{pmatrix}$. To find a second vector $\boldsymbol{x}_2$ orthogonal to $\boldsymbol{x}_1$ using Gram-Schmidt, we start with another vector in the plane, e.g., $\mathbf{y}_2 = \begin{pmatrix}1\\0\\-1\end{pmatrix}$.

   We compute $\boldsymbol{x}_2 = \mathbf{y}_2 - \mathrm{proj}_{\boldsymbol{x}_1}(\mathbf{y}_2) = \begin{pmatrix}1/2\\1/2\\-1\end{pmatrix}$, which we can scale to $\begin{pmatrix}1\\1\\-2\end{pmatrix}$.

Example 3: Repeated eigenvalues (continued)

5. Compute the $\boldsymbol{u}_i$ for $i = 1, 2$:
   - $\boldsymbol{u}_1 = \frac{1}{\sigma_1} A\boldsymbol{v}_1 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ -1 \end{pmatrix}$
   - $\boldsymbol{u}_2 = \frac{1}{\sigma_2} A\boldsymbol{v}_2 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{pmatrix} \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ 1 \\ -2 \end{pmatrix} = \frac{1}{\sqrt{18}} \begin{pmatrix} 0 \\ 3 \\ -3 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$
6. To complete the orthonormal basis U of $\mathcal{R}^3$, we find $\boldsymbol{u}_3$ orthogonal to $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$. By inspection (or cross product), $\boldsymbol{u}_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$.
7. Finally, $U = \begin{pmatrix} \frac{2}{\sqrt{6}} & 0 & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{3}} \end{pmatrix}$.

The SVD (singular values decomposition)

Singular values
The SVD
An application of the SVD – Image compression
Computing the SVD
Applications of the SVD – Least squares
Summary – Least squares methods

Pseudoinverse of a matrix

Definition 82 (Pseudoinverse)

$A = U\Sigma V^T$ an SVD for $A \in \mathcal{M}_{mn}$, where

$$\Sigma = \begin{pmatrix} D & 0 \\ 0 & 0 \end{pmatrix}, \text{ with } D = \text{diag}(\sigma_1, \ldots, \sigma_r)$$

(D contains the nonzero singular values of A ordered as usual)

The **pseudoinverse** (or **Moore-Penrose inverse**) of A is $A^+ \in \mathcal{M}_{nm}$ given by

$$A^+ = V\Sigma^+ U^T$$

with

$$\Sigma^+ = \begin{pmatrix} D^{-1} & 0 \\ 0 & 0 \end{pmatrix} \in \mathcal{M}_{nm}$$

Properties of the pseudoinverse

The pseudoinverse A^+ generalises the concept of matrix inverse and satisfies the following properties (known as the Moore-Penrose conditions):

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$
4. $(A^+A)^T = A^+A$

Other useful properties:

- If A is square and invertible, then $A^+ = A^{-1}$
- $(A^+)^+ = A$
- $(A^T)^+ = (A^+)^T$
- If A has full column rank, then $A^+ = (A^TA)^{-1}A^T$
- If A has full row rank, then $A^+ = A^T(AA^T)^{-1}$

Least squares revisited

Theorem 83

Let $A \in \mathcal{M}_{mn}$, $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{b} \in \mathbb{R}^m$. The least squares problem $A\boldsymbol{x} = \boldsymbol{b}$ has a unique least squares solution $\tilde{\boldsymbol{x}}$ of minimal length (closest to the origin) given by

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$$

where A^+ is the pseudoinverse of A

What does “minimal length” mean?

When A has *full column rank* (i.e., its columns are linearly independent), the least squares solution is unique:

$$\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$$

However, if the columns of A are *linearly dependent*, the matrix $A^T A$ is not invertible. In this case, there are **infinitely many** least squares solutions that minimise $\|A\boldsymbol{x} - \boldsymbol{b}\|$

Among all these possible solutions, the pseudoinverse A^+ selects the unique solution $\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$ that has the **smallest Euclidean norm** $\|\tilde{\boldsymbol{x}}\|$

Geometrically, it is the solution closest to the origin in $\mathbb{R}^n$

Example: Computing the pseudoinverse

Let $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. The SVD of A is $U\Sigma V^T$ with:

$$U = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \quad V = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

To find A^+, we form Σ^+ by taking the reciprocal of the non-zero singular values (leaving zeros as zero):

$$\Sigma^+ = \begin{pmatrix} 1/2 & 0 \\ 0 & 0 \end{pmatrix}$$

Then the pseudoinverse is $A^+ = V\Sigma^+ U^T$:

$$A^+ = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1/2 & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}^T = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

Example: Least squares using the pseudoinverse

Consider $A\boldsymbol{x} = \boldsymbol{b}$ where $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ and $\boldsymbol{b} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$

The system $\begin{pmatrix} x_1 + x_2 \\ x_1 + x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$ has no exact solution

The least squares approach minimises the error
$\|A\boldsymbol{x} - \boldsymbol{b}\|^2 = (x_1 + x_2 - 3)^2 + (x_1 + x_2 - 1)^2$. The minimum error is achieved when $x_1 + x_2 = 2$

Any vector $\boldsymbol{x} = \begin{pmatrix} x_1 \\ 2 - x_1 \end{pmatrix}$ is a least squares solution (e.g., $\begin{pmatrix} 2 \\ 0 \end{pmatrix}$, $\begin{pmatrix} 0 \\ 2 \end{pmatrix}$)

The **minimal length** solution is given by the pseudoinverse:

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b} = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

which is indeed the point on the line $x_1 + x_2 = 2$ closest to the origin

The SVD (singular values decomposition)

- Singular values
- The SVD
- An application of the SVD – Image compression
- Computing the SVD
- Applications of the SVD – Least squares
- **Summary – Least squares methods**

The least squares problem

Problem Statement:

Given a system $A\boldsymbol{x} = \boldsymbol{b}$ where $A \in \mathcal{M}_{mn}$, $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{b} \in \mathbb{R}^m$ (typically $m > n$), find $\tilde{\boldsymbol{x}}$ that minimizes

$$\|\boldsymbol{b} - A\boldsymbol{x}\|^2 = \sum_{i=1}^{m} (b_i - \sum_{j=1}^{n} A_{ij} x_j)^2$$

Geometric interpretation: Find the vector $A\tilde{\boldsymbol{x}}$ in the column space of A that is closest to $\boldsymbol{b}$

Solution: $A\tilde{\boldsymbol{x}} = \text{proj}_{\text{col}(A)}(\boldsymbol{b})$

Method 1: Normal equations

The normal equations:

$$A^T A\tilde{\boldsymbol{x}} = A^T \boldsymbol{b}$$

When this works:

- Always has at least one solution
- Any solution $\tilde{\boldsymbol{x}}$ to the normal equations is a least squares solution

Computational issues:

- Forming $A^T A$ can be numerically unstable
- Condition number of $A^T A$ is the square of the condition number of A
- Still useful for theoretical analysis

Method 2: when A Has linearly independent columns

Condition: $A \in \mathcal{M}_{mn}$ has linearly independent columns

Then: $A^T A$ is invertible and the least squares solution is **unique**

$$\tilde{\boldsymbol{x}} = (A^T A)^{-1} A^T \boldsymbol{b}$$

Properties:

- $A^T A \in \mathcal{M}_n$ is square, symmetric, and positive definite
- $(A^T A)^{-1} A^T$ is called the *left pseudoinverse* of A
- This gives the unique least squares solution

Drawback: Computing $(A^T A)^{-1}$ directly can be numerically unstable

Method 3: QR factorization

QR Factorization: If $A \in \mathcal{M}_{mn}$ has linearly independent columns, then

$$A = QR$$

where $Q \in \mathcal{M}_{mn}$ has orthonormal columns and $R \in \mathcal{M}_n$ is upper triangular and nonsingular

Least squares solution:

$$\tilde{\boldsymbol{x}} = R^{-1} Q^T \boldsymbol{b}$$

Advantages:

- More numerically stable than forming $A^T A$
- R is upper triangular $\Rightarrow$ solving $R\tilde{\boldsymbol{x}} = Q^T \boldsymbol{b}$ by back substitution
- Condition number of R equals condition number of A
- Gram-Schmidt or Householder reflections can compute QR factorization

Method 4: Singular Value Decomposition (SVD)

SVD: For any $A \in \mathcal{M}_{mn}$,

$$A = U\Sigma V^T$$

where $U \in \mathcal{M}_m$ orthogonal, $V \in \mathcal{M}_n$ orthogonal, $\Sigma \in \mathcal{M}_{mn}$ with $\Sigma_{ii} = \sigma_i \geq 0$ (singular values)

Pseudoinverse: $A^+ = V\Sigma^+ U^T$ where Σ^+ has $(\Sigma^+)_{ii} = 1/\sigma_i$ if $\sigma_i > 0$, else 0

Least squares solution:

$$\tilde{\boldsymbol{x}} = A^+ \boldsymbol{b}$$

Key advantages:

- Works for *any* matrix A (even when columns are linearly dependent)
- Gives the solution of *minimal length* when multiple solutions exist
- Most numerically stable method
- Reveals the rank of A through the number of non-zero singular values

When to use which method

Method	**When to use**	**Advantages/Drawbacks**
Normal equations	Theory, small problems	Simple, but unstable
$(A^T A)^{-1} A^T$	A has LI columns	Explicit formula, unstable
QR Factorization	A has LI columns	Stable, efficient
SVD	Any A, rank-deficient	Most stable, handles all cases

Use QR for well-conditioned problems with LI columns, SVD for rank-deficient or ill-conditioned problems

Principal component analysis

MATH 2740 – Mathematics of Data Science – Unit U05

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Dimensionality reduction

In the previous unit, we saw that SVD could be used to reduce the dimension of data (matrices)

Now we look at another dimensionality reduction technique: principal component analysis (PCA)

PCA is often used as a blackbox technique, here we take a look at the math behind it

What is PCA?

Linear algebraic technique

Helps reduce a complex dataset to a lower dimensional one

Non-parametric method: does not assume anything about data distribution (distribution from the statistical point of view)

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

A 2D example

Dataset (link) of height and weight of some hockey players

```
# From https://figshare.com/ndownloader/files/5303173
data = read.csv("https://github.com/julien-arino/math-of-data-science/raw/re
dim(data)

## [1] 6292   13
```

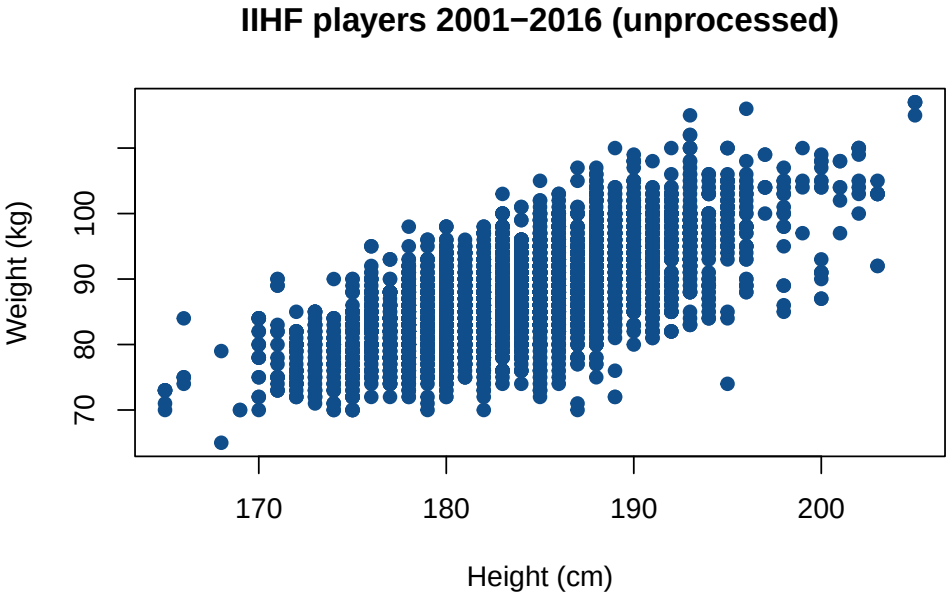
In case you are wondering, this is a database of ice hockey players at IIHF world championships, 2001-2016, assembled by the dataset's author

See some comments here

```r
head(data, n=3)
```

```
##   year country no                name position side height weight      bi
## 1 2001     RUS 10    tverdovsky oleg        D    L    185     84 1976-05
## 2 2001     RUS  2  vichnevsky vitali        D    L    188     86 1980-03
## 3 2001     RUS 26 petrochinin evgeni        D    L    182     95 1976-02
##                      club      age cohort      bmi
## 1   anaheim mighty ducks  24.95277   1976 24.54346
## 2   anaheim mighty ducks  21.11978   1980 24.33228
## 3 severstal cherepovetal  25.22930   1976 28.68011
```

As usual, it is a good idea to plot this to get a sense of the lay of the land


IIHF players 2001−2016 (unprocessed)
Weight (kg)
70
80
90
100
170
180
190
200
Height (cm)

The author of the study is interested in the evolution of weights, so it is likely that the same person will be in the dataset several times

Let us check this: first check below will be `FALSE` if the number of unique names does not match the number of rows in the dataset

```
length(unique(data$name)) == dim(data)[1]

## [1] FALSE

length(unique(data$name))

## [1] 3278

dim(data)[1]

## [1] 6292
```

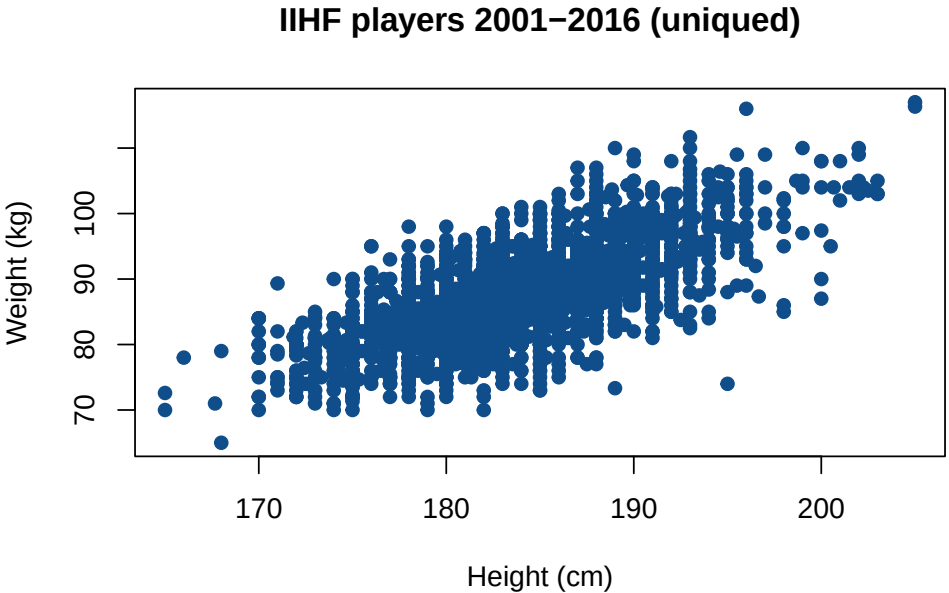
We are not interested in the evolution of weights, so we simplify: if more than one record for someone, take average of recorded weights and heights

To be extra careful, could check as well that there are no major variations on player height (homonymies?)

```
data_simplified = data.frame(name = unique(data$name))
w = c()
h = c()
for (n in data_simplified$name) {
    tmp = data[which(data$name == n),]
    h = c(h, mean(tmp$height))
    w = c(w, mean(tmp$weight))
}
data_simplified$weight = w
data_simplified$height = h
```

```r
data = data_simplified
head(data_simplified, n = 6)
```

```
##                   name weight height
## 1    tverdovsky oleg    84.0  185.0
## 2  vichnevsky vitali    86.0  188.0
## 3 petrochinin evgeni    95.0  182.0
## 4    zhdan alexander    85.5  178.5
## 5    orekhovsky oleg    88.0  175.0
## 6      zhukov sergei    92.5  193.0
```


IIHF players 2001−2016 (uniqued)
Weight (kg)
70
80
90
100
170
180
190
200
Height (cm)

Centre the data

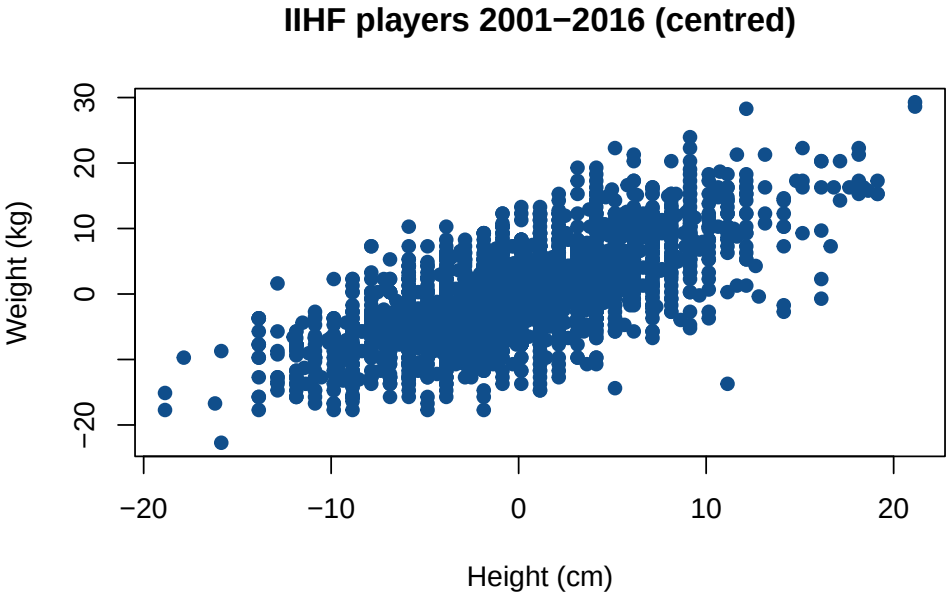
```
mean(data$weight)

## [1] 87.71555

mean(data$height)

## [1] 183.8596

data$weight.c = data$weight-mean(data$weight)
data$height.c = data$height-mean(data$height)
```


IIHF players 2001−2016 (centred)
Weight (kg)
30
20
10
0
−20
−20
−10
0
10
20
Height (cm)

Setting things up

Each player is a row in the matrix (an *observation*), each variable (*height* and *weight*) is a column

After deduplication, we have an 3278×2 matrix (actually, 3278×4 if we consider the uncentred and centred variables, but we will use one or the other, not both uncentred and centred)

We want to find what carries the most information

For this, we are going to project the information in a new basis in which the first "dimension" will carry most information (in a sense we'll define later), the second dimension will carry a little less, etc.

In order to do so, we need to learn how to change bases

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Change of basis

Theory
A simple example
Basis for a trajectory in $\mathbb{R}^2$

In the following,

$$[\boldsymbol{x}]_{\mathcal{B}}$$

denotes the coordinates of $\boldsymbol{x}$ in the basis $\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$, i.e., $[\boldsymbol{x}]_{\mathcal{B}} = (b_1, \ldots, b_n)^T$ such that $\boldsymbol{x} = b_1\boldsymbol{u}_1 + \cdots + b_n\boldsymbol{u}_n$

The aim of a change of basis is to express vectors in another coordinate system (another basis) $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$

We do so by finding a matrix allowing to move from one basis to another

Change of basis

Definition 80 (Change of basis matrix)

Let $\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ be bases of vector space V

The **change of basis matrix** $P_{\mathcal{C} \leftarrow \mathcal{B}} \in \mathcal{M}_n$ from basis $\mathcal{B}$ to $\mathcal{C}$ is

$$P_{\mathcal{C} \leftarrow \mathcal{B}} = [[\boldsymbol{u}_1]_{\mathcal{C}} \cdots [\boldsymbol{u}_n]_{\mathcal{C}}]$$

i.e., its columns are the coordinate vectors $[\boldsymbol{u}_1]_{\mathcal{C}}, \ldots, [\boldsymbol{u}_n]_{\mathcal{C}}$ of vectors in $\mathcal{B}$ with respect to $\mathcal{C}$

Example – $\mathcal{B}$ is the canonical basis of $\mathbb{R}^3$

Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis of $\mathbb{R}^3$ and $\mathcal{C} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$, with

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

To find $P_{\mathcal{C}\leftarrow\mathcal{B}} = [[\boldsymbol{i}]_{\mathcal{C}} \;\; [\boldsymbol{j}]_{\mathcal{C}} \;\; [\boldsymbol{k}]_{\mathcal{C}}]$, express each canonical vector in terms of $\mathcal{C}$:

$$\boldsymbol{i} = \boldsymbol{v}_1, \quad \boldsymbol{j} = \boldsymbol{v}_2 - \boldsymbol{v}_1, \quad \boldsymbol{k} = \boldsymbol{v}_3 - \boldsymbol{v}_2$$

$$\Longrightarrow \; [\boldsymbol{i}]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad [\boldsymbol{j}]_{\mathcal{C}} = \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{k}]_{\mathcal{C}} = \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

The change of basis matrix from $\mathcal{B}$ to $\mathcal{C}$ is

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = [[\boldsymbol{i}]_{\mathcal{C}} \;\; [\boldsymbol{j}]_{\mathcal{C}} \;\; [\boldsymbol{k}]_{\mathcal{C}}] = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

Example – $\mathcal{C}$ is the canonical basis of $\mathbb{R}^3$

Now consider the reverse: let $\mathcal{B} = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \boldsymbol{u}_3\}$ with

$$\boldsymbol{u}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

and let $\mathcal{C} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis of $\mathbb{R}^3$

Since $\mathcal{C}$ is the canonical basis, the coordinates in $\mathcal{C}$ are simply the vector itself:

$$[\boldsymbol{u}_1]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad [\boldsymbol{u}_2]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{u}_3]_{\mathcal{C}} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Thus, the change of basis matrix from $\mathcal{B}$ to the canonical basis $\mathcal{C}$ is

$$P_{\mathcal{C} \leftarrow \mathcal{B}} = [[\boldsymbol{u}_1]_{\mathcal{C}} \ \ [\boldsymbol{u}_2]_{\mathcal{C}} \ \ [\boldsymbol{u}_3]_{\mathcal{C}}] = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Theorem 81

$\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ bases of vector space V and $P_{\mathcal{C}\leftarrow\mathcal{B}}$ a change of basis matrix from $\mathcal{B}$ to $\mathcal{C}$

1. $\forall \boldsymbol{x} \in V$, $P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}} = [\boldsymbol{x}]_{\mathcal{C}}$
2. $P_{\mathcal{C}\leftarrow\mathcal{B}}$ *s.t.* $\forall \boldsymbol{x} \in V$, $P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}} = [\boldsymbol{x}]_{\mathcal{C}}$ *is* ***unique***
3. $P_{\mathcal{C}\leftarrow\mathcal{B}}$ *invertible and* $P^{-1}_{\mathcal{C}\leftarrow\mathcal{B}} = P_{\mathcal{B}\leftarrow\mathcal{C}}$

Example – Linking the two directions

Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}\}$ be the canonical basis and $\mathcal{C} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$ with

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{v}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

From our two previous examples:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \quad \text{and} \quad P_{\mathcal{B}\leftarrow\mathcal{C}} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

By item 3 in Theorem 81, $P^{-1}_{\mathcal{C}\leftarrow\mathcal{B}} = P_{\mathcal{B}\leftarrow\mathcal{C}}$. Indeed:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} P_{\mathcal{B}\leftarrow\mathcal{C}} = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \mathbb{I}$$

Since $P_{\mathcal{B}\leftarrow\mathcal{C}} = [\boldsymbol{v}_1 \;\; \boldsymbol{v}_2 \;\; \boldsymbol{v}_3]$ is immediate, we have $P_{\mathcal{C}\leftarrow\mathcal{B}} = [\boldsymbol{v}_1 \;\; \boldsymbol{v}_2 \;\; \boldsymbol{v}_3]^{-1}$

Row-reduction method for changing bases

Theorem 82

$\mathcal{B} = \{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and $\mathcal{C} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ bases of vector space V. Let $\mathcal{E}$ be any basis for V,

$$B = [[\boldsymbol{u}_1]_{\mathcal{E}}, \ldots, [\boldsymbol{u}_n]_{\mathcal{E}}] \ \textit{and}\ C = [[\boldsymbol{v}_1]_{\mathcal{E}}, \ldots, [\boldsymbol{v}_n]_{\mathcal{E}}]$$

and let $[C|B]$ be the augmented matrix constructed using C and B. Then

$$RREF\,([C|B]) = [\mathbb{I}|P_{\mathcal{C}\leftarrow\mathcal{B}}]$$

If working in $\mathbb{R}^n$, this is quite useful with $\mathcal{E}$ the standard basis of $\mathbb{R}^n$ (it does not matter if $\mathcal{B} = \mathcal{E}$)

So the question now becomes

How do we find what new basis to look at our data in?

(Changing the basis does not change the data, just the view you have of it)

(Think of what happens when you do a headstand.. your up becomes down, your right and left switch, but the world does not change, just your view of it)

(Changes of bases are *fundamental* operations in Science)

Change of basis

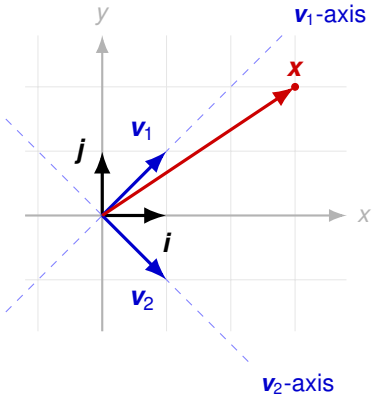
Theory
A simple example
Basis for a trajectory in $\mathbb{R}^2$

Example: change of basis

Problem: Find the change of basis matrix from basis $\mathcal{B}$ to basis $\mathcal{C}$ in $\mathbb{R}^2$, where

$$\mathcal{B} = \left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\} \quad \text{and} \quad \mathcal{C} = \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}$$

Then use this matrix to find the coordinates of $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ in basis $\mathcal{C}$



Step 1 – Set up the matrices

- $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}\}$ is the standard basis of $\mathbb{R}^2$, so using $\mathcal{E} = \mathcal{B}$, we have

$$[\boldsymbol{i}]_{\mathcal{E}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad [\boldsymbol{j}]_{\mathcal{E}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- For $\mathcal{C}$:

$$[\boldsymbol{v}_1]_{\mathcal{E}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad [\boldsymbol{v}_2]_{\mathcal{E}} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Step 2 – Row reduce

Using Theorem 82, we form the augmented matrix $[C|B]$:

$$[C|B] = \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right]$$

Row reduce to RREF:

$$\left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 1 & -1 & 0 & 1 \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - R_1} \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 0 & -2 & -1 & 1 \end{array}\right]$$

$$\xrightarrow{R_2 \leftarrow -\frac{1}{2}R_2} \left[\begin{array}{cc|cc} 1 & 1 & 1 & 0 \\ 0 & 1 & \frac{1}{2} & -\frac{1}{2} \end{array}\right]$$

$$\xrightarrow{R_1 \leftarrow R_1 - R_2} \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & 1 & \frac{1}{2} & -\frac{1}{2} \end{array}\right]$$

Step 3 – Extract the change of basis matrix

From the RREF form $[\mathbb{I}|P_{\mathcal{C}\leftarrow\mathcal{B}}]$, we get:

$$P_{\mathcal{C}\leftarrow\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$$

Verification: Let's check that this matrix works correctly.

- $[\boldsymbol{i}]_{\mathcal{C}} = P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{i}]_{\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix}$
- Check: $\frac{1}{2}\begin{pmatrix} 1 \\ 1 \end{pmatrix} + \frac{1}{2}\begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

Step 4 – Coordinates of $\boldsymbol{x}$ in basis $\mathcal{C}$

Now we find $[\boldsymbol{x}]_{\mathcal{C}}$ for $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$:

$$[\boldsymbol{x}]_{\mathcal{C}} = P_{\mathcal{C} \leftarrow \mathcal{B}} [\boldsymbol{x}]_{\mathcal{B}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$$

Verification: Check that this gives us back the original vector:

$$\frac{5}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} \frac{5}{2} + \frac{1}{2} \\ \frac{5}{2} - \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Answer: The coordinates of $\boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ in basis $\mathcal{C}$ are $[\boldsymbol{x}]_{\mathcal{C}} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$

Alternative method – Direct calculation

We could also solve this directly by setting up the system:

$$\boldsymbol{x} = c_1 \boldsymbol{v}_1 + c_2 \boldsymbol{v}_2$$

$$\begin{pmatrix} 3 \\ 2 \end{pmatrix} = c_1 \begin{pmatrix} 1 \\ 1 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

This gives us the system:

$$\begin{aligned} c_1 + c_2 &= 3 \\ c_1 - c_2 &= 2 \end{aligned}$$

Solving: $c_1 = \frac{5}{2}$, $c_2 = \frac{1}{2}$

This confirms our result: $[\boldsymbol{x}]_{\mathcal{C}} = \begin{pmatrix} \frac{5}{2} \\ \frac{1}{2} \end{pmatrix}$

Computation in R – Finding $P_{\mathcal{C}\leftarrow\mathcal{B}}$

Set up $C = [\boldsymbol{v}_1, \boldsymbol{v}_2]$, $B = [\boldsymbol{i}, \boldsymbol{j}]$, form the augmented matrix $[C|B]$ and row reduce:

```
C = matrix(c(1, 1, 1, -1), nrow = 2)
B = diag(2)
CB = cbind(C, B)
(R = pracma::rref(CB))
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    0  0.5  0.5
## [2,]    0    1  0.5 -0.5
```

```
(P = R[, 3:4])
```

```
##      [,1] [,2]
## [1,]  0.5  0.5
## [2,]  0.5 -0.5
```

Computation in R – Coordinates of $\boldsymbol{x}$

Now compute $[\boldsymbol{x}]_{\mathcal{C}} = P_{\mathcal{C}\leftarrow\mathcal{B}}[\boldsymbol{x}]_{\mathcal{B}}$ and verify that $C[\boldsymbol{x}]_{\mathcal{C}} = \boldsymbol{x}$:

```
x = c(3, 2)
(x_C = P %*% x)

##      [,1]
## [1,]  2.5
## [2,]  0.5

# Verification: reconstruct x in the canonical basis
C %*% x_C

##      [,1]
## [1,]    3
## [2,]    2
```

Change of basis

Theory

A simple example

Basis for a trajectory in $\mathbb{R}^2$

Example – A trajectory in $\mathbb{R}^2$

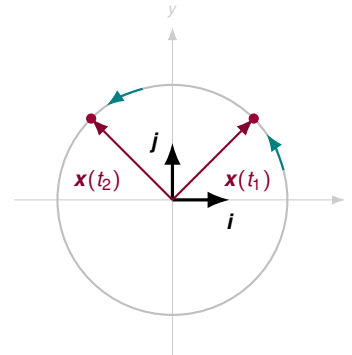
Let $\mathcal{B} = \{\boldsymbol{i}, \boldsymbol{j}\}$ be the canonical basis of $\mathbb{R}^2$.

Consider a trajectory along the unit circle:

$$\boldsymbol{x}(t) = \begin{pmatrix} \cos t \\ \sin t \end{pmatrix}, \quad t \in \mathbb{R}$$

This is a function $\boldsymbol{x} : \mathbb{R} \to \mathbb{R}^2$ that we call a **parametric equation**:

- t is the parameter (often interpreted as time)
- For each t, $\boldsymbol{x}(t)$ is a point in $\mathbb{R}^2$; as t varies, $\boldsymbol{x}(t)$ traces out the trajectory



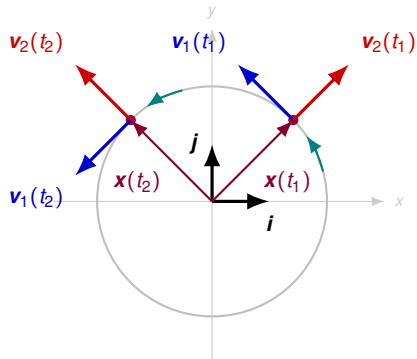
A moving basis along the trajectory

At each point $\boldsymbol{x}(t)$ on the trajectory, define a moving basis $\mathcal{C}(t) = \{\boldsymbol{v}_1(t), \boldsymbol{v}_2(t)\}$:

- $\boldsymbol{v}_1(t) = \boldsymbol{x}'(t) = \begin{pmatrix} -\sin t \\ \cos t \end{pmatrix}$ (unit tangent)
- $\boldsymbol{v}_2(t) = \begin{pmatrix} \cos t \\ \sin t \end{pmatrix}$ (outward unit normal)

Since $\boldsymbol{v}_1(t) \bullet \boldsymbol{v}_2(t) = 0$ and $\|\boldsymbol{v}_1\| = \|\boldsymbol{v}_2\| = 1$, $\mathcal{C}(t)$ is an orthonormal basis for all t

As t changes, the basis $\{\boldsymbol{v}_1(t), \boldsymbol{v}_2(t)\}$ moves and rotates along with $\boldsymbol{x}(t)$



Coordinates in the moving basis

The change of basis matrix from $\mathcal{C}(t)$ to the canonical basis $\mathcal{B}$ is

$$P_{\mathcal{B}\leftarrow\mathcal{C}(t)} = [\boldsymbol{v}_1(t) \;\; \boldsymbol{v}_2(t)] = \begin{pmatrix} -\sin t & \cos t \\ \cos t & \sin t \end{pmatrix}$$

Since $P_{\mathcal{B}\leftarrow\mathcal{C}(t)}$ is symmetric and orthogonal, the change of basis matrix to $\mathcal{C}(t)$ is:

$$P_{\mathcal{C}(t)\leftarrow\mathcal{B}} = P^{-1}_{\mathcal{B}\leftarrow\mathcal{C}(t)} = P^{T}_{\mathcal{B}\leftarrow\mathcal{C}(t)} = \begin{pmatrix} -\sin t & \cos t \\ \cos t & \sin t \end{pmatrix}$$

In the moving basis $\mathcal{C}(t)$, the coordinates of position and velocity are:

$$[\boldsymbol{x}(t)]_{\mathcal{C}(t)} = P_{\mathcal{C}(t)\leftarrow\mathcal{B}} \begin{pmatrix} \cos t \\ \sin t \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad [\boldsymbol{x}'(t)]_{\mathcal{C}(t)} = P_{\mathcal{C}(t)\leftarrow\mathcal{B}} \begin{pmatrix} -\sin t \\ \cos t \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

In the local frame, the particle is always at distance 1 along the normal axis with velocity 1 along the tangent

Choosing the inward normal $-\boldsymbol{v}_2(t)$ *simply gives* $[\boldsymbol{x}(t)]_{\mathcal{C}(t)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Why probability?

We said earlier that we would look for a basis in which the first dimension carries most information

But how do we define *information*?

We use concepts from probability and statistics to do so

A good measure of information is *variance* (how much data varies around the mean)

Brief “review” of some probability concepts

Proper definition of *probability* requires to use *measure theory*.. will not get into details here

A **random variable** X is a *measurable* function $X : \Omega \to E$, where Ω is a set of outcomes (*sample space*) and E is a measurable space

$\mathbb{P}(X \in S \subseteq E) = \mathbb{P}(\omega \in \Omega | X(\omega) \in S)$

Distribution function of a r.v., $F(x) = \mathbb{P}(X \leq x)$, describes the distribution of a r.v.

R.v. can be discrete or continuous or .. other things.

Variance

Definition 83 (Variance)

Let X be a random variable. The **variance** of X is given by

$$\operatorname{Var} X = E\left[(X - E(X))^2\right]$$

where E is the expected value

Covariance

Definition 84 (Covariance)

Let X, Y be jointly distributed random variables. The **covariance** of X and Y is given by

$$\operatorname{cov}(X, Y) = E\left[(X - E(X))\,(Y - E(Y))\right]$$

Note that $\operatorname{cov}(X, X) = E\left[(X - E(X))^2\right] = \operatorname{Var} X$

In practice: “true law” versus “observation”

In statistics: we reason on the *true law* of distributions, but we usually have only access to a sample

We then use **estimators** to .. estimate the value of a parameter, e.g., the mean, variance and covariance

Definition 85 (Unbiased estimators of the mean and variance)

Let $x_1, \ldots, x_n$ be data points (the *sample*) and

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

be the **mean** of the data. An unbiased estimator of the variance of the sample is

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2$$

Unbiased: the expected value of the estimator equals the true parameter value ($E(\bar{x}) = E(X)$ and $E(\sigma^2) = \text{Var } X$): on average, it does not systematically overestimate or underestimate the parameter

Variance of hockey players' height and weight

We compute the sample variance in R using the function `var`:

```
var(data$height)

## [1] 29.66176

var(data$weight)

## [1] 47.81112
```

The variance is about 29.7 cm^2 for height and 47.8 kg^2 for weight

Normalising does not change the variance

Centring (normalising) shifts data by a constant, leaving spread unchanged:

$$\text{Var}\,(X - c) = \text{Var}\,X$$

Indeed, computing the variance using the centred values:

```
var(data$height.c)
```

```
## [1] 29.66176
```

```
var(data$weight.c)
```

```
## [1] 47.81112
```

The variance is identical to that of the uncentred data

Definition 86 (Unbiased estimator of the covariance - 2D case)

Let $(x_1, y_1), \ldots, (x_n, y_n)$ be data points,

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \text{ and } \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

be the means of the data. An unbiased estimator of the covariance of the sample is

$$\text{cov}(x, y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

Computation of covariance in R – “By hand” and with a function

We compute the means and sum directly from the definition:

```
n = length(data$height)
x_bar = sum(data$height) / n
y_bar = sum(data$weight) / n
sum((data$height - x_bar) * (data$weight - y_bar)) / (n - 1)

## [1] 26.63506

cov(data$height, data$weight)

## [1] 26.63506
```

The covariance is about 26.6 cm · kg, matching R’s built-in `cov`

What does covariance do?

Variance explains how data disperses around the mean, in a 1-D case

Covariance measures the relationship between two dimensions. E.g., height and weight

More than the exact value, the sign is important:

- $\text{cov}(X, Y) > 0$: both dimensions change in the same "direction"; e.g., larger height usually means higher weight
- $\text{cov}(X, Y) < 0$: both dimensions change in reverse directions; e.g., time spent on social media and performance in this class
- $\text{cov}(X, Y) = 0$: the dimensions are independent from one another; e.g., sex/gender and "intelligence"

The covariance matrix

Definition 87 (Covariance matrix)

Suppose p random variables $X_1, \ldots, X_p$. Then the covariance matrix is the symmetric matrix

$$\begin{pmatrix} \text{Var } X_1 & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p) \\ \text{cov}(X_1, X_2) & \text{Var } X_2 & \cdots & \text{cov}(X_2, X_p) \\ \vdots & \vdots & & \vdots \\ \text{cov}(X_1, X_p) & \text{cov}(X_2, X_p) & \cdots & \text{Var } X_p \end{pmatrix}$$

The actual form of the matrix is below but the form above stems from the properties of covariance

$$\begin{pmatrix} \text{cov}(X_1, X_1) & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p) \\ \text{cov}(X_2, X_1) & \text{cov}(X_2, X_2) & \cdots & \text{cov}(X_2, X_p) \\ \vdots & \vdots & & \vdots \\ \text{cov}(X_p, X_1) & \text{cov}(X_p, X_2) & \cdots & \text{cov}(X_p, X_p) \end{pmatrix}$$

Sample covariance matrix

$X \in \mathcal{M}_{mp}$ the data, then the (sample) covariance matrix S_X takes the form

$$S_X = \frac{1}{n-1} X^T X$$

where the data is centred!

Sometimes you will see $S_X = 1/(n-1)XX^T$. This is for matrices with observations in columns and variables in rows. Just remember that you want the covariance matrix to have size the number of variables, not observations, this will give you the order in which to take the product

Covariance matrix of hockey players in R

When passed a data frame or matrix, `cov` computes the covariance matrix:

```
cov(data[, c("height", "weight")])

##           height   weight
## height 29.66176 26.63506
## weight 26.63506 47.81112
```

The diagonal entries are the variances (Var (height) ≈ 29.7, Var (weight) ≈ 47.8) and the off-diagonal entries are cov(height, weight) ≈ 26.6

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Setting things up

I will use notation (mostly) as in Joliffe's *Principal Component Analysis* (PDF of older version available for free from UofM Libraries)

$\boldsymbol{x} = (x_1, \ldots, x_p)$ vector of p random variables

We seek a linear function $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ with maximum variance, where $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$, i.e.,

$$\boldsymbol{\alpha}_1^T \boldsymbol{x} = \sum_{j=1}^{p} \alpha_{1j} x_j$$

Then we seek a linear function $\boldsymbol{\alpha}_2^T \boldsymbol{x}$ with maximum variance, uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$

And we continue...

At *k*th stage, we find a linear function $\boldsymbol{\alpha}_k^T \boldsymbol{x}$ with maximum variance, uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}, \ldots, \boldsymbol{\alpha}_{k-1}^T \boldsymbol{x}$

$\boldsymbol{\alpha}_i^T \boldsymbol{x}$ is the *i*th **principal component** (PC)

Case of known covariance matrix

Suppose we know Σ, covariance matrix of $\boldsymbol{x}$ (i.e., typically: we know $\boldsymbol{x}$)

Then the kth PC is

$$z_k = \boldsymbol{\alpha}_k^T \boldsymbol{x}$$

where $\boldsymbol{\alpha}_k$ is an eigenvector of Σ corresponding to the kth largest eigenvalue λ_k

If, additionally, $\|\boldsymbol{\alpha}_k\| = \boldsymbol{\alpha}_k^T \boldsymbol{\alpha} = 1$, then $\lambda_k = \text{Var } z_k$

Why is that?

Let us start with

$$\boldsymbol{\alpha}_1^T \boldsymbol{x}$$

We want maximum variance, where $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$, i.e.,

$$\boldsymbol{\alpha}_1^T \boldsymbol{x} = \sum_{j=1}^{p} \alpha_{1j} x_j$$

with the constraint that $\|\boldsymbol{\alpha}_1\| = 1$

We have

$$\text{Var } \boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

Objective

We want to maximise Var $\boldsymbol{\alpha}_1^T \boldsymbol{x}$, i.e.,

$$\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

under the constraint that $\|\boldsymbol{\alpha}_1\| = 1$

$\Longrightarrow$ use **Lagrange multipliers**

Maximisation using Lagrange multipliers

(A.k.a. super-brief intro to multivariable calculus)

We want the max of $f(x_1, \ldots, x_n)$ under the constraint $g(x_1, \ldots, x_n) = k$

1. Solve

$$\nabla f(x_1, \ldots, x_n) = \lambda \nabla g(x_1, \ldots, x_n)$$
$$g(x_1, \ldots, x_n) = k$$

   where $\nabla = (\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n})$ is the **gradient operator**
2. Plug all solutions into $f(x_1, \ldots, x_n)$ and find maximum values (provided values exist and $\nabla g \neq \mathbf{0}$ there)

λ is the **Lagrange multiplier**

The gradient

(Continuing our super-brief intro to multivariable calculus)

$f : \mathbb{R}^n \to \mathbb{R}$ function of several variables, $\nabla = \left(\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n}\right)$ the gradient operator

Then

$$\nabla f = \left(\frac{\partial}{\partial x_1} f, \ldots, \frac{\partial}{\partial x_n} f\right)$$

So ∇f is a *vector-valued* function, $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$; also written as

$$\nabla f = f_{x_1}(x_1, \ldots, x_n)\boldsymbol{e}_1 + \cdots f_{x_n}(x_1, \ldots, x_n)\boldsymbol{e}_n$$

where f_{x_i} is the partial derivative of f with respect to x_i and $\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n\}$ is the standard basis of $\mathbb{R}^n$

Bear with me..

(You may experience a brief period of discomfort)

$\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$ and $\|\boldsymbol{\alpha}_1\|^2 = \boldsymbol{\alpha}_1^T \boldsymbol{\alpha_1}$ are functions of $\boldsymbol{\alpha}_1 = (\alpha_{11}, \ldots, \alpha_{1p})$

In the notation of the previous slide, we want the max of

$$f(\alpha_{11}, \ldots, \alpha_{1p}) := \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$$

under the constraint that

$$g(\alpha_{11}, \ldots, \alpha_{1p}) := \boldsymbol{\alpha}_1^T \boldsymbol{\alpha_1} = 1$$

and with gradient operator

$$\nabla = \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right)$$

Effect of ∇ on g

g is easiest to see:

$$\begin{aligned}
\nabla g(\alpha_{11}, \ldots, \alpha_{1p}) &= \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right) (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} \alpha_{11} \\ \vdots \\ \alpha_{1p} \end{pmatrix} \\
&= \left(\frac{\partial}{\partial \alpha_{11}}, \ldots, \frac{\partial}{\partial \alpha_{1p}} \right) \left(\alpha_{11}^2 + \cdots + \alpha_{1p}^2 \right) \\
&= (2\alpha_{11}, \ldots, 2\alpha_{1p}) \\
&= 2\boldsymbol{\alpha}_1
\end{aligned}$$

(And that's a general result: $\nabla \|\boldsymbol{x}\|_2^2 = 2\boldsymbol{x}$ with $\|\cdot\|_2$ the Euclidean norm)

Effect of ∇ on f

Expand (write $\Sigma = [s_{ij}]$ and do not exploit symmetry)

$$\begin{aligned}
\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} s_{11} & s_{12} & \cdots & s_{1p} \\ s_{21} & s_{22} & \cdots & s_{2p} \\ \vdots & \vdots & & \vdots \\ s_{p1} & s_{p2} & & s_{pp} \end{pmatrix} \begin{pmatrix} \alpha_{11} \\ \alpha_{12} \\ \vdots \\ \alpha_{1p} \end{pmatrix} \\
&= (\alpha_{11}, \ldots, \alpha_{1p}) \begin{pmatrix} s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p} \\ s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p} \\ \vdots \\ s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p} \end{pmatrix} \\
&= (s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\alpha_{11} \\
&\quad + (s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p})\alpha_{12} \\
&\quad \vdots \\
&\quad + (s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p})\alpha_{1p}
\end{aligned}$$

We have

$$\begin{aligned}\boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = &(s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\alpha_{11} \\ &+ (s_{21}\alpha_{11} + s_{22}\alpha_{12} + \cdots + s_{2p}\alpha_{1p})\alpha_{12} \\ &\vdots \\ &+ (s_{p1}\alpha_{11} + s_{p2}\alpha_{12} + \cdots + s_{pp}\alpha_{1p})\alpha_{1p}\end{aligned}$$

$$\begin{aligned}\implies \frac{\partial}{\partial \alpha_{11}} \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= (s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p}) + s_{11}\alpha_{11} \\ &\quad + s_{21}\alpha_{12} + \cdots + s_{p1}\alpha_{1p} \\ &= s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p} \\ &\quad + s_{11}\alpha_{11} + s_{21}\alpha_{12} + \cdots + s_{p1}\alpha_{1p} \\ &= 2(s_{11}\alpha_{11} + s_{12}\alpha_{12} + \cdots + s_{1p}\alpha_{1p})\end{aligned}$$

(last equality stems from symmetry of Σ)

In general, for $i = 1, \ldots, p$,

$$\begin{aligned}\frac{\partial}{\partial \alpha_{1i}} \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 &= s_{i1}\alpha_{11} + s_{i2}\alpha_{12} + \cdots + s_{ip}\alpha_{1p} \\ &\quad + s_{i1}\alpha_{11} + s_{2i}\alpha_{12} + \cdots + s_{pi}\alpha_{1p} \\ &= 2(s_{i1}\alpha_{11} + s_{i2}\alpha_{12} + \cdots + s_{ip}\alpha_{1p})\end{aligned}$$

(because of symmetry of Σ)

As a consequence,

$$\nabla \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = 2\Sigma \boldsymbol{\alpha}_1$$

So solving

$$\nabla f(x_1, \ldots, x_n) = \lambda \nabla g(x_1, \ldots, x_n)$$

means solving

$$2\Sigma\boldsymbol{\alpha}_1 = \lambda 2\boldsymbol{\alpha}_1$$

i.e.,

$$\Sigma\boldsymbol{\alpha}_1 = \lambda\boldsymbol{\alpha}_1$$

$\implies$ $(\lambda, \boldsymbol{\alpha}_1)$ eigenpair of Σ, with $\boldsymbol{\alpha}_1$ having unit length

Picking the right eigenvalue

$(\lambda, \boldsymbol{\alpha}_1)$ eigenpair of Σ, with $\boldsymbol{\alpha}_1$ having unit length

But which λ to choose?

Recall that we want Var $\boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1$ maximal

We have

$$\text{Var } \boldsymbol{\alpha}_1^T \boldsymbol{x} = \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_1 = \boldsymbol{\alpha}_1^T (\Sigma \boldsymbol{\alpha}_1) = \boldsymbol{\alpha}_1^T (\lambda \boldsymbol{\alpha}_1) = \lambda (\boldsymbol{\alpha}_1^T \boldsymbol{\alpha}_1) = \lambda$$

$\Longrightarrow$ we pick $\lambda = \lambda_1$, the largest eigenvalue (covariance matrix symmetric so eigenvalues real)

What we have this far..

The first principal component is $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ and has variance λ_1, where λ_1 the largest eigenvalue of Σ and $\boldsymbol{\alpha}_1$ an associated eigenvector with $\|\boldsymbol{\alpha}_1\| = 1$

We want the second principal component to be *uncorrelated* with $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ and to have maximum variance Var $\boldsymbol{\alpha}_2^T \boldsymbol{x} = \boldsymbol{\alpha}_2^T \Sigma \boldsymbol{\alpha}_2$, under the constraint that $\|\boldsymbol{\alpha}_2\| = 1$

$\boldsymbol{\alpha}_2^T \boldsymbol{x}$ uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ if $\text{cov}(\boldsymbol{\alpha}_1^T \boldsymbol{x}, \boldsymbol{\alpha}_2^T \boldsymbol{x}) = 0$

We have

$$\begin{aligned}
\text{cov}(\boldsymbol{\alpha}_1^T \boldsymbol{x}, \boldsymbol{\alpha}_2^T \boldsymbol{x}) &= \boldsymbol{\alpha}_1^T \Sigma \boldsymbol{\alpha}_2 \\
&= \boldsymbol{\alpha}_2^T \Sigma^T \boldsymbol{\alpha}_1 \\
&= \boldsymbol{\alpha}_2^T \Sigma \boldsymbol{\alpha}_1 \quad [\Sigma \text{ symmetric}] \\
&= \boldsymbol{\alpha}_2^T (\lambda_1 \boldsymbol{\alpha}_1) \\
&= \lambda \boldsymbol{\alpha}_2^T \boldsymbol{\alpha}_1
\end{aligned}$$

So $\boldsymbol{\alpha}_2^T \boldsymbol{x}$ uncorrelated to $\boldsymbol{\alpha}_1^T \boldsymbol{x}$ if $\boldsymbol{\alpha}_1 \perp \boldsymbol{\alpha}_2$

This is beginning to sound a lot like Gram-Schmidt, no?

In short

Take whatever covariance matrix is available to you (known Σ or sample S_X) – assume sample from now on for simplicity

For $i = 1, \ldots, p$, the ith principal component is

$$z_i = \boldsymbol{v}_i^T \boldsymbol{x}$$

where $\boldsymbol{v}_i$ eigenvector of S_X associated to the ith largest eigenvalue λ_i

If $\boldsymbol{v}_i$ is normalised, then $\lambda_i = \text{Var } z_k$

A running example: hockey players

Change of basis

A crash course on probability

Back to PCA

Hockey players

Covariance

The function `cov` returns the covariance of two samples

Note that the functions deals equally well with data that is not centred as with data that is centred

```
cov(data$height, data$weight)

## [1] 25.98646

cov(data$height.c, data$weight.c)

## [1] 25.98646
```

Covariance matrix

As we could see from plotting the data, there is a positive linear relationship between the two variables

Let us compute the sample covariance matrix

```
X = as.matrix(data[,c("height.c", "weight.c")])
S = 1/(dim(X)[1]-1)*t(X) %*% X
S

##           height.c weight.c
## height.c 28.98592 25.98646
## weight.c 25.98646 48.49557
```

Covariance matrix

The off-diagonal entries do match the computed covariance. Let us check that the variances are indeed a match too.

```
var(X[,1])

## [1] 28.98592

var(X[,2])

## [1] 48.49557
```

Hey, that works. Is math not cool? ;)

Principal components

Now compute the principal components. We need eigenvalues and eigenvectors

```
ev = eigen(S)
ev
```

```
## eigen() decomposition
## $values
## [1] 66.49777 10.98372
##
## $vectors
##           [,1]       [,2]
## [1,] 0.5694576 -0.8220207
## [2,] 0.8220207  0.5694576
```

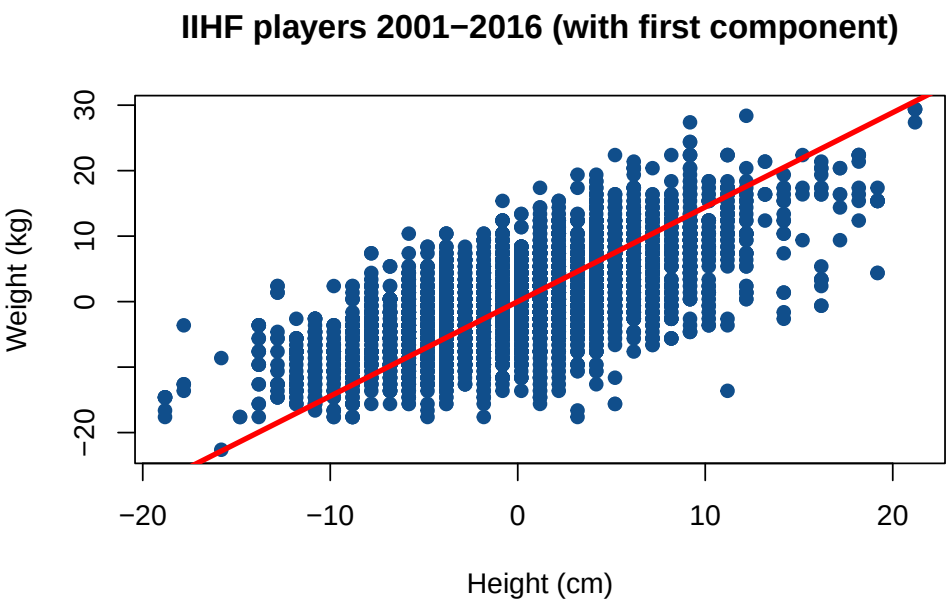
(`eigen` returns eigenvalues sorted in decreasing order and normalised eigenvectors)

First principal component

Let us plot this first eigenvector (well, the line carrying this first eigenvector)

To use the function `abline`, we need to give the coefficients of the line in the form of (intercept,slope). Intercept is easy, as the line goes through the origin (by construction and because we have centred the data). The slope is also quite simple..

```
plot(data$height.c, data$weight.c,
    pch = 19, col = "dodgerblue4",
    main = "IIHF players 2001-2016 (with first component)",
    xlab = "Height (cm)", ylab = "Weight (kg)")
abline(a = 0, b = ev$vectors[2,1]/ev$vectors[1,1],
       col = "red", lwd = 3)
```


IIHF players 2001−2016 (with first component)
Weight (kg)
−20
0
10
20
30
−20
−10
0
10
20
Height (cm)

Rotating the data

Let us rotate the data so that the red line becomes the x-axis

To do that, we use a rotation matrix

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

To find the angle θ, recall that $\tan\theta$ is equal to opposite length over adjacent length, i.e.,

$$\tan\theta = \frac{\texttt{ev\$vectors}[2,1]}{\texttt{ev\$vectors}[1,1]}$$

So we just use the $\arctan$ of this

Note that angles are in radians

Rotating the data

```
theta = atan(ev$vectors[2,1]/ev$vectors[1,1])
theta
```

```
## [1] 0.9649505
```

```
R_theta = matrix(c(cos(theta), -sin(theta),
                   sin(theta), cos(theta)),
                 nr = 2, byrow = TRUE)
R_theta
```

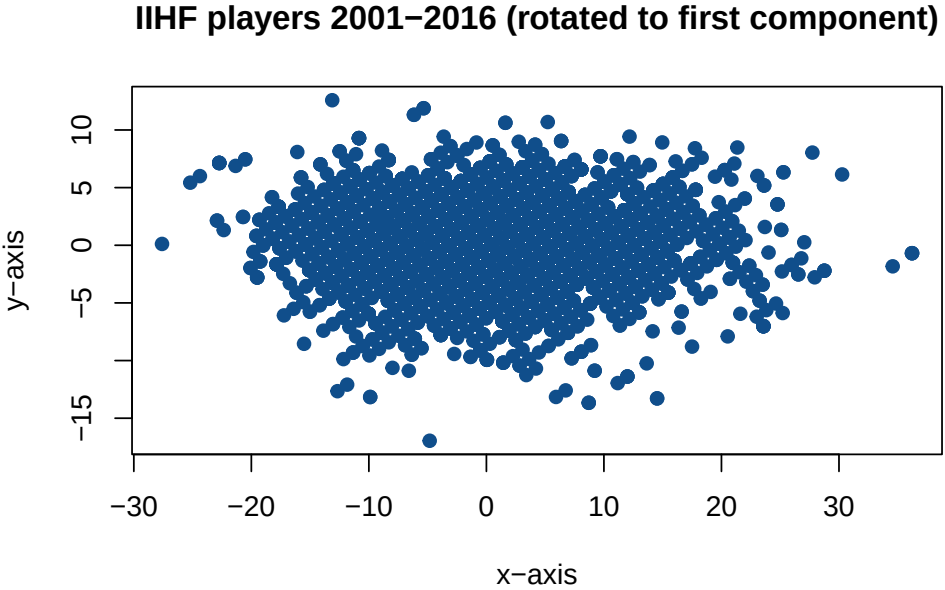
```
##           [,1]       [,2]
## [1,] 0.5694576 -0.8220207
## [2,] 0.8220207  0.5694576
```

Rotating the data

And now we rotate the points

(In this case, we think of the points as vectors, of course)

```
tmp_in = matrix(c(data$weight.c, data$height.c),
                nc = 2)
tmp_out = c()
for (i in 1:dim(tmp_in)[1]) {
    tmp_out = rbind(tmp_out,
                    t(R_theta %*% tmp_in[i,]))
}
data$weight.c_r = tmp_out[,1]
data$height.c_r = tmp_out[,2]
```

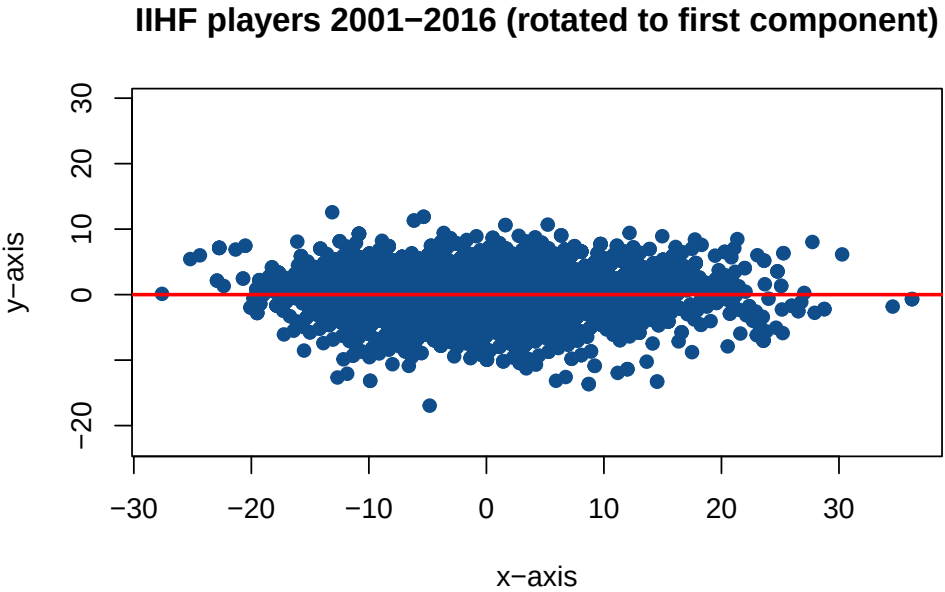

IIHF players 2001−2016 (rotated to first component)
y−axis
10
5
0
−5
−15
−30
−20
−10
0
10
20
30
x−axis

Principal components

Note that the axes have changed quite a lot, hence the very different aspect

Let us plot with the same range as for the non-rotated data for the y-axis

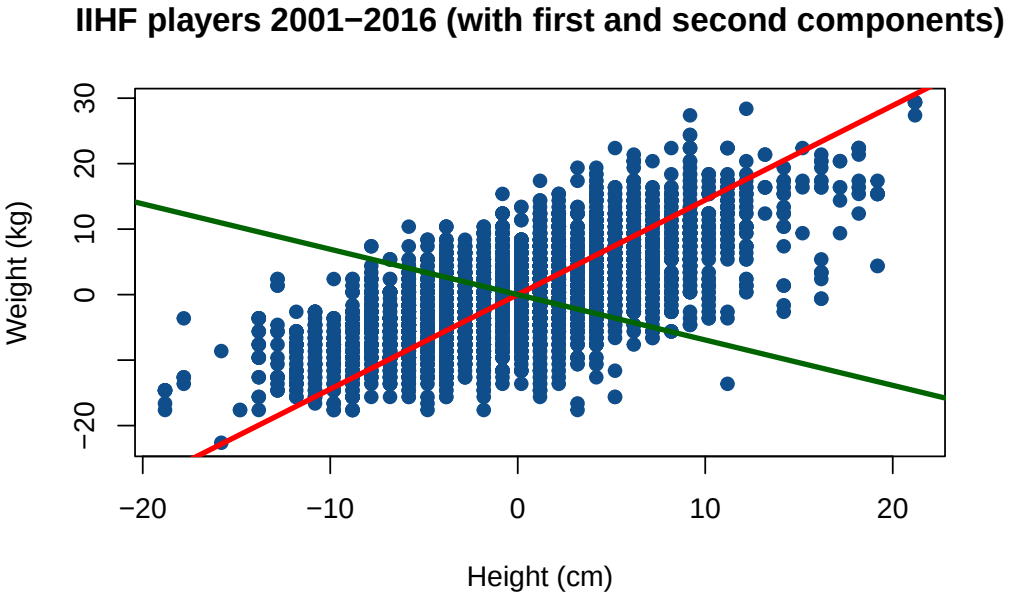
```
plot(data$height.c_r, data$weight.c_r,
    pch = 19, col = "dodgerblue4",
    xlab = "x-axis", ylab = "y-axis",
    main = "IIHF players 2001-2016 (rotated to first component)",
    ylim = range(data$weight.c))
abline(h = 0, col = "red", lwd = 2)
```


IIHF players 2001−2016 (rotated to first component)
y−axis
x−axis
30
20
10
0
−20
−30
−20
−10
0
10
20
30

First and second principal components

Plot the first and second eigenvectors

```
plot(data$height.c, data$weight.c,
    pch = 19, col = "dodgerblue4",
    main = "IIHF players 2001-2016 (with first and second components)",
    xlab = "Height (cm)", ylab = "Weight (kg)")
abline(a = 0, b = ev$vectors[2,1]/ev$vectors[1,1],
       col = "red", lwd = 3)
abline(a = 0, b = ev$vectors[2,2]/ev$vectors[1,2],
       col = "darkgreen", lwd = 3)
```


IIHF players 2001−2016 (with first and second components)
Weight (kg)
30
20
10
0
−20
−20
−10
0
10
20
Height (cm)

Proper change of basis

Let us change the basis so that, in the new basis, the first component is the x-axis and the second component is the y-axis

We want to use Theorem 82

We need the coordinates of the new basis in the canonical basis of $\mathbb{R}^2$

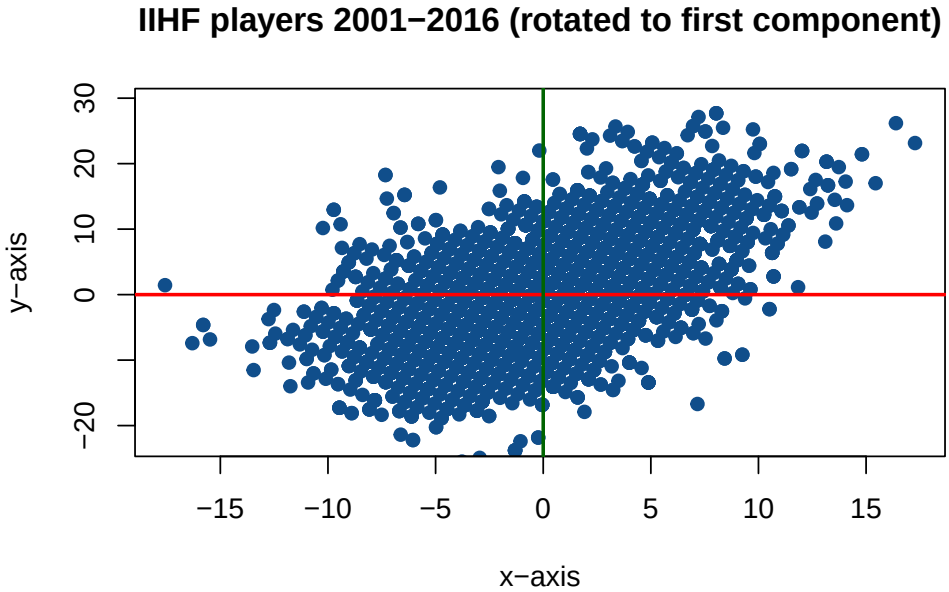
Since both axes go through the origin, we can just use $y = ax$, with a the slope of the lines and, say, $x = 1$, i.e., $(x, y) = (1, a)$

We then normalise the resulting vectors

Proper change of basis

```
red_line = c(1, ev$vectors[2,1]/ev$vectors[1,1])
red_line = red_line/sqrt(sum(red_line^2))
green_line = c(1, ev$vectors[2,2]/ev$vectors[1,2])
green_line = green_line/sqrt(sum(green_line^2))
augmented_M = cbind(red_line,green_line, diag(2))
P = rref(augmented_M)[,3:4]

tmp_in = matrix(c(data$weight.c, data$height.c), nc = 2)
tmp_out = c()
for (i in 1:dim(tmp_in)[1]) {
    tmp_out = rbind(tmp_out, t(P %*% tmp_in[i,]))
}
data$weight.c_r2 = tmp_out[,1]
data$height.c_r2 = tmp_out[,2]
```


IIHF players 2001−2016 (rotated to first component)
y−axis
30
20
10
0
−20
−15
−10
−5
0
5
10
15
x−axis

PCA using built-in functions

Now do things “properly”

```
GS = pracma::gramSchmidt(A = ev$vectors, tol = 1e-10)
GS

## $Q
##            [,1]       [,2]
## [1,] 0.5694576 -0.8220207
## [2,] 0.8220207  0.5694576
##
## $R
##      [,1]          [,2]
## [1,]    1 -5.551115e-17
## [2,]    0  1.000000e+00
```

PCA using built-in functions

Now recall we saw a theorem that told us how to construct a new basis..

```
A=matrix(c(GS$Q,1,0,0,1), nr = 2)
A
```

```
##           [,1]       [,2] [,3] [,4]
## [1,] 0.5694576 -0.8220207    1    0
## [2,] 0.8220207  0.5694576    0    1
```

```
pracma::rref(A)
```

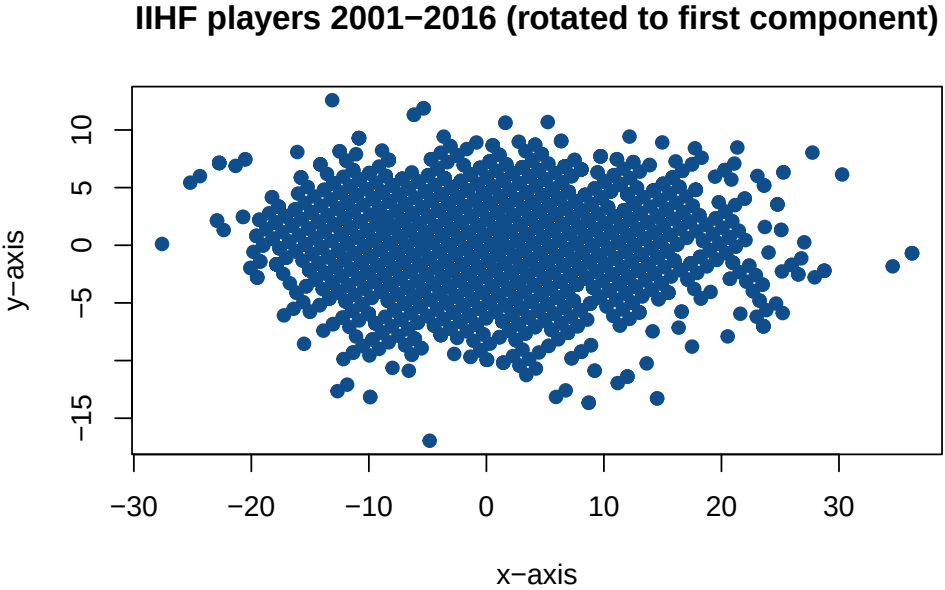
```
##      [,1] [,2]       [,3]      [,4]
## [1,]    1    0  0.5694576 0.8220207
## [2,]    0    1 -0.8220207 0.5694576
```

PCA using built-in functions

```
P = pracma::rref(A)[,c(3,4)]

##             [,1]      [,2]
## [1,]  0.5694576 0.8220207
## [2,] -0.8220207 0.5694576

X.new = X %*% t(P)
```


IIHF players 2001−2016 (rotated to first component)
y−axis
10
5
0
−5
−15
−30
−20
−10
0
10
20
30
x−axis

Hyperplanes, SVM and least-squares classification

MATH 2740 – Mathematics of Data Science – Unit U06

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Unsupervised vs supervised linear methods

In Unit 05 (PCA), we studied **unsupervised** dimensionality reduction:

- Given unlabelled feature vectors $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^d$
- Find orthogonal directions that maximise total sample variance

Here, we study **supervised binary classification**:

- Given a labelled training dataset of n observations

$$(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)$$

  where $\boldsymbol{x}_i \in \mathbb{R}^d$ is a feature vector and $y_i \in \{-1, +1\}$ (or $\{1, 2\}$) indicates the class to which $\boldsymbol{x}_i$ belongs
- Find a linear decision boundary or 1D projection that separates the two classes and predicts the label $\hat{y}(\boldsymbol{x})$ of any new point $\boldsymbol{x} \in \mathbb{R}^d$

Does maximum variance always separate classes?

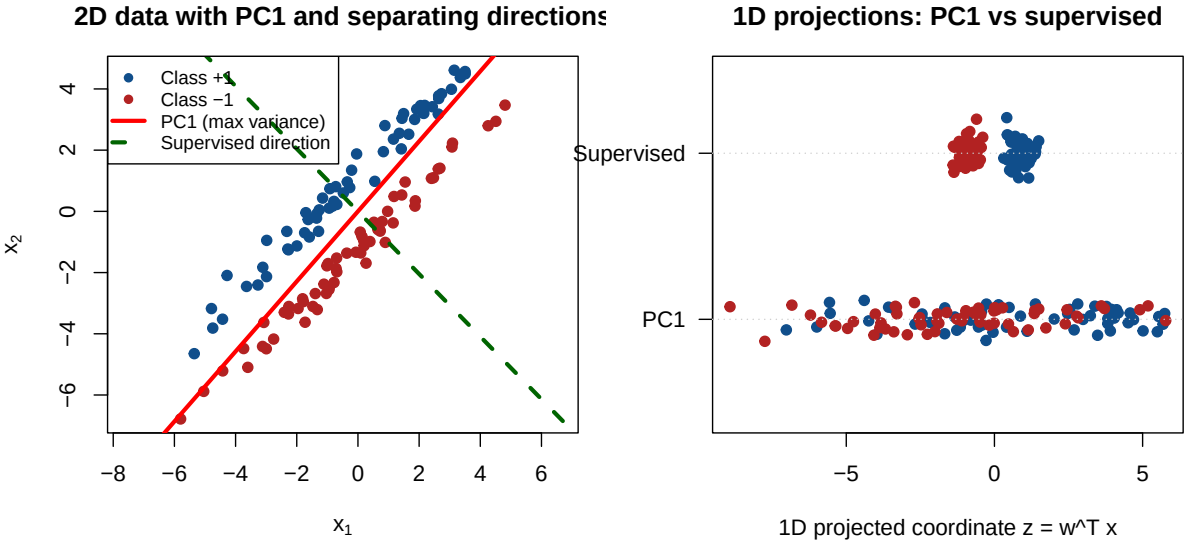
Suppose two classes form elongated, parallel point clouds that are closely spaced along their perpendicular direction

- Because PCA ignores the class labels y_i, it pools all n points together and finds the direction of largest overall spread
- When the spread *along* the clouds is larger than the distance *between* the clouds, the first principal component (PC1) runs parallel to the clouds
- Projecting onto PC1 mixes the two classes completely, whereas projecting onto the perpendicular direction separates them cleanly

When PCA fails to separate classes

Simulate two elongated parallel classes in $\mathbb{R}^2$ and compare PC1 (red) with the supervised separating direction (green)

```
set.seed(2740)
n_c = 60
t1 = rnorm(n_c, 0, 2.5); t2 = rnorm(n_c, 0, 2.5)
X1 = cbind(t1 + rnorm(n_c, 0, 0.25), t1 + 1.2 + rnorm(n_c, 0, 0.25))
X2 = cbind(t2 + rnorm(n_c, 0, 0.25), t2 - 1.2 + rnorm(n_c, 0, 0.25))
X_all = rbind(X1, X2)
y_all = c(rep(1, n_c), rep(-1, n_c))
pc1 = eigen(cov(X_all))$vectors[, 1]
SW = (n_c - 1) * (cov(X1) + cov(X2))
w_sep = solve(SW, colMeans(X1) - colMeans(X2))
w_sep = w_sep / sqrt(sum(w_sep^2))
```


2D data with PC1 and separating directions
Class +1
Class −1
PC1 (max variance)
Supervised direction
x2
x1
−8
−6
−4
−2
0
2
4
6
−6
−4
−2
0
2
4
1D projections: PC1 vs supervised
Supervised
PC1
−5
0
5
1D projected coordinate z = w^T x

Two linear classification frameworks in this unit

Both methods use a linear score $f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b$ to classify points, but choose $\boldsymbol{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ using different linear algebraic principles:

1. **Geometry of hyperplanes and support vector machines (SVM)**: characterise decision boundaries via normal vectors, orthogonal projections and maximum-margin separation

2. **Least-squares classification**: treat binary labels $y_i \in \{-1, +1\}$ as regression targets and solve the normal equations $(X^TX)\tilde{\boldsymbol{w}} = X^T\boldsymbol{y}$ from Unit 03

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Geometry of hyperplanes and support vector machines

Hyperplanes and normal vectors

Orthogonal distance to a hyperplane

Linear separability and hard-margin SVM

Hyperplanes in $\mathbb{R}^d$

Definition 88 (Hyperplane in $\mathbb{R}^d$)

*Let $\boldsymbol{w} \in \mathbb{R}^d \setminus \{\mathbf{0}\}$ be a non-zero vector and $b \in \mathbb{R}$ be a scalar. The **hyperplane** determined by $\boldsymbol{w}$ and b is the set of points*

$$\mathcal{H} = \left\{ \boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T \boldsymbol{x} + b = 0 \right\} = \left\{ \boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w} \bullet \boldsymbol{x} + b = 0 \right\}$$

*The vector $\boldsymbol{w}$ is called a **normal vector** to $\mathcal{H}$ and b is called the **offset** (or **bias**)*

- In $\mathbb{R}^2$, a hyperplane is a line $w_1 x_1 + w_2 x_2 + b = 0$
- In $\mathbb{R}^3$, a hyperplane is a plane $w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$
- Multiplying $\boldsymbol{w}$ and b by any non-zero constant $\alpha \neq 0$ leaves the set $\mathcal{H}$ unchanged

Linear subspace vs affine subspace

When is a hyperplane $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ a vector subspace of $\mathbb{R}^d$?

- A vector subspace must contain the zero vector $\mathbf{0}$, and $\mathbf{0} \in \mathcal{H} \iff \boldsymbol{w}^T\mathbf{0} + b = 0 \iff b = 0$
- When $b = 0$, the set $\mathcal{H}_0 = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T\boldsymbol{x} = 0\} = \{\boldsymbol{w}\}^\perp$ is the orthogonal complement of span($\boldsymbol{w}$), which is a $(d-1)$-dimensional vector subspace of $\mathbb{R}^d$
- When $b \neq 0$, $\mathcal{H}$ does not pass through the origin; it is an **affine subspace** obtained by translating $\mathcal{H}_0$ by any fixed point $\boldsymbol{x}^* \in \mathcal{H}$:

$$\mathcal{H} = \boldsymbol{x}^* + \mathcal{H}_0 = \left\{\boldsymbol{x}^* + \boldsymbol{u} : \boldsymbol{w}^T\boldsymbol{u} = 0\right\}$$

Example in $\mathbb{R}^2$ – Hyperplanes with $b = 0$ and $b \neq 0$

In $\mathbb{R}^2$, hyperplanes are lines orthogonal to $\boldsymbol{w} \neq \mathbf{0}$

- **Line through the origin ($b = 0$):**

  $$\mathcal{H}_0 = \left\{ \boldsymbol{x} \in \mathbb{R}^2 : \boldsymbol{w}^T \boldsymbol{x} = 0 \right\} = \{\boldsymbol{w}\}^\perp$$

  Since $\mathbf{0} \in \mathcal{H}_0$, it is a 1D **vector subspace**

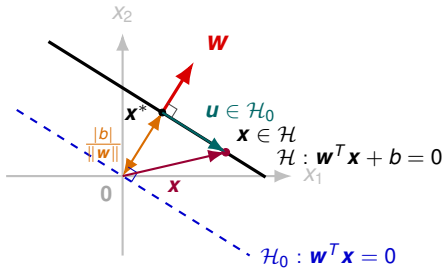
- **Line not through origin ($b \neq 0$):**

  $$\mathcal{H} = \left\{ \boldsymbol{x} \in \mathbb{R}^2 : \boldsymbol{w}^T \boldsymbol{x} + b = 0 \right\}$$

  $\mathbf{0} \notin \mathcal{H}$; **affine subspace** translated by $\boldsymbol{x}^* \in \mathcal{H}$:

  $$\mathcal{H} = \boldsymbol{x}^* + \mathcal{H}_0 = \{\boldsymbol{x}^* + \boldsymbol{u} : \boldsymbol{u} \in \mathcal{H}_0\}$$

- Every $\boldsymbol{x} \in \mathcal{H}$ satisfies $\boldsymbol{w}^T \boldsymbol{x} = -b$



Why $\boldsymbol{w}$ is orthogonal to the hyperplane

Proposition 89 (Orthogonality of the normal vector)

Let $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$. For any two points $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{H}$, the displacement vector $\boldsymbol{x}_1 - \boldsymbol{x}_2$ lying along $\mathcal{H}$ is orthogonal to $\boldsymbol{w}$:

$$\boldsymbol{w} \bullet (\boldsymbol{x}_1 - \boldsymbol{x}_2) = \boldsymbol{w}^T(\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$$

Proof:

- Since $\boldsymbol{x}_1 \in \mathcal{H}$ and $\boldsymbol{x}_2 \in \mathcal{H}$, both points satisfy the hyperplane equation:

$$\boldsymbol{w}^T\boldsymbol{x}_1 + b = 0 \qquad \text{and} \qquad \boldsymbol{w}^T\boldsymbol{x}_2 + b = 0$$

- Subtracting the second equation from the first eliminates b:

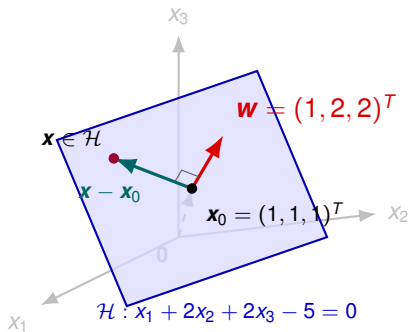
$$(\boldsymbol{w}^T\boldsymbol{x}_1 + b) - (\boldsymbol{w}^T\boldsymbol{x}_2 + b) = \boldsymbol{w}^T(\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$$

- Hence $\boldsymbol{w} \perp (\boldsymbol{x}_1 - \boldsymbol{x}_2)$ for every pair of points $\boldsymbol{x}_1, \boldsymbol{x}_2$ on $\mathcal{H}$ □

Point-normal form of a (hyper)plane

A hyperplane is determined by a **point** $\boldsymbol{x}_0 \in \mathcal{H}$ and a **normal vector** $\boldsymbol{w} \neq \boldsymbol{0}$

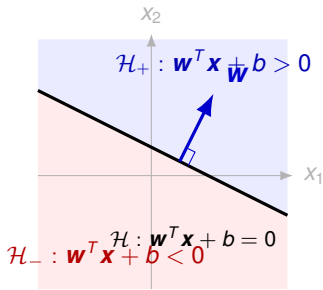
- For any $\boldsymbol{x} \in \mathcal{H}$, the displacement $\boldsymbol{x} - \boldsymbol{x}_0$ lies along $\mathcal{H}$, so $\boldsymbol{w} \perp (\boldsymbol{x} - \boldsymbol{x}_0)$:
  $\boldsymbol{w}^T(\boldsymbol{x} - \boldsymbol{x}_0) = 0$ (**point-normal form**)
- Expanding gives $\boldsymbol{w}^T\boldsymbol{x} - \boldsymbol{w}^T\boldsymbol{x}_0 = 0$, which is
  $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ with $b = -\boldsymbol{w}^T\boldsymbol{x}_0$
- **Example in $\mathbb{R}^3$:** Point $\boldsymbol{x}_0 = (1, 1, 1)^T$ and normal $\boldsymbol{w} = (1, 2, 2)^T$:
  $$1(x_1 - 1) + 2(x_2 - 1) + 2(x_3 - 1) = 0$$
  $$\iff x_1 + 2x_2 + 2x_3 - 5 = 0$$
  Here $b = -\boldsymbol{w}^T\boldsymbol{x}_0 = -(1 + 2 + 2) = -5$



Half-spaces and linear classifiers

A hyperplane $\mathcal{H}$ divides $\mathbb{R}^d \setminus \mathcal{H}$ into two open **half-spaces**:

$$\mathcal{H}_+ = \left\{ \boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T \boldsymbol{x} + b > 0 \right\}$$
$$\mathcal{H}_- = \left\{ \boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T \boldsymbol{x} + b < 0 \right\}$$



- The normal vector $\boldsymbol{w}$ points into the positive half-space $\mathcal{H}_+$
- The corresponding **linear classifier** assigns class labels via the sign function:

$$\hat{y}(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^T \boldsymbol{x} + b) = \begin{cases} +1 & \text{if } \boldsymbol{x} \in \mathcal{H}_+ \\ -1 & \text{if } \boldsymbol{x} \in \mathcal{H}_- \end{cases}$$

Example – Hyperplane geometry and classification in $\mathbb{R}^2$

Using the line $\mathcal{H}$ in $\mathbb{R}^2$ given by $2x_1 - x_2 - 3 = 0$, classify $\boldsymbol{p} = (3,1)^T$ and $\boldsymbol{q} = (0,0)^T$

- **Normal vector and offset:** $\boldsymbol{w} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$ and $b = -3$, with norm $\|\boldsymbol{w}\| = \sqrt{2^2 + (-1)^2} = \sqrt{5}$
- **Two points on $\mathcal{H}$:** setting $x_1 = 0$ gives $\boldsymbol{x}_1 = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$; setting $x_2 = 1$ gives $\boldsymbol{x}_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$
- **Orthogonality check:** $\boldsymbol{x}_2 - \boldsymbol{x}_1 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$ and $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2(2) + (-1)(4) = 0$
- **Classifying test points with $\hat{y}(\boldsymbol{x}) = \text{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b)$:**
  - For $\boldsymbol{p} = (3,1)^T$: $\boldsymbol{w}^T\boldsymbol{p} + b = 2(3) - 1(1) - 3 = +2 > 0 \implies \hat{y}(\boldsymbol{p}) = +1$ ($\boldsymbol{p} \in \mathcal{H}_+$)
  - For $\boldsymbol{q} = (0,0)^T$: $\boldsymbol{w}^T\boldsymbol{q} + b = 2(0) - 1(0) - 3 = -3 < 0 \implies \hat{y}(\boldsymbol{q}) = -1$ ($\boldsymbol{q} \in \mathcal{H}_-$)

Geometry of hyperplanes and support vector machines

Hyperplanes and normal vectors

Orthogonal distance to a hyperplane

Linear separability and hard-margin SVM

Orthogonal distance from a point to a hyperplane

Theorem 90 (Distance from a point to a hyperplane)

Let $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T\boldsymbol{x} + b = 0\}$ be a hyperplane with normal vector $\boldsymbol{w} \neq \boldsymbol{0}$, and let $\boldsymbol{x}_0 \in \mathbb{R}^d$ be any point. The Euclidean distance from $\boldsymbol{x}_0$ to $\mathcal{H}$ is

$$d(\boldsymbol{x}_0, \mathcal{H}) = \min_{\boldsymbol{x} \in \mathcal{H}} \|\boldsymbol{x}_0 - \boldsymbol{x}\| = \frac{|\boldsymbol{w}^T\boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$$

- Setting $\boldsymbol{x}_0 = \boldsymbol{0}$ gives the orthogonal distance from the origin to $\mathcal{H}$:

$$d(\boldsymbol{0}, \mathcal{H}) = \frac{|b|}{\|\boldsymbol{w}\|}$$

- The quantity $\frac{\boldsymbol{w}^T\boldsymbol{x}_0 + b}{\|\boldsymbol{w}\|}$ is the **signed distance** from $\boldsymbol{x}_0$ to $\mathcal{H}$ (positive on $\mathcal{H}_+$, negative on $\mathcal{H}_-$)

Proof – Distance from a point to a hyperplane

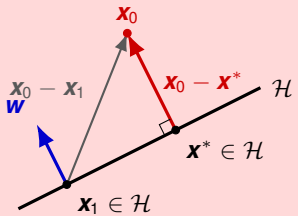
Let $\boldsymbol{x}^* \in \mathcal{H}$ be the orthogonal projection of $\boldsymbol{x}_0$ onto $\mathcal{H}$, and pick any $\boldsymbol{x}_1 \in \mathcal{H}$

- Since $\boldsymbol{x}_0 - \boldsymbol{x}^* \perp \mathcal{H}$, it equals the orthogonal projection of $\boldsymbol{x}_0 - \boldsymbol{x}_1$ onto $\boldsymbol{w}$:

$$\boldsymbol{x}_0 - \boldsymbol{x}^* = \text{proj}_{\boldsymbol{w}}(\boldsymbol{x}_0 - \boldsymbol{x}_1) = \frac{\boldsymbol{w}^T(\boldsymbol{x}_0 - \boldsymbol{x}_1)}{\|\boldsymbol{w}\|^2}\boldsymbol{w}$$

- Because $\boldsymbol{x}_1 \in \mathcal{H}$, we have $\boldsymbol{w}^T\boldsymbol{x}_1 = -b$, so $\boldsymbol{w}^T(\boldsymbol{x}_0 - \boldsymbol{x}_1) = \boldsymbol{w}^T\boldsymbol{x}_0 + b$
- Taking the Euclidean norm gives

$$d(\boldsymbol{x}_0, \mathcal{H}) = \|\boldsymbol{x}_0 - \boldsymbol{x}^*\| = \frac{|\boldsymbol{w}^T\boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$$



Example – Distance to a line in $\mathbb{R}^2$

Example 1 ($\mathbb{R}^2$): Find the distance from $\boldsymbol{x}_0 = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$ to the line $\mathcal{H} : 3x_1 - 4x_2 + 2 = 0$ and the closest point $\boldsymbol{x}^* \in \mathcal{H}$

- Here $\boldsymbol{w} = (3, -4)^T$ and $b = 2$, so $\|\boldsymbol{w}\| = \sqrt{3^2 + (-4)^2} = \sqrt{25} = 5$
- Signed numerator: $\boldsymbol{w}^T \boldsymbol{x}_0 + b = 3(4) - 4(3) + 2 = 2 > 0$ ($\boldsymbol{x}_0$ lies in $\mathcal{H}_+$)
- Distance: $d(\boldsymbol{x}_0, \mathcal{H}) = \frac{|2|}{5} = \frac{2}{5} = 0.4$, and closest point:

$$\boldsymbol{x}^* = \boldsymbol{x}_0 - \frac{\boldsymbol{w}^T \boldsymbol{x}_0 + b}{\|\boldsymbol{w}\|^2} \boldsymbol{w} = \begin{pmatrix} 4 \\ 3 \end{pmatrix} - \frac{2}{25} \begin{pmatrix} 3 \\ -4 \end{pmatrix} = \begin{pmatrix} 94/25 \\ 83/25 \end{pmatrix}$$

Example – Distance to a plane in $\mathbb{R}^3$

Example 2 ($\mathbb{R}^3$): Find the distance from $\boldsymbol{x}_0 = (1, -2, 3)^T$ to the plane $\mathcal{H} : x_1 + 2x_2 + 2x_3 - 5 = 0$

- Here $\boldsymbol{w} = (1, 2, 2)^T$, $b = -5$ and $\|\boldsymbol{w}\| = \sqrt{1^2 + 2^2 + 2^2} = 3$, so

$$d(\boldsymbol{x}_0, \mathcal{H}) = \frac{|1(1) + 2(-2) + 2(3) - 5|}{3} = \frac{|-2|}{3} = \frac{2}{3}$$

Geometry of hyperplanes and support vector machines

- Hyperplanes and normal vectors
- Orthogonal distance to a hyperplane
- **Linear separability and hard-margin SVM**

Linearly separable point sets

Definition 91 (Linear separability)

Two sets of points $X_+ = \{\boldsymbol{x}_i : y_i = +1\}$ *and* $X_- = \{\boldsymbol{x}_i : y_i = -1\}$ *in* $\mathbb{R}^d$ *are* ***linearly separable*** *if there exist* $\boldsymbol{w} \in \mathbb{R}^d \setminus \{\mathbf{0}\}$ *and* $b \in \mathbb{R}$ *such that*

$$\boldsymbol{w}^T \boldsymbol{x}_i + b > 0 \quad \text{for all } \boldsymbol{x}_i \in X_+ \qquad \text{and} \qquad \boldsymbol{w}^T \boldsymbol{x}_i + b < 0 \quad \text{for all } \boldsymbol{x}_i \in X_-$$

- Since $y_i \in \{-1, +1\}$, both inequalities can be written as a single condition:

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 0 \quad \text{for all } i = 1, \ldots, n$$

- Whenever a finite dataset is linearly separable, there are infinitely many separating hyperplanes – how should we pick the most robust one?

When are point sets not linearly separable? The XOR example

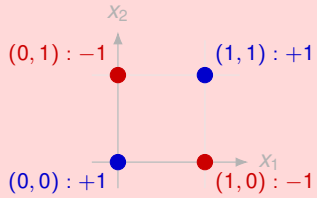
Not every two-class dataset in $\mathbb{R}^2$ is linearly separable

Example (XOR configuration):

- Let $X_+ = \{(0,0)^T, (1,1)^T\}$ $(y = +1)$ and $X_- = \{(1,0)^T, (0,1)^T\}$ $(y = -1)$
- Suppose $\boldsymbol{w}^T\boldsymbol{x} + b > 0$ on X_+ and $\boldsymbol{w}^T\boldsymbol{x} + b < 0$ on X_-:

$$\begin{aligned} \text{on } X_+ : &\quad b > 0 \text{ and } w_1 + w_2 + b > 0 \\ \text{on } X_- : &\quad w_1 + b < 0 \text{ and } w_2 + b < 0 \end{aligned}$$

- Adding the first pair gives $w_1 + w_2 + 2b > 0$, whereas adding the second pair gives $w_1 + w_2 + 2b < 0$ – contradiction!



Support vectors and canonical margin hyperplanes

Suppose $(\boldsymbol{w}, b)$ separates a finite dataset, so $\min_{1 \le i \le n} y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = m > 0$

Dividing both $\boldsymbol{w}$ and b by m does not change the decision boundary $\mathcal{H} = \{\boldsymbol{x} : \boldsymbol{w}^T \boldsymbol{x} + b = 0\}$, and scales the minimum functional margin to 1:

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) \ge 1 \quad \text{for all } i = 1, \ldots, n$$

In this **canonical form**, the slab free of training points is bounded by two parallel hyperplanes:

- $\mathcal{H}_{+1} = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T \boldsymbol{x} + b = +1\}$ (positive margin boundary)
- $\mathcal{H}_{-1} = \{\boldsymbol{x} \in \mathbb{R}^d : \boldsymbol{w}^T \boldsymbol{x} + b = -1\}$ (negative margin boundary)

The training points $\boldsymbol{x}_i$ that lie directly on $\mathcal{H}_{+1}$ or $\mathcal{H}_{-1}$ (satisfying $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$) are called **support vectors**

Geometric derivation of the SVM margin width

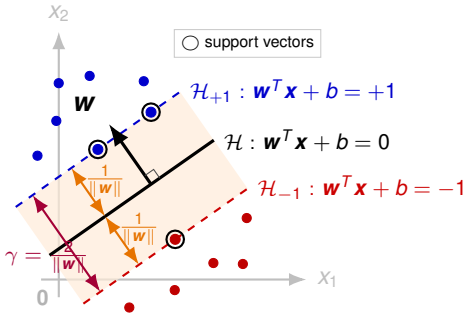
Using Theorem 90, how wide is the margin between $\mathcal{H}_{+1}$ and $\mathcal{H}_{-1}$?

- For any point $\boldsymbol{x}_+ \in \mathcal{H}_{+1}$, we have $\boldsymbol{w}^T\boldsymbol{x}_+ + b = +1$, so its distance to $\mathcal{H}$ is

$$d(\boldsymbol{x}_+, \mathcal{H}) = \frac{|\boldsymbol{w}^T\boldsymbol{x}_+ + b|}{\|\boldsymbol{w}\|} = \frac{1}{\|\boldsymbol{w}\|}$$

- For any point $\boldsymbol{x}_- \in \mathcal{H}_{-1}$, we have $\boldsymbol{w}^T\boldsymbol{x}_- + b = -1$, so its distance to $\mathcal{H}$ is also $\frac{1}{\|\boldsymbol{w}\|}$
- Therefore, the total **margin width** between $\mathcal{H}_{+1}$ and $\mathcal{H}_{-1}$ is

$$\gamma = \frac{2}{\|\boldsymbol{w}\|}$$



Hard-margin support vector machine (SVM)

To make the separating slab as wide as possible, we want to maximise $\gamma = \frac{2}{\|\boldsymbol{w}\|}$, which is equivalent to minimising $\|\boldsymbol{w}\|$ (or $\frac{1}{2}\|\boldsymbol{w}\|^2$):

$$\min_{\boldsymbol{w}\in\mathbb{R}^d,\, b\in\mathbb{R}} \frac{1}{2}\|\boldsymbol{w}\|^2 \quad \text{subject to} \quad y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 \quad \text{for } i = 1, \ldots, n$$

- This is the **hard-margin support vector machine (SVM)** optimisation problem
- The optimal hyperplane is completely determined by the support vectors lying on the margin boundaries $\boldsymbol{w}^T\boldsymbol{x}_i + b = \pm 1$
- For small problems in $\mathbb{R}^2$, we can find $(\boldsymbol{w}, b)$ directly by **geometry and linear equations**, without any Lagrange multiplier machinery

Example 1 – Hard-margin SVM for 2 points in $\mathbb{R}^2$

Problem: Find the canonical maximum-margin hyperplane $(\boldsymbol{w}, b)$, decision boundary and margin width γ for

$$\boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (y_1 = -1) \qquad \text{and} \qquad \boldsymbol{x}_2 = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \quad (y_2 = +1)$$

Step 1 (Geometry):

- With one point in each class, both $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are support vectors, and the decision boundary $\mathcal{H}$ is the perpendicular bisector of the segment $\boldsymbol{x}_1\boldsymbol{x}_2$
- The normal vector $\boldsymbol{w}$ must point from $\boldsymbol{x}_1$ ($y_1 = -1$) towards $\boldsymbol{x}_2$ ($y_2 = +1$), so $\boldsymbol{w}$ is a positive multiple of $\boldsymbol{x}_2 - \boldsymbol{x}_1$:

$$\boldsymbol{x}_2 - \boldsymbol{x}_1 = \begin{pmatrix} 3-1 \\ 5-1 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \quad \Longrightarrow \quad \boldsymbol{w} = \alpha \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad (\alpha > 0)$$

Example 1 – Scaling $(\boldsymbol{w}, b)$ and computing the margin

Step 2 (Canonical equations): Since $\boldsymbol{x}_1 \in \mathcal{H}_{-1}$ and $\boldsymbol{x}_2 \in \mathcal{H}_{+1}$, we require

$$\boldsymbol{w}^T \boldsymbol{x}_1 + b = -1 \qquad \text{and} \qquad \boldsymbol{w}^T \boldsymbol{x}_2 + b = +1$$

- Subtracting the first equation from the second gives $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2$:

$$\alpha \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 2 \iff 10\alpha = 2 \iff \alpha = \frac{1}{5} \quad \Longrightarrow \quad \boldsymbol{w} = \begin{pmatrix} 1/5 \\ 2/5 \end{pmatrix}$$

- Substituting $\boldsymbol{w}$ into $\boldsymbol{w}^T \boldsymbol{x}_1 + b = -1$ gives the offset b:

$$\frac{1}{5}(1) + \frac{2}{5}(1) + b = -1 \quad \Longrightarrow \quad b = -1 - \frac{3}{5} = -\frac{8}{5}$$

Step 3 (Decision boundary and margin width):

- Decision boundary: $\frac{1}{5}x_1 + \frac{2}{5}x_2 - \frac{8}{5} = 0 \iff x_1 + 2x_2 - 8 = 0$ (passes through the midpoint $\boldsymbol{m} = (2, 3)^T$)
- Margin width: $\|\boldsymbol{w}\| = \sqrt{(1/5)^2 + (2/5)^2} = \frac{1}{\sqrt{5}}$, so $\gamma = \frac{2}{\|\boldsymbol{w}\|} = 2\sqrt{5} = \|\boldsymbol{x}_2 - \boldsymbol{x}_1\|$

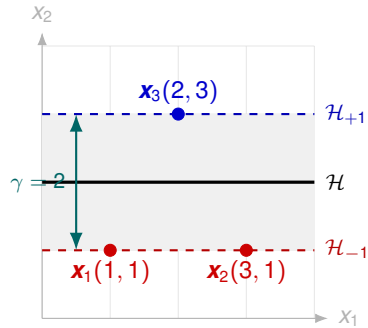
Example 2 – Hard-margin SVM for 3 points in $\mathbb{R}^2$

Problem: Find $(\boldsymbol{w}, b)$ and margin γ for:

- Class -1: $\boldsymbol{x}_1 = (1,1)^T$ and $\boldsymbol{x}_2 = (3,1)^T$
- Class $+1$: $\boldsymbol{x}_3 = (2,3)^T$

Step 1 (Direction of the boundary):

- Both $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ belong to class -1 and are equidistant from $\boldsymbol{x}_3$, so all three points are support vectors
- Since $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{H}_{-1}$, the vector $\boldsymbol{x}_2 - \boldsymbol{x}_1 = (2,0)^T$ lies along $\mathcal{H}_{-1}$
- By orthogonality, $\boldsymbol{w}^T(\boldsymbol{x}_2 - \boldsymbol{x}_1) = 2w_1 = 0 \implies w_1 = 0$, so $\boldsymbol{w} = (0, w_2)^T$



Example 2 – Solving the linear equations

Step 2 (Canonical equations): Substituting $\boldsymbol{w} = (0, w_2)^T$ into $\boldsymbol{w}^T \boldsymbol{x}_i + b = y_i$ for the support vectors gives

$$\begin{aligned} \text{at } \boldsymbol{x}_1 = (1,1)^T \text{ and } \boldsymbol{x}_2 = (3,1)^T: \quad & w_2(1) + b = -1 \\ \text{at } \boldsymbol{x}_3 = (2,3)^T: \quad & w_2(3) + b = +1 \end{aligned}$$

- Subtracting the first equation from the second gives $2w_2 = 2 \implies w_2 = 1$
- Substituting $w_2 = 1$ into $w_2 + b = -1$ gives $b = -2$, so

$$\boldsymbol{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \qquad \text{and} \qquad b = -2$$

- Decision boundary: $x_2 - 2 = 0$ (the horizontal line $x_2 = 2$)
- Margin width: $\|\boldsymbol{w}\| = 1 \implies \gamma = \frac{2}{\|\boldsymbol{w}\|} = 2$

What if we add non-support-vector points?

Suppose we add two more points to Example 2:

$$\boldsymbol{x}_4 = \begin{pmatrix} 2 \\ 5 \end{pmatrix} \quad (y_4 = +1) \qquad \text{and} \qquad \boldsymbol{x}_5 = \begin{pmatrix} 2 \\ -1 \end{pmatrix} \quad (y_5 = -1)$$

- Check the functional margin $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = y_i(x_{i2} - 2)$ at the new points:

$$y_4(\boldsymbol{w}^T \boldsymbol{x}_4 + b) = (+1)(5 - 2) = 3 > 1$$
$$y_5(\boldsymbol{w}^T \boldsymbol{x}_5 + b) = (-1)(-1 - 2) = 3 > 1$$

- Because both points lie strictly outside the margin slab (with $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 1$), they are **not support vectors**
- Moving or deleting $\boldsymbol{x}_4$ and $\boldsymbol{x}_5$ (as long as they stay outside the margin) does **not** change $(\boldsymbol{w}, b)$ at all!

Supervised vs unsupervised linear methods

Geometry of hyperplanes and support vector machines

Least-squares classification

Framing classification as linear least squares

Can we fit a linear classifier $\hat{y}(\boldsymbol{x}) = \operatorname{sgn}(\boldsymbol{w}^T\boldsymbol{x} + b)$ using the method of least squares from Unit 03?

We ask the affine score $f(\boldsymbol{x}_i) = \boldsymbol{w}^T\boldsymbol{x}_i + b$ to match the numeric class label $y_i \in \{-1, +1\}$ as closely as possible:

$$\boldsymbol{w}^T\boldsymbol{x}_i + b \approx y_i \quad \text{for } i = 1, \ldots, n$$

To write this as a single matrix product, absorb the intercept b by defining the **augmented feature vector** and **augmented weight vector**:

$$\tilde{\boldsymbol{x}}_i = \begin{pmatrix} \boldsymbol{x}_i \\ 1 \end{pmatrix} \in \mathbb{R}^{d+1}, \qquad \tilde{\boldsymbol{w}} = \begin{pmatrix} \boldsymbol{w} \\ b \end{pmatrix} \in \mathbb{R}^{d+1}$$

so that $\boldsymbol{w}^T\boldsymbol{x}_i + b = \tilde{\boldsymbol{x}}_i^T\tilde{\boldsymbol{w}}$

Matrix formulation and normal equations

Stack the augmented rows $\tilde{\boldsymbol{x}}_i^T$ into the design matrix $X \in \mathcal{M}_{n,d+1}$ and the labels into $\boldsymbol{y} \in \{-1, +1\}^n$:

$$X = \begin{pmatrix} \boldsymbol{x}_1^T & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_n^T & 1 \end{pmatrix} \in \mathcal{M}_{n,d+1}, \qquad \boldsymbol{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n$$

Minimising the sum of squared errors $\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2 = \sum_{i=1}^n (\boldsymbol{w}^T \boldsymbol{x}_i + b - y_i)^2$ yields the **normal equations**:

$$X^T X \tilde{\boldsymbol{w}} = X^T \boldsymbol{y} \quad \Longrightarrow \quad \tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y}$$

- What we are explaining here **only works when we have more observations than features**: we require $n \geq d + 1$ and $\text{rank}(X) = d + 1$ for $X^T X \in \mathcal{M}_{d+1}$ to be invertible
- If $n < d + 1$ or if features are collinear, $X^T X$ is singular and $(X^T X)^{-1}$ does not exist

Example – 4-point least-squares classifier in $\mathbb{R}^2$

Problem: Find the least-squares linear classifier $(\boldsymbol{w}, b)$ for the $n = 4$ points:

- Class -1 ($y_1 = y_2 = -1$): $\boldsymbol{x}_1 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$ and $\boldsymbol{x}_2 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$
- Class $+1$ ($y_3 = y_4 = +1$): $\boldsymbol{x}_3 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\boldsymbol{x}_4 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

Step 1 (Set up X and $\boldsymbol{y}$):

$$X = \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \in \mathcal{M}_{4,3}, \qquad \boldsymbol{y} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \in \mathbb{R}^4$$

Example – Solving the normal equations by hand

Step 2 (Compute $X^T X$ and $X^T \boldsymbol{y}$):

$$X^T X = \begin{pmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{pmatrix}, \qquad X^T \boldsymbol{y} = \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix}$$

Step 3 (Solve for $\tilde{\boldsymbol{w}}$): Since $X^T X = \mathrm{diag}(2, 2, 4)$ is diagonal, its inverse is immediate:

$$\tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y} = \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 1/4 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \implies \boldsymbol{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad b = 0$$

The decision boundary is $\boldsymbol{w}^T \boldsymbol{x} + b = x_1 + x_2 = 0$ (the line $x_2 = -x_1$), which separates the two classes cleanly

Example – Fitted values, residuals and classifying new points

With $\tilde{\boldsymbol{w}} = (1, 1, 0)^T$ ($\boldsymbol{w} = (1, 1)^T$ and $b = 0$), let us check the fitted scores $\hat{\boldsymbol{y}} = X\tilde{\boldsymbol{w}}$ and residual vector $\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$:

$$\hat{\boldsymbol{y}} = X\tilde{\boldsymbol{w}} = \begin{pmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix} = \boldsymbol{y} \quad \Longrightarrow \quad \boldsymbol{e} = \boldsymbol{y} - X\tilde{\boldsymbol{w}} = \boldsymbol{0}$$

- Because these 4 points happen to lie on the lines $x_1 + x_2 = -1$ and $x_1 + x_2 = +1$, the least-squares residual is zero!
- For a new observation $\boldsymbol{x}_{\text{new}} = \begin{pmatrix} 2 \\ -0.5 \end{pmatrix}$, we compute $\boldsymbol{w}^T\boldsymbol{x}_{\text{new}} + b = 2 - 0.5 = 1.5 > 0$, so $\hat{y}(\boldsymbol{x}_{\text{new}}) = \text{sgn}(1.5) = +1$

Sensitivity of least-squares classification to outliers

Why is least squares not always suitable for classification?

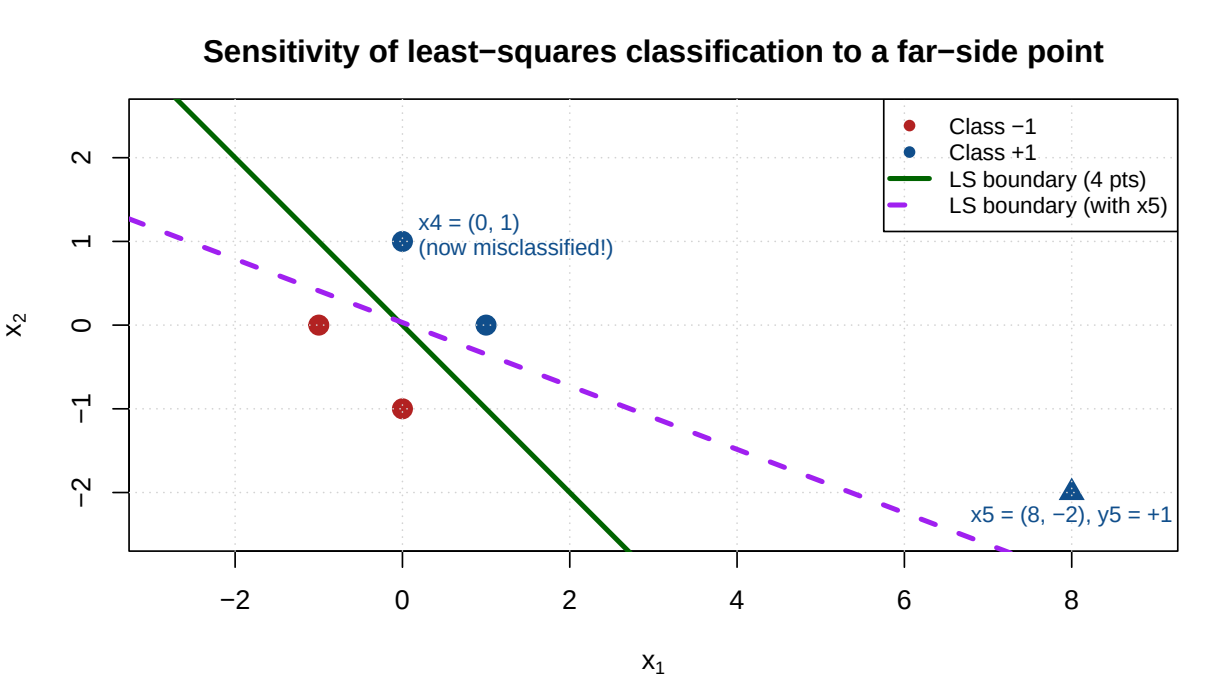
- The squared error $(\boldsymbol{w}^T \boldsymbol{x}_i + b - y_i)^2$ penalises predictions that are **too far on the correct side** of the decision boundary
- Suppose a point with $y_i = +1$ lies deep inside $\mathcal{H}_+$ so that $\boldsymbol{w}^T \boldsymbol{x}_i + b = 10 > 0$: it is *correctly classified*, yet least squares assigns a large squared penalty $(10 - 1)^2 = 81$
- To shrink that large residual, least squares tilts and shifts the hyperplane towards the extreme point, which can push the boundary across points near the margin and cause misclassifications

R: least-squares classifier and outlier sensitivity

Compare the 4-point least-squares boundary before and after adding a single "easy" Class $+1$ point at $\boldsymbol{x}_5 = (8, -2)^T$ (which lies well inside $x_1 + x_2 > 0$)

```
X4 = matrix(c(-1, 0, 1,  0, -1, 1,  1, 0, 1,  0, 1, 1),
            nrow = 4, byrow = TRUE)
y4 = c(-1, -1, 1, 1)
w_tilde_4 = solve(t(X4) %*% X4, t(X4) %*% y4)
# Add one outlier far on the correct (+1) side: x5 = (8, -2)^T
X5 = rbind(X4, c(8, -2, 1)); y5 = c(y4, 1)
w_tilde_5 = solve(t(X5) %*% X5, t(X5) %*% y5)
cbind(w_4pts = round(w_tilde_4, 3), w_5pts = round(w_tilde_5, 3))
```

```
##      [,1]   [,2]
## [1,]    1  0.433
## [2,]    1  1.142
## [3,]    0 -0.035
```


Sensitivity of least−squares classification to a far−side point
Class −1
Class +1
LS boundary (4 pts)
LS boundary (with x5)
x4 = (0, 1)
(now misclassified!)
x5 = (8, −2), y5 = +1
x1
x2
−2
0
2
4
6
8
−2
−1
0
1
2

Ridge regularisation when $X^T X$ is singular

What if we have more features than observations ($n < d + 1$) or collinear features?

- In that case, $\text{rank}(X) < d + 1$, so $X^T X$ is singular and cannot be inverted
- To fix this, we add a **ridge penalty** with regularisation parameter $\lambda > 0$:

$$\min_{\tilde{\boldsymbol{w}} \in \mathbb{R}^{d+1}} \left(\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2 + \lambda \|\tilde{\boldsymbol{w}}\|^2 \right)$$

- Setting the gradient to zero yields the regularised normal equations:

$$(X^T X + \lambda \mathbb{I})\tilde{\boldsymbol{w}} = X^T \boldsymbol{y} \quad \Longrightarrow \quad \tilde{\boldsymbol{w}}_{\text{ridge}} = (X^T X + \lambda \mathbb{I})^{-1} X^T \boldsymbol{y}$$

- Adding $\lambda \mathbb{I}$ shifts all eigenvalues of $X^T X$ by $+\lambda > 0$: all its eigenvalues are strictly positive (we call such a matrix **positive definite**; we will discuss this in more detail later), which guarantees that it is always invertible

Summary – Comparing linear projection and classification methods

Method	Supervised?	Criterion optimised	Linear algebraic solution
PCA (Unit 05)	No	Max variance $\boldsymbol{w}^T \Sigma \boldsymbol{w}$	Top eigenvector of Σ
Hard-margin SVM	Yes	Max margin $\frac{2}{\|\boldsymbol{w}\|}$	Support vectors $\boldsymbol{w}^T \boldsymbol{x}_i + b = \pm 1$
Least squares	Yes	Min error $\|X\tilde{\boldsymbol{w}} - \boldsymbol{y}\|^2$	$\tilde{\boldsymbol{w}} = (X^T X)^{-1} X^T \boldsymbol{y}$